

System Software Development 2024-25

Centre Number: 71955

Candidate Number: 2217

Contents

CONTENTS	2
BACKGROUND	5
USER REQUIREMENTS	6
FUNCTIONAL REQUIREMENTS	6
<i>General Requirements</i>	6
<i>In-Game Requirements</i>	7
NON-FUNCTIONAL REQUIREMENTS	8
INITIAL DESIGNS	9
FLOW CHART OF PROGRAM	9
HOMESCREEN	10
WELCOMESCREEN	11
CREATEACCOUNT	12
LOGIN	13
LOBBY	14
DIFFICULTYSELECT	15
SETTINGS	16
LEADERBOARD	17
QUESTIONDESIGNER	18
MULTICHOICE	22
CHECKBOX	23
TRUEORFALSE	24
FILLINTHEBLANK	25
DRAGANDDROP	26
ENDSCREEN	27
TEXTVIEWER	28
TUTORIAL	29
FINAL DESIGNS	30
HOMESCREEN	30
<i>Detailed Functionality</i>	32
WELCOMESCREEN	40
<i>Detailed Functionality</i>	42
CREATEACCOUNT	43
<i>Detailed Functionality</i>	49
LOGIN	51
<i>Detailed Functionality</i>	55
LOBBY	56
<i>Detailed Functionality</i>	60
DIFFICULTYSELECT	61
<i>Detailed Functionality</i>	64
SETTINGS	65
<i>Detailed Functionality</i>	71

LEADERBOARD	75
<i>Detailed Functionality</i>	79
QUESTIONDESIGNER	81
<i>Detailed Functionality</i>	94
MULTICHOICE	100
<i>Detailed Functionality</i>	105
CHECKBOX	109
<i>Detailed Functionality</i>	114
TRUEORFALSE	118
<i>Detailed Functionality</i>	121
FILLINTHEBLANK	124
<i>Detailed Functionality</i>	127
DRAGANDDROP	130
<i>Detailed Functionality</i>	140
ENDSCREEN	144
<i>Detailed Functionality</i>	149
TEXTVIEWER	151
<i>Detailed Functionality</i>	153
TUTORIAL	154
<i>Detailed Functionality</i>	156
REQUIREMENTS MET	157
TESTING	159
HOMESCREEN	159
WELCOMESCREEN	166
CREATEACCOUNT	167
LOGIN	175
LOBBY	179
DIFFICULTYSELECT	184
LEADERBOARD	188
SETTINGS	195
QUESTIONDESIGNER	213
MULTICHOICE	235
CHECKBOX	249
TRUEORFALSE	262
FILLINTHEBLANK	272
DRAGANDDROP	281
ENDSCREEN	293
TEXTVIEWER	301
TUTORIAL	306
FAILED TESTS	314
<i>CreateAccount</i>	314
<i>Login</i>	315
<i>Leaderboard</i>	316
<i>QuestionDesigner</i>	317
<i>TextViewer</i>	318

<i>EndScreen</i>	319
<i>TrueOrFalse</i>	323
<i>FillInTheBlank</i>	324
TESTING CONCLUSION	325
EVALUATION	326
PEER REVIEW QUESTIONS	326
EVALUATION OF FUNCTIONAL GENERAL REQUIREMENTS	326
EVALUATION OF FUNCTIONAL IN-GAME REQUIREMENTS	334
EVALUATION OF NON-FUNCTIONAL REQUIREMENTS	337
CONCLUSION	340
<i>Evaluation of my final product</i>	340
<i>Self-Evaluation</i>	342
<i>Final Thoughts</i>	343

Background

I was given the task to create an event driven object-oriented program in C#, I decide that I wanted to make a quiz about computers.

I've always had an interest in computers, and I think it is important for others to know the basics of computers and computing terminology. So, I will develop a quiz program that allows a user to sign into an account, asks them a series of questions, stores their score and statistics and display their score on a leaderboard. I want to make the quiz accessible to everyone, even if they aren't that knowledgeable about computers, but that still provides a challenge to those who know about computers by implementing a difficulty system that alters the score given to the user as a result. The users will also be able to add new questions to the quiz so the user can keep the quiz up to date with questions that reflect modern computing and so that the user can keep the quiz from becoming stale. There should also be a settings page that allows users to change the settings of the game along with a tutorial screen that tells users how to play the game.

User Requirements

Functional Requirements

General Requirements

My application will:

1. Use forms that are loaded onto a panel instead of pop-up windows to make navigation throughout the program smooth, with the option to go back and forth between panels.
2. The program will open with a welcome screen, that allows you to either go to the sign in screen or the create account screen.
3. The accounts must:
 - a. Have usernames no longer than 12 characters and no shorter than 4 characters.
 - b. Be stored in a secure way.
 - c. Not contain duplicate usernames.
 - d. Have strong passwords that are not easy to guess (mixed case, has at least one symbol & number and longer than 3 characters).
 - e. Usernames must be made up of only alpha-numeric characters.
 - f. Store statistics about the user's previous games, such as Lifetime correct & Lifetime incorrect answers.
 - g. Store the settings that user has enabled.
4. The account creation screen must:
 - a. Provide meaningful feedback to users as to why their username/password did not pass validation.
 - b. Allow users to unhide their password to double check their passwords spelling.
 - c. Make users type in their password again to confirm their password.
5. The user must be able to sign into their account, if they have one registered.
6. The login screen must not specify which inputted credential was wrong, only that one or both were wrong, to mitigate brute force account cracking attempts.
7. The Login screen must allow users to unhide their password to check its spelling.
8. Have a lobby where the user can choose to go to any of the main sections and which also displays statistics about their account.
9. The user must be able to add their own questions to the quiz to add replay value to the quiz.
10. The user must be able to change the difficulty of the quiz.

11. The user must be able to access the leaderboard from the lobby and at the end of the game.
12. The leaderboard must be sortable by each category stored in it, either by ascending order or descending order.
13. The user must be able to change the volume of the quiz or mute it.
14. The quiz will end with an optional end screen, which will have an option to take them to the leaderboard or return to the lobby.
15. The user must be able to sign out of their account and log in with a different account without closing the program
16. There must be an admin mode which allows the user to delete the file that contains the accounts and the file that contains the leaderboard for maintenance reasons, it will also show the user the answer to the current question if they are playing the quiz and allow the user to make custom end messages.
17. There must be a help centre to explain the features of the quiz.
18. The quiz must call the user by their name instead of using generic terms.
19. The quiz must give users meaningful error messages if an unrecoverable error occurs or in an error needs user intervention.
20. There should be a settings screen where the user can change settings, like whether or not to skip the end screen.
21. The user should be able to view editable text files in the program and delete lines from them.
22. The users name, score, lifetimecorrect answers and lifetime incorrect answers will be saved to the leaderboard.

In-Game Requirements

1. The quiz will play a sound to indicate as to whether or not the user answered the question correctly.
2. There must be special events that can happen during the quiz that will make the question harder to answer.
3. The quiz will contain 20 questions.
4. The sections of the quiz must be randomised, and no sections should show up twice in one playthrough, to make the quiz more interesting and add replay value.
5. The questions for the quiz must be randomised and never show up twice in one playthrough to make the quiz more interesting and replayable.
6. The user must be able to see their current score, question number and section of the quiz.

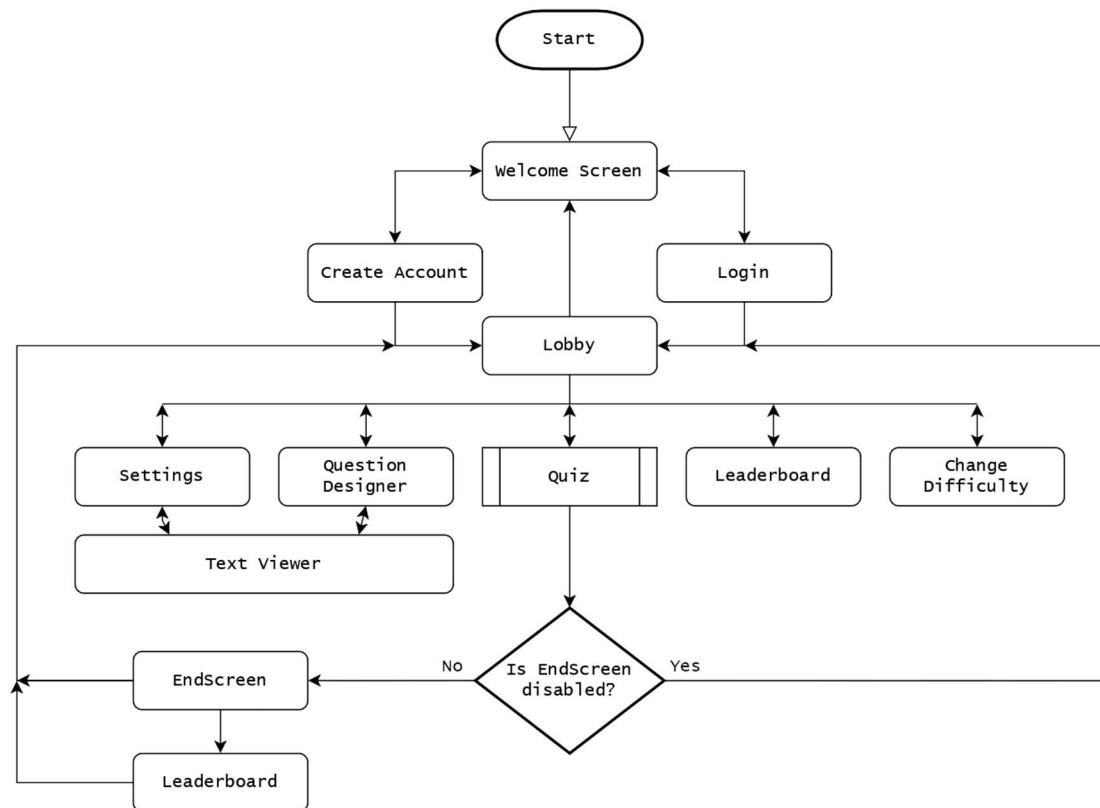
7. The user should hit a button to check their answer, instead of it being checked as soon as the click on an option.
8. Points gained should be proportionate to the user's difficulty.
9. The user must supply an answer to the question.

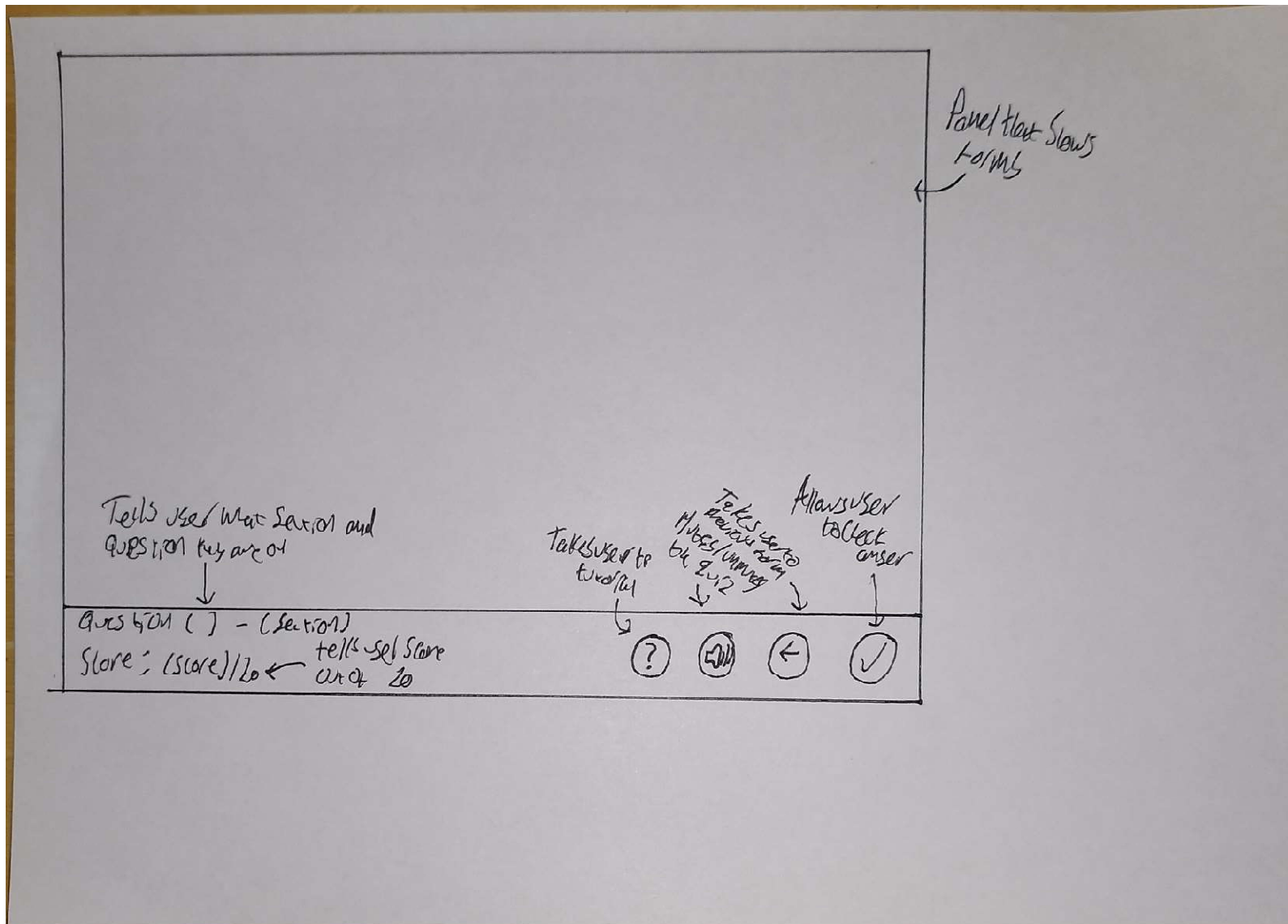
Non-Functional Requirements

1. The quiz must be fun and engaging to play.
2. The quiz's UI must be easy to understand and simple to use.
3. The rules of the quiz must be simple and not overly complex.
4. The forms for the game must have a common look and feel.
5. The Quiz must be robust.
6. The main section of the game must have accessibility features to allow for more people to play the quiz.

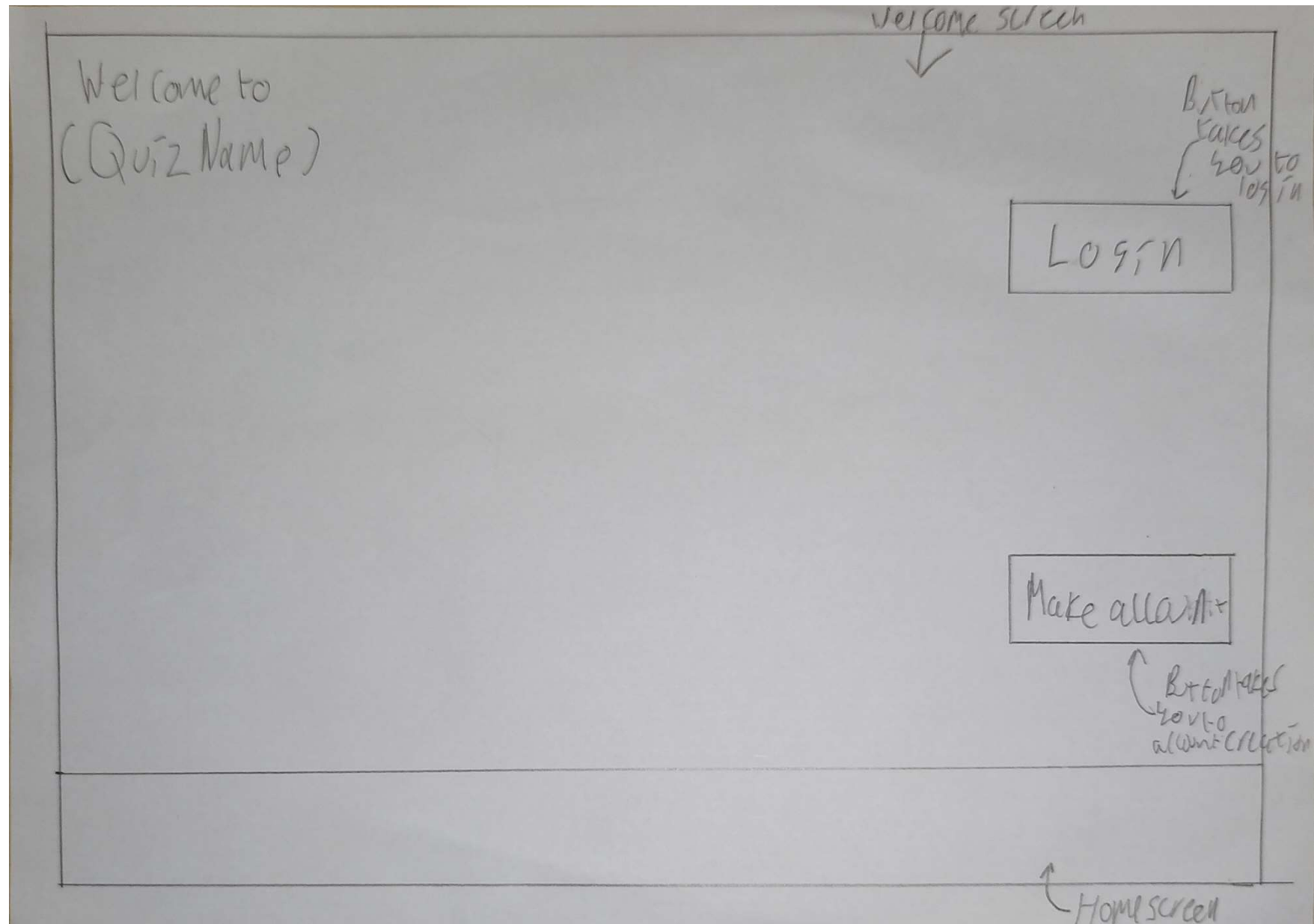
Initial Designs

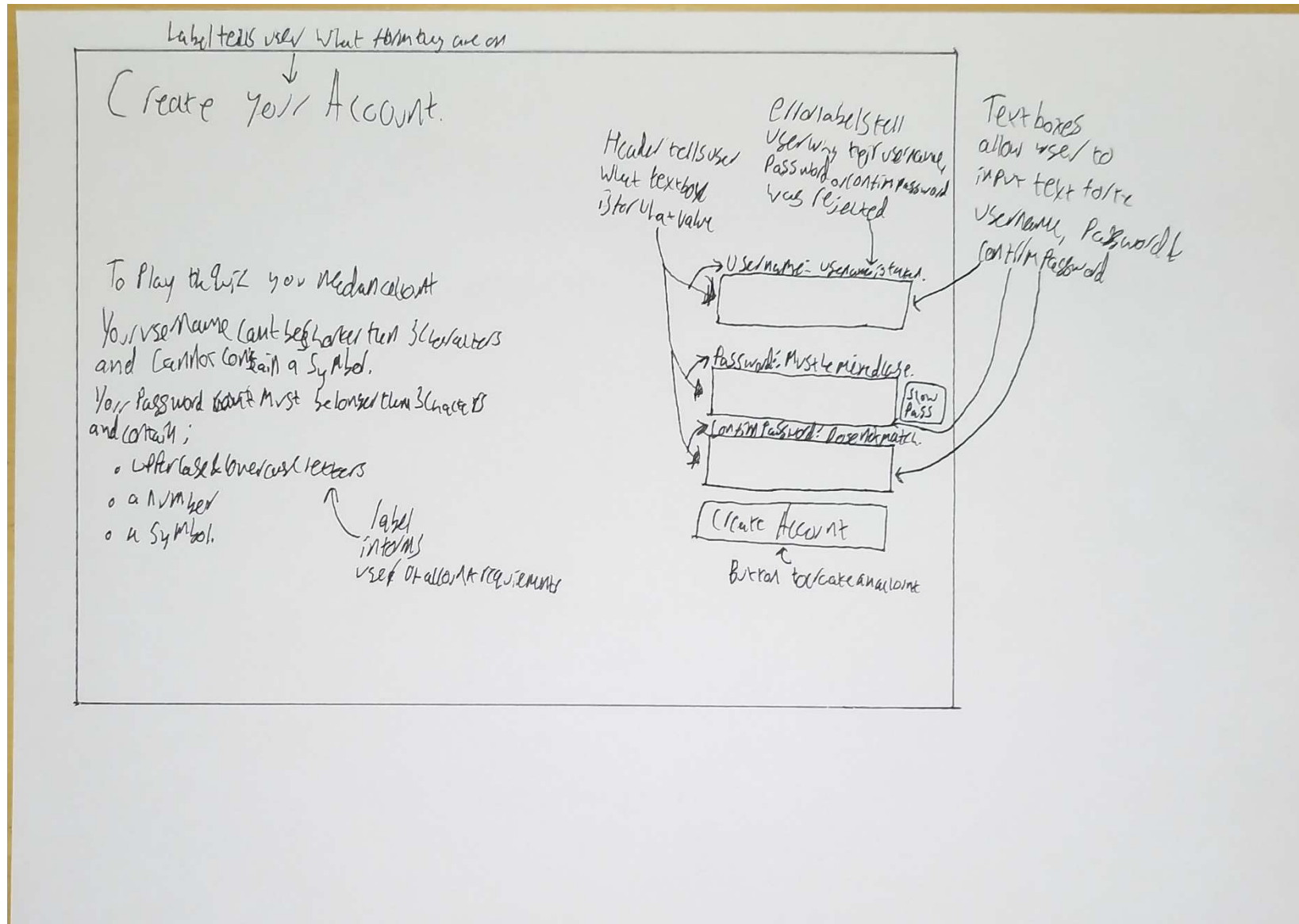
Flow Chart of Program

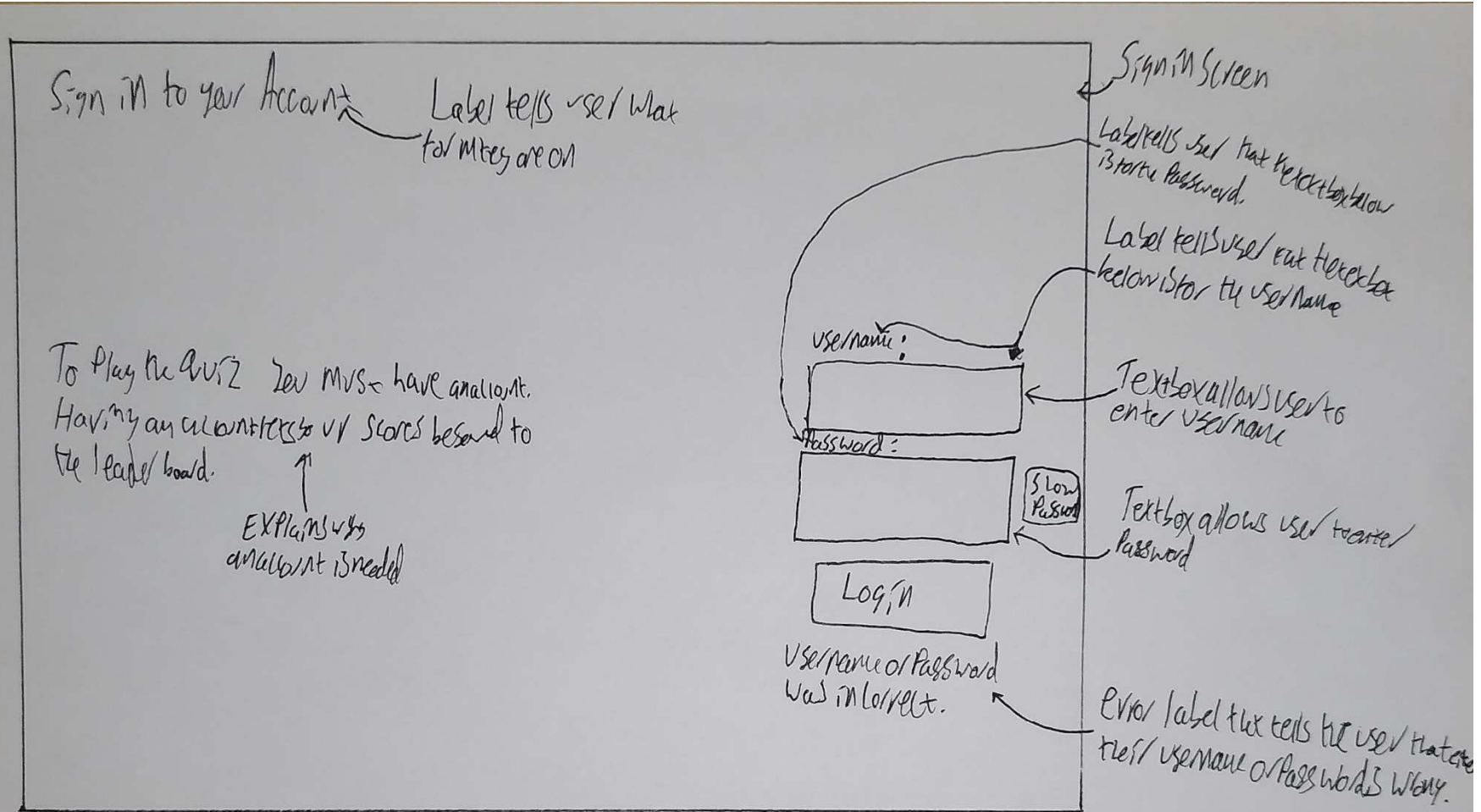


HomeScreen

WelcomeScreen



Create Account

Login

Lobby

Hello, {insert username here}

Lifetime correct: 3

Lifetime incorrect: 0

Checkbox: 3/0

Multiple Choice: 0/0

Filling the Blank: 0/0

True or False: 0/0

Label tells user their lifetime correct/incorrect, broken down by question category

Change Difficulty

Takes user to the Difficulty Selection form

Leaderboard

Takes the user to the leaderboard form

Play Quiz

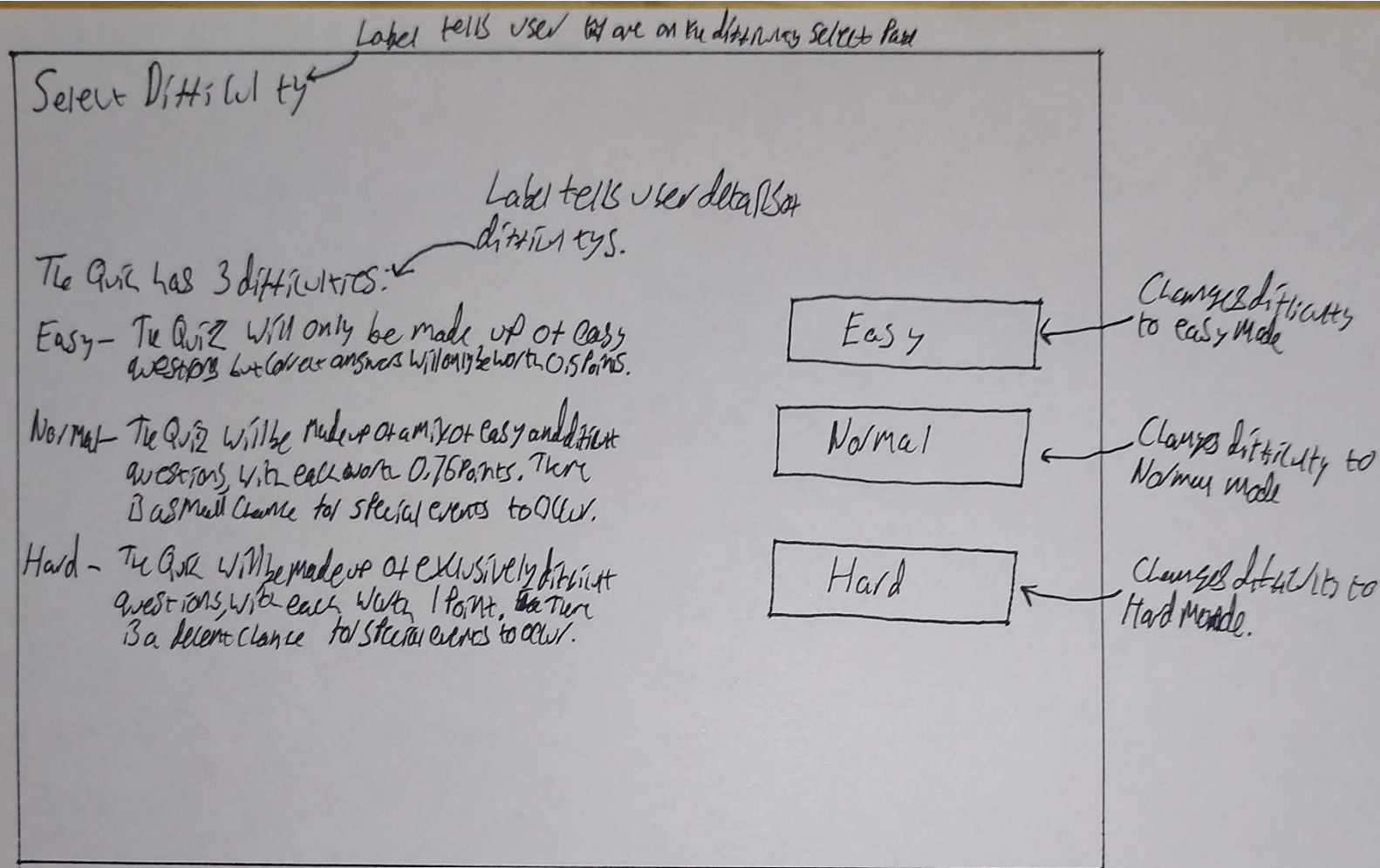
Starts the quiz.

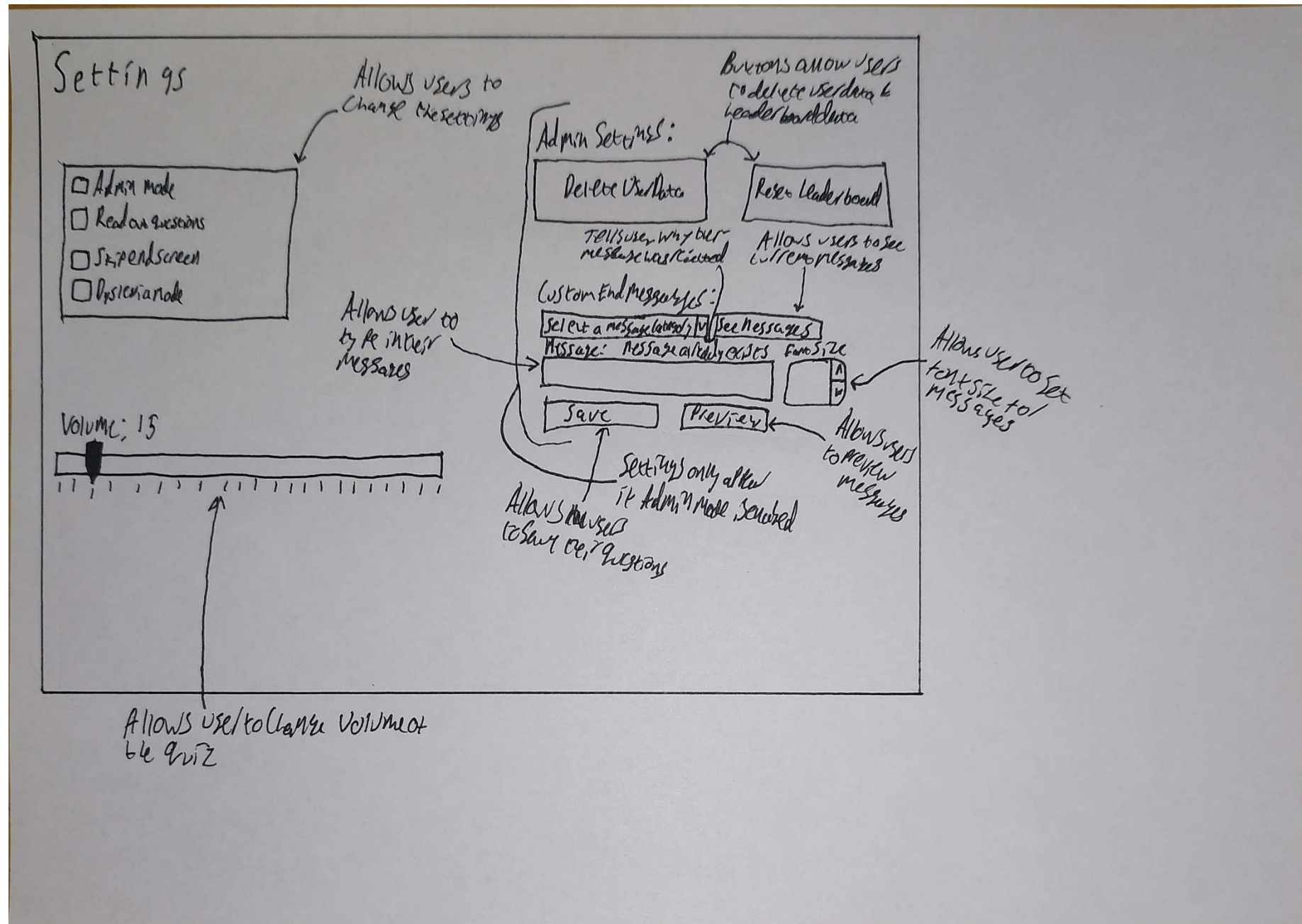
Question Designer

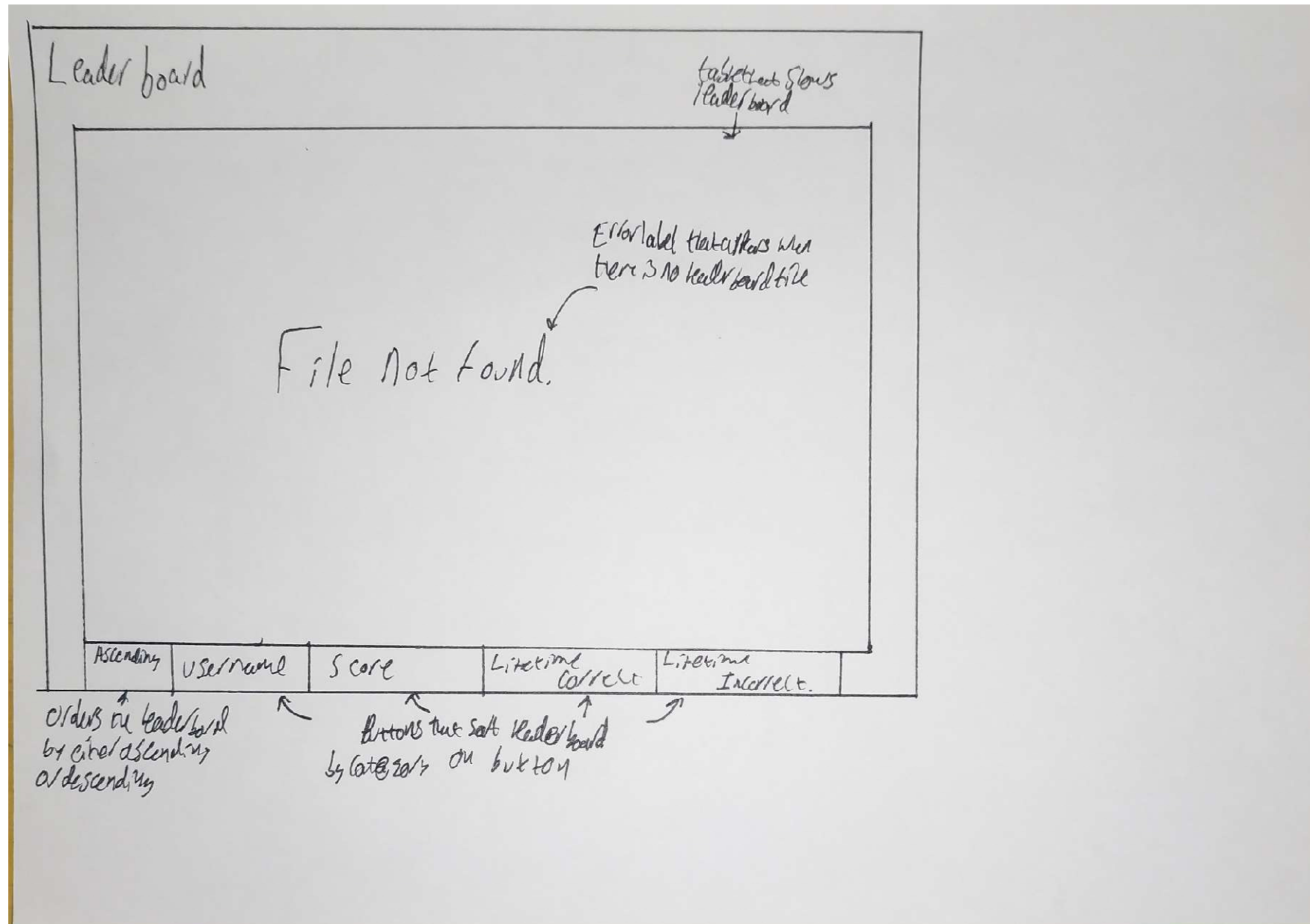
Takes the user to the Question Designer form

Settings

Takes the user to the Settings form

DifficultySelect

Settings

Leaderboard

Question Designer

Label needs user to be only question design form.

Question Designer

Multiple Choice

Save

Preview

Easy Question

Hard Question

Questions

Allows user to select binary questions in selected type

Allows user to type integer question and decorations for it

Question

1st Option

2nd Option

3rd Option

4th Option

Allows user to select font size of question label

0

0

0

0

Allows user to preview question and how it will look like

allows the user to pick the question is either easy or hard.

Allows user to select binary questions in selected type

Allows user to type integer question and decorations for it

Allows user to select font size of question label

Allows user to select which option is the answer

Question Designer

Check box ☐ See questions

Save Preview

Easy Question

Hard Question

Allows user to type in their question and the options for it

Question

1st Option

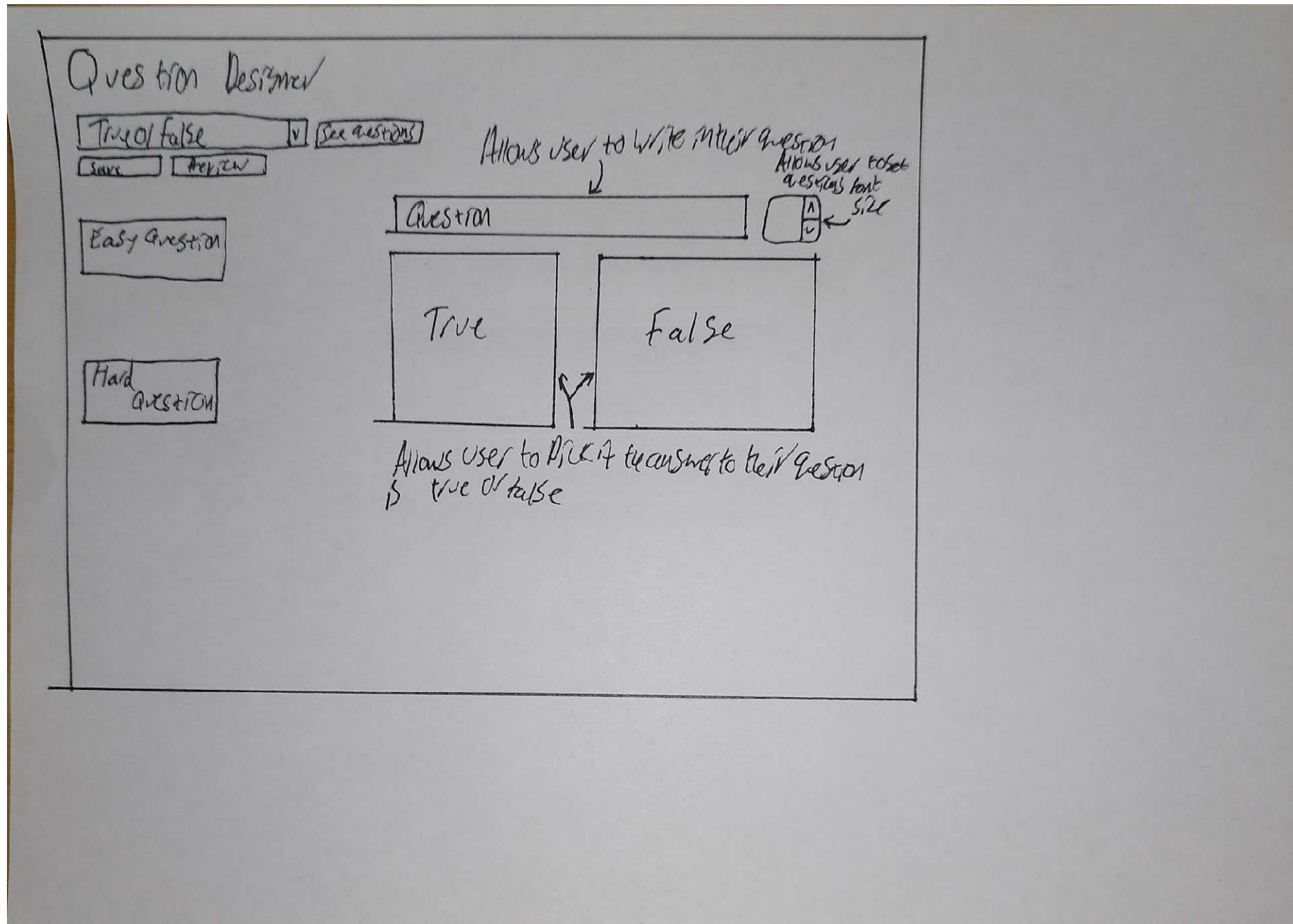
2nd Option

3rd Option

4th Option

Allows user to pick one or more of the options as the answer to the question

Allows user to set question font size



Question Designer

Fill in the Blank ☐ ☐ see questions

Save Preview

Easy Question

Hard Question

Allows user to write question and up to 4 possible answers for the question

Question

1st ~~Option~~ Answer ☐ Add Answer

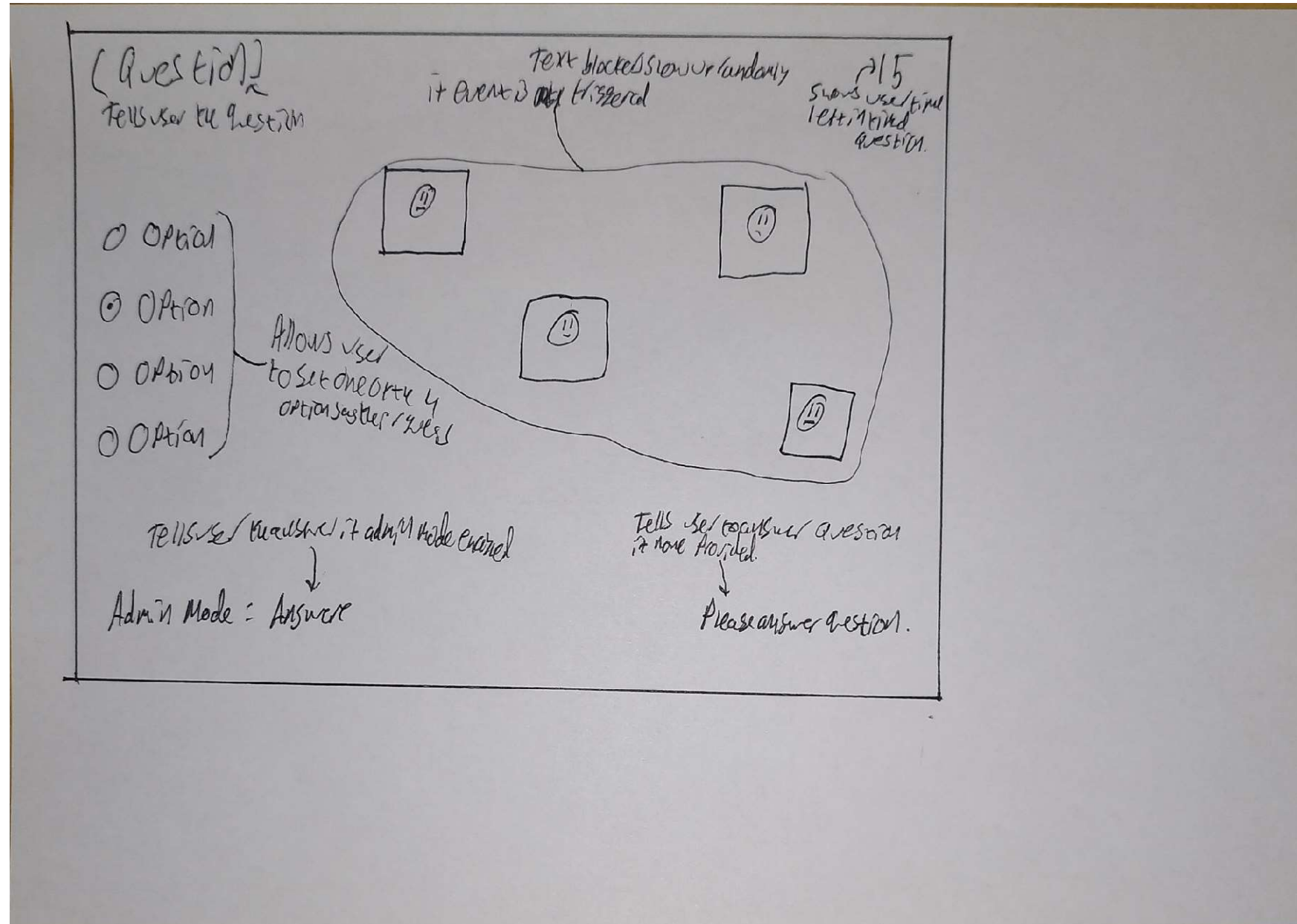
2nd ~~Option~~ Alt Answer 1 ☐ Add Answer

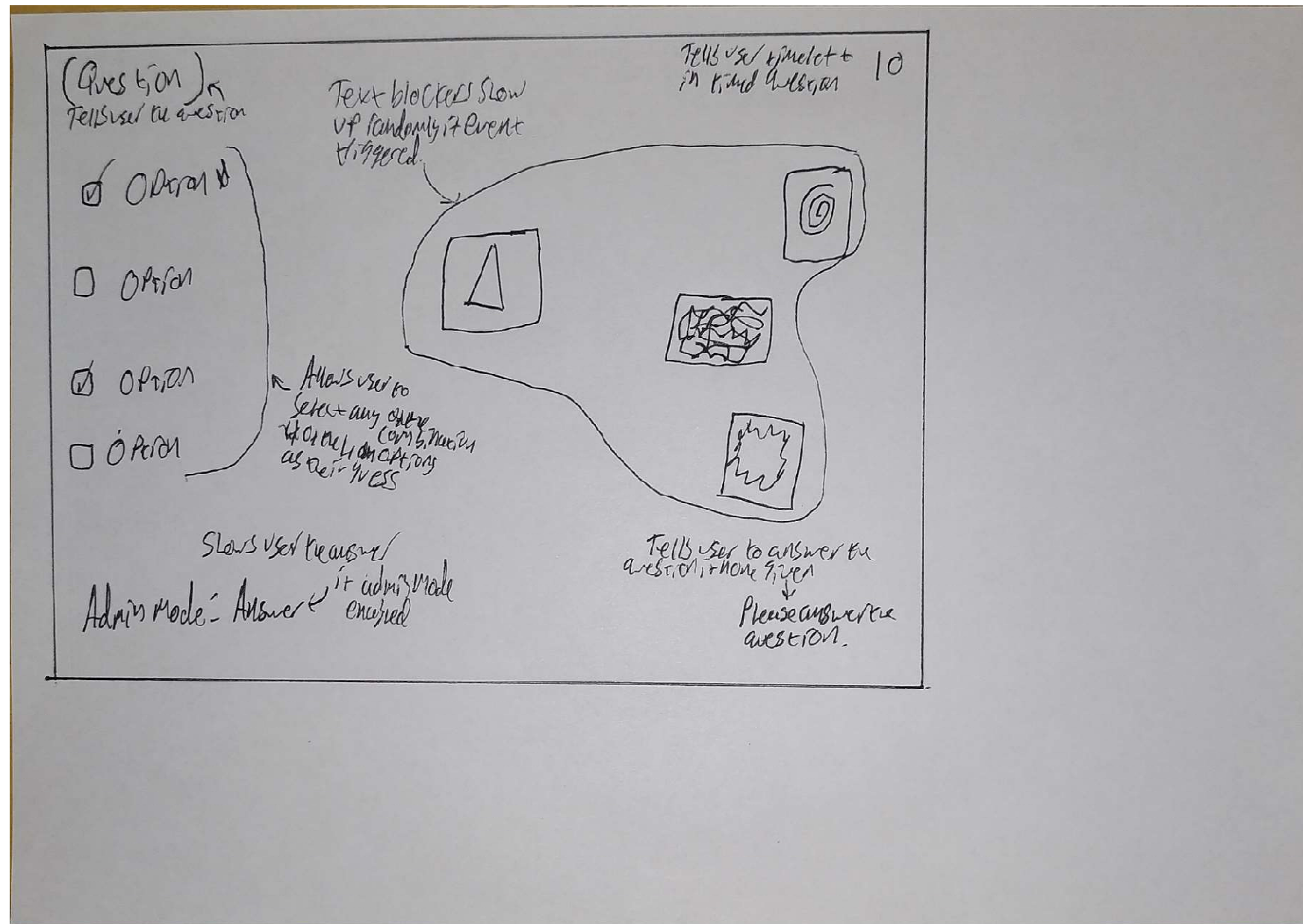
Alt Answer 2 ☐ Add Answer

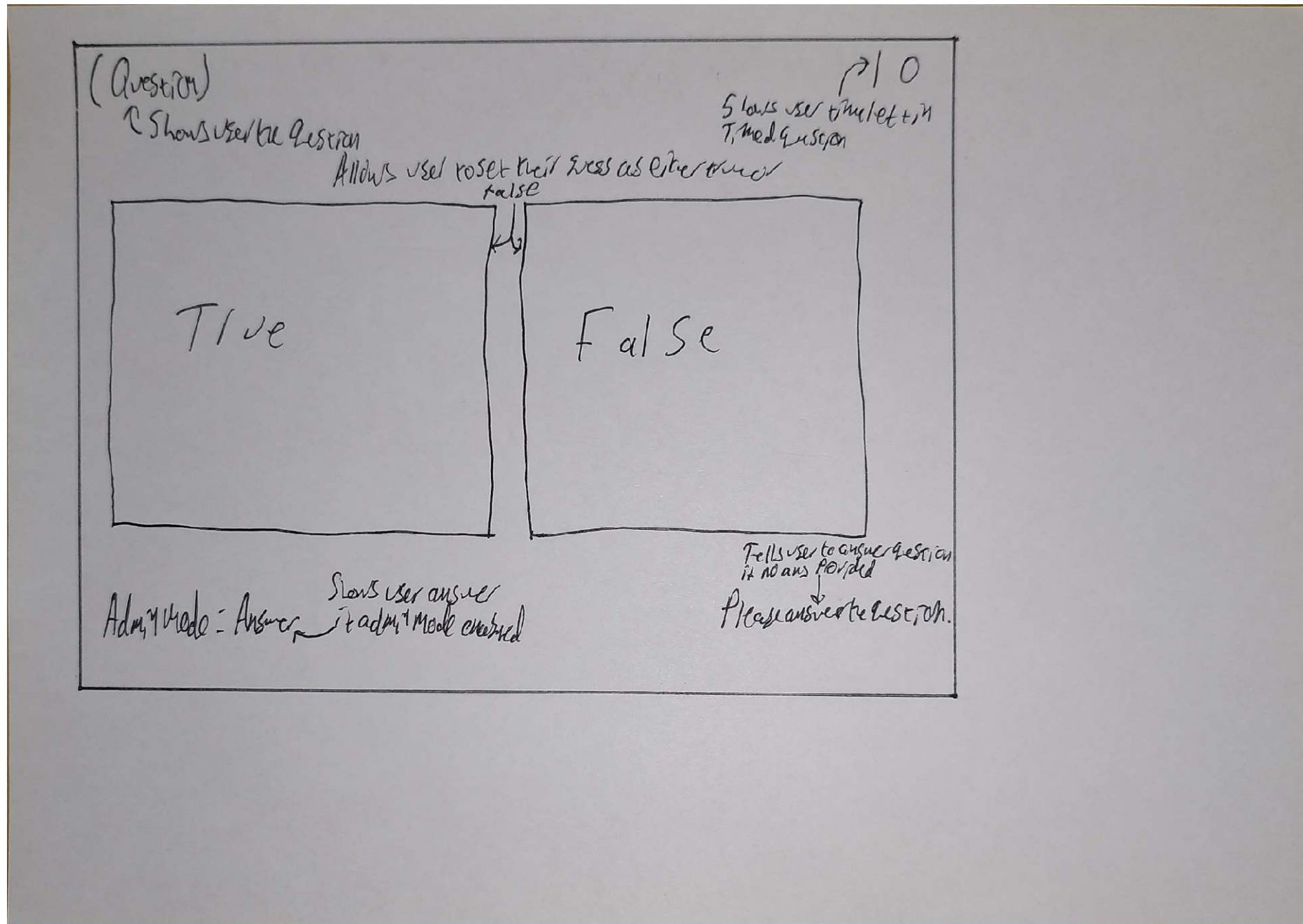
Alt Answer 3

Allows user to add up to 3 alternative answers to question

Allows user to set font size for Queses tool

MultiChoice

Checkbox

TrueOrFalse

FillInTheBlank

(Question) Tells user the question

Tells user time
left in timed
question → 1/5

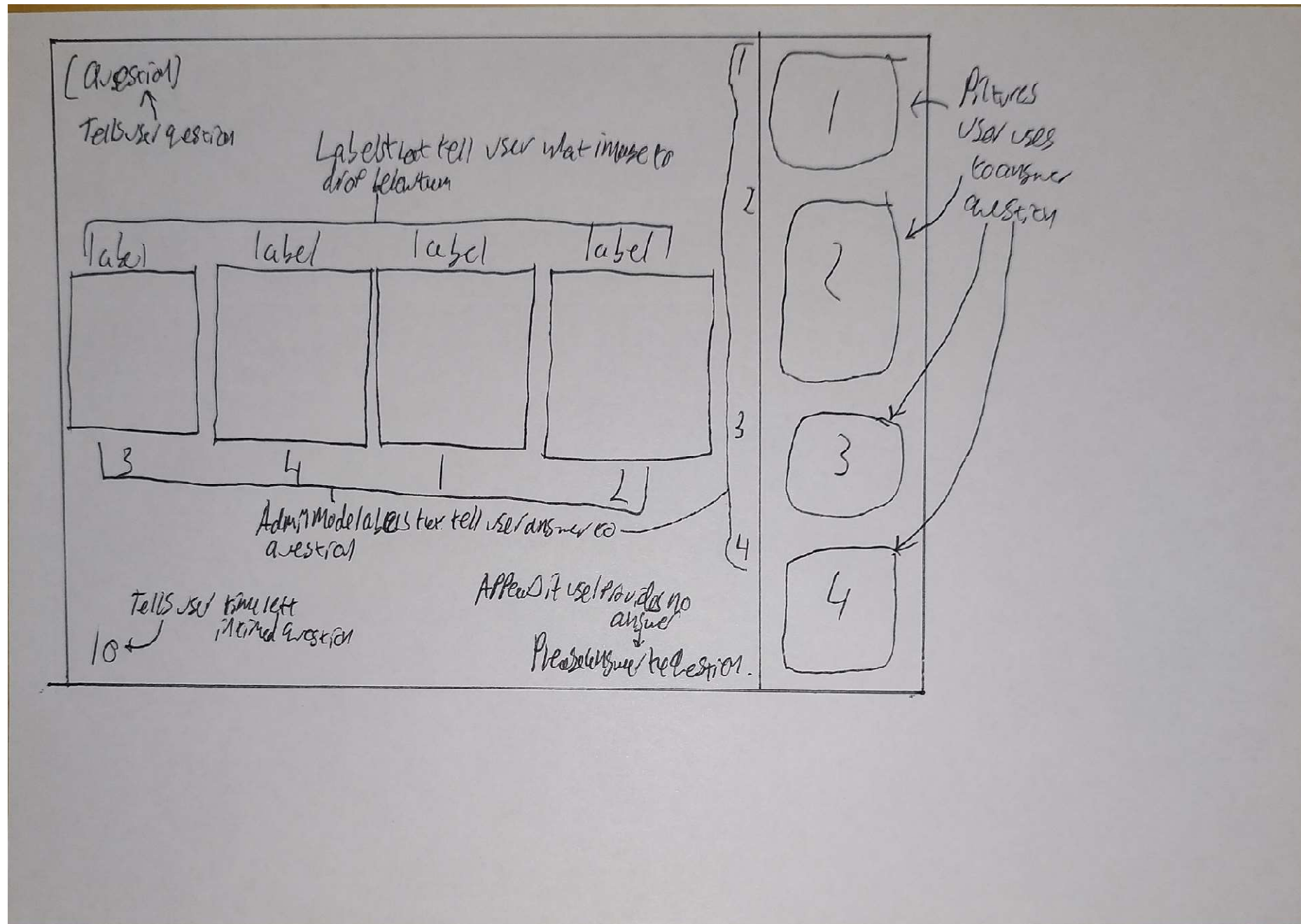
Answer Here

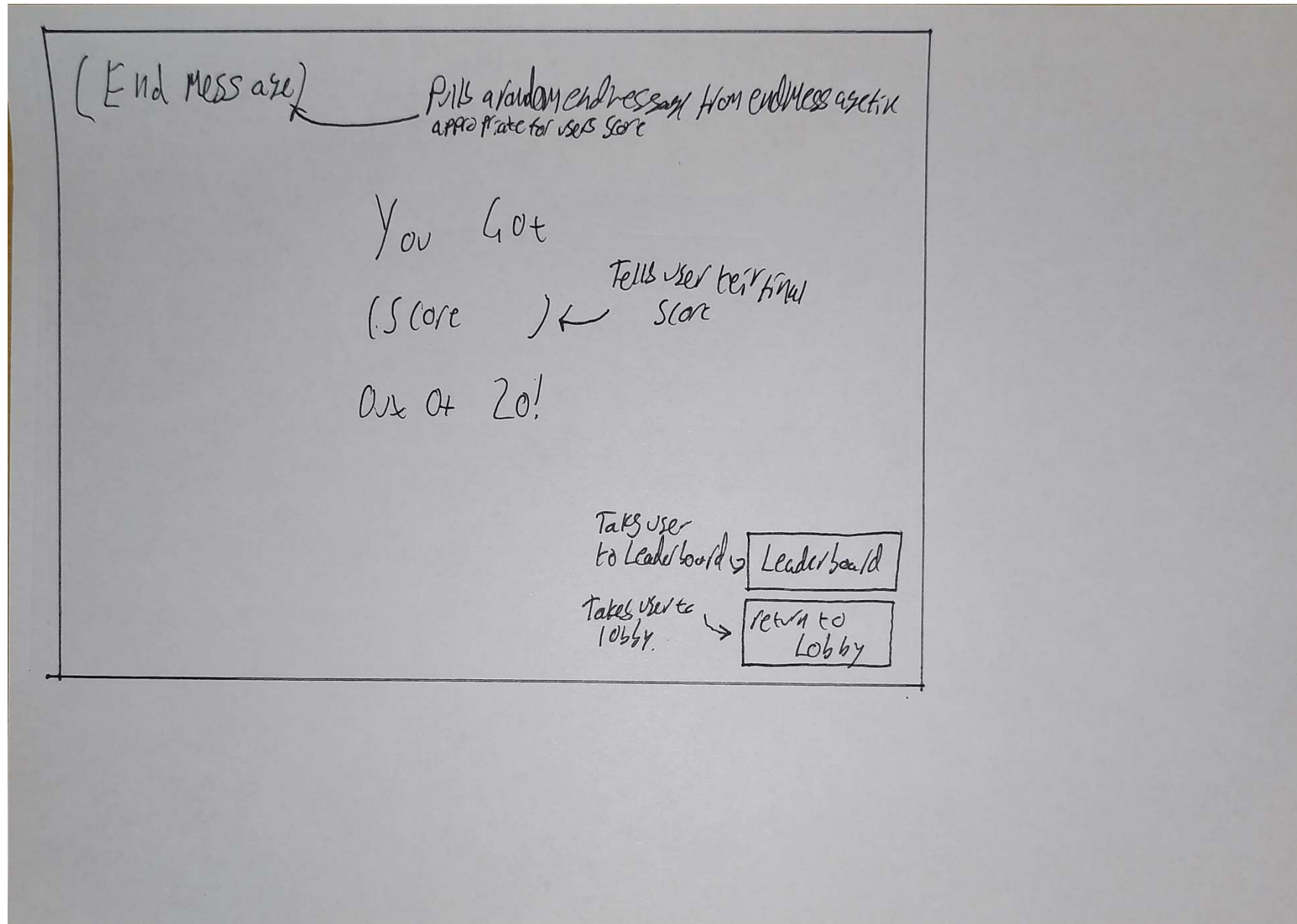
Allows user
to type in answer

Tells user answer if Admin mode
enabled
Admin Mode: Answer

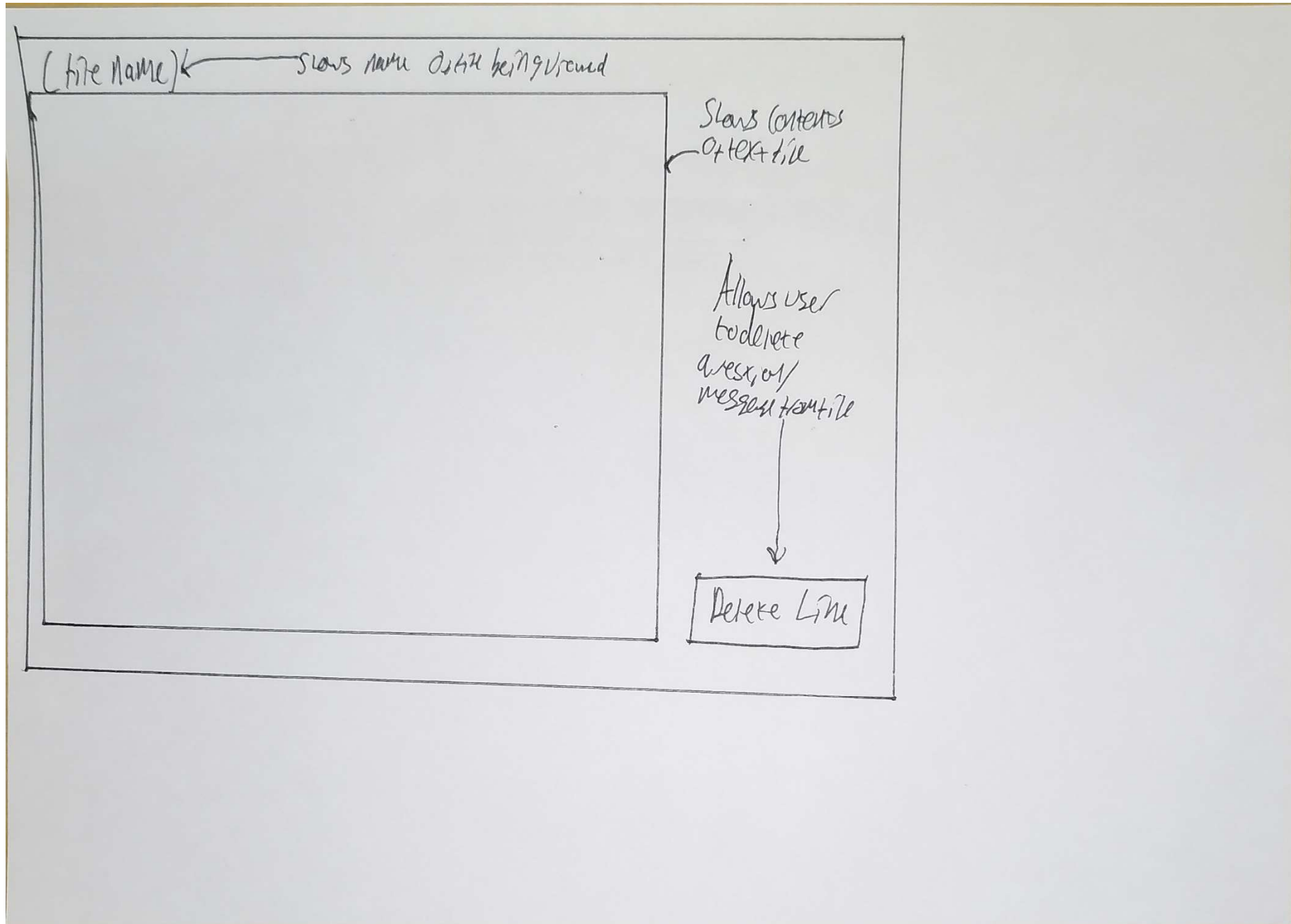
Shows user does not provide
answer.

Please answer the question

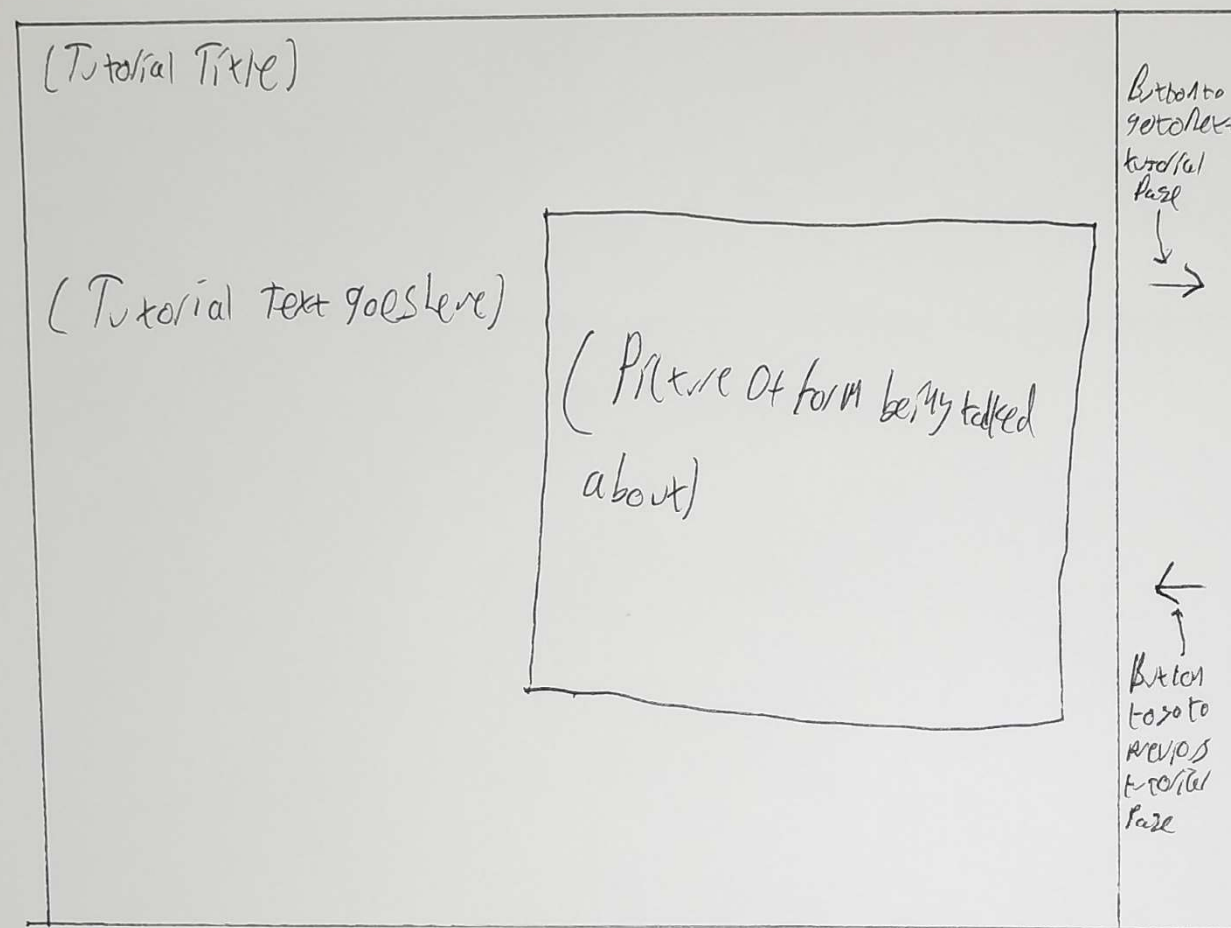
DragAndDrop

EndScreen

TextViewer



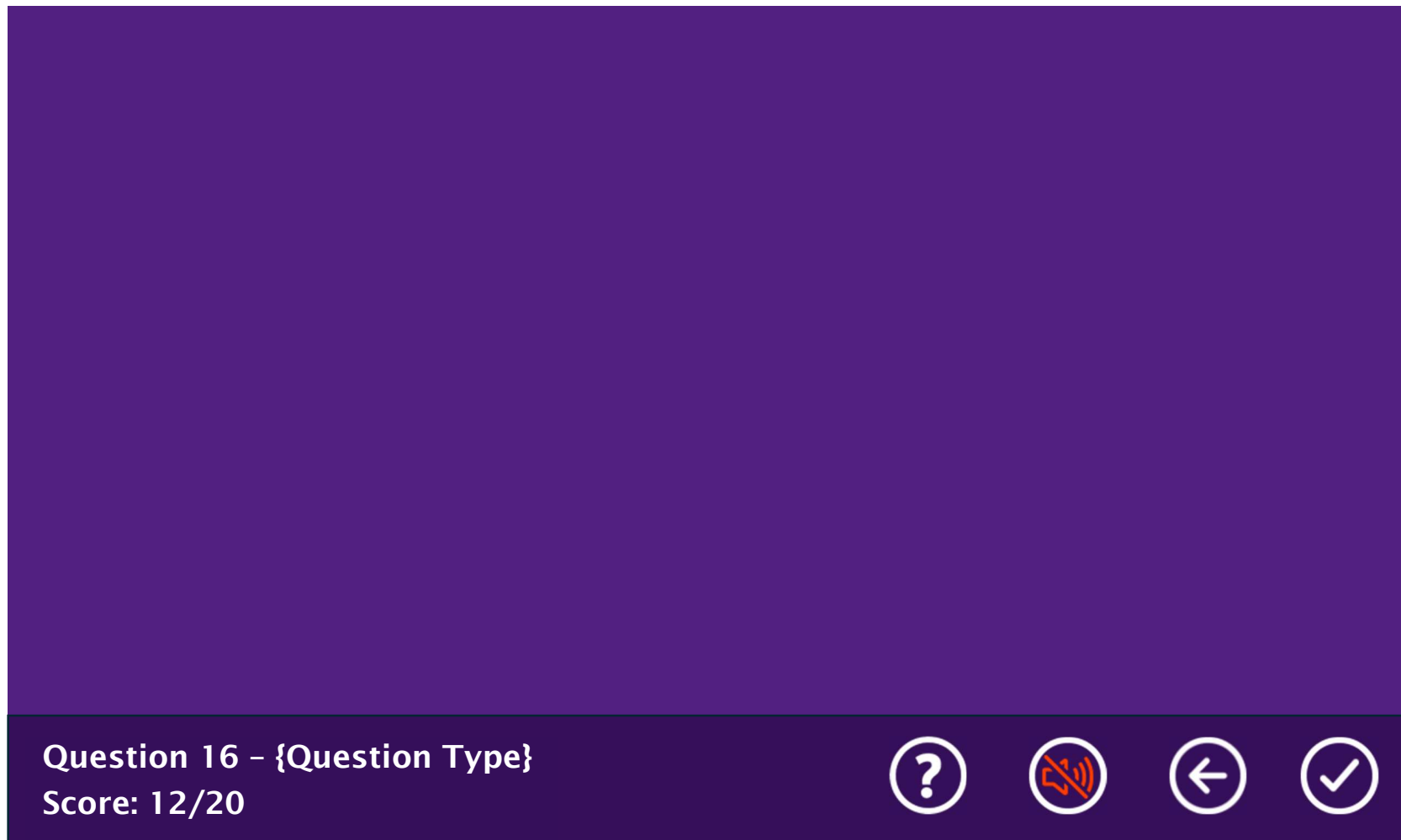
Tutorial



Final Designs

*During development I encountered a bug with WinForms that meant that some UI elements would appear in different positions when the program was ran, compared to the design window. I could not find a fix for this bug, so I had to move UI elements on my design windows in Visual Studio to compensate for the elements moving. Final Designs reflect how the forms are intended to look in-game, rather than how Visual Studio's design window shows the form.

HomeScreen



Component	Function	Properties	Trigger
HomePanel	The HomePanel displays the form that the user is currently interacting with, e.g. the Login Screen.	<u>BackColour</u> : 82, 32, 129 <u>Size</u> : 1600, 750 <u>Location</u> : 0, 0	N/A
QuestionUI	Shows the user what question they are on and what section it belongs to. It also shows the user's current score.	<u>Font</u> : Lucida Sans, 21.75pt, Demibold Roman <u>BackColour</u> : 54, 15, 90 <u>Size</u> : 836, 68 <u>Location</u> : 33, 770	Update Event
BackBtn	Takes the user back to the previous form they were on.	<u>Image</u> : BackButton64.png/Logout64.png <u>BackColour</u> : 54, 15, 90 <u>Size</u> : 68, 68 <u>Location</u> : 1485, 770	Click
MuteToggle	Mutes the Quiz's audio when clicked, unmutes it when clicked again.	<u>Image</u> : UnMuted64.png/Muted64Red.png <u>BackColour</u> : 54, 15, 90 <u>Size</u> : 68, 68 <u>Location</u> : 1370, 770	Click
CheckAns	Checks the answer the user has given for a question to see if it is right or not / Applies the settings the user has changed	<u>Image</u> : Check64.png <u>BackColour</u> : 54, 15, 90 <u>Size</u> : 68, 68 <u>Location</u> : 1255, 770	Click

TutorialBtn	Takes the user to the Tutorial Screen when clicked	<u>Image:</u> Tutorial64.png <u>BackColour:</u> 54, 15, 90 <u>Size:</u> 68, 68 <u>Location:</u> 1255, 770	Click
-------------	--	--	-------

**The dark purple bar at the bottom of HomeScreen is always shown to the user and is separate from the other screens; any buttons shown on this bar in other designs show the arrangement of said buttons on HomeScreen whenever that screen is shown on the panel.

Detailed Functionality

Component	Detailed Functionality
ShowWelcome	This function triggers whenever HomeScreen loads. This function will create a new instance of WelcomeScreen, it will then load the form onto the panel and setup EventHandlers for both the ShowCreate & ShowLogin events, which call events of the same names when invoked. It will then move the BackBtn to 1485, 770, the MuteToggle moves to 1370, 770 and CheckAns moves to 1255, 770. Finally, it sets the text of the window to "Welcome to The Circuit Breaker Quiz", change the image for BackBtn to BackButton64.png and then hides all UI buttons in the lower bar, before showing the WelcomeScreen form on the panel.
UpdateQNumber	When a question form invokes the UpdateQNumber event, this function runs, which populates the variables in QuestionUI.Text (current question number, current score, current section of the Quiz).
BackBtn_click	This event is triggered by clicking the BackBtn. When the event is triggered the BackBtn moves to 1485, 770, the MuteToggle moves to 1370, 770 and CheckAns moves to 1255, 770. It then enters a switch case that looks at the Target property of the Target class to see which form it needs to go back to. If Target.Target is set to "welcome" it will create a new WelcomeScreen object and assign it to a variable called Welcome. It will configure it so it can be shown on the panel and set up EventHandlers for the ShowCreate and ShowLogin events, which trigger the ShowCreate and ShowLogin functions in HomeScreen respectively.

	It will then hide all of the UI buttons on the bottom bar and change the text of the window to "Welcome to The Circuit Breaker Quiz" before unhiding the WelcomeScreen form on the panel. If Target.Target is set to "lobby" it will create a new Lobby object and assign it the variable lobby and configures it so it can be shown on the panel. It changes the text of the window to "Lobby The Circuit Breaker Quiz" and changes Target.Target to "welcome". EventHandlers are set up for the ShowDifficulty, NextSection, ShowLeaderboard, ShowQuestionDesigner and ShowSettings events, which all trigger functions of the same names in HomeScreen. It will also move TutorialBtn to 1255, 770. The image displayed on BackBtn is changed to Logout64.png, Common_Variables.score variables is set to 0, Common_Variables.q_number is set to 1 and Common_Variables.UsedQuestions is cleared. The UsedSections list is cleared, and all 5 question types are added back into the list so that they can show up again. Finally, CheckAns, MuteToggle and QuestionUI are all hidden, the lobby object is unhidden on the panel and a new event is created that triggers the CloseForm function on Lobby, when clicked. If Target.Target is set to "settings" it will run the same code as ShowSettings, if Target.Target is set to "designer" it will run the same code as ShowQuestionDesigner.
MuteToggle_click	Before a sound effect is played in the Quiz an if statement checks if Common_Variables.muted is false, if it is false the sound effect will play. When this button is clicked the MuteToggle_click method is ran, it first checks to see if the game is muted and plays click.wav if it isn't, it then checks the current state of Common_Variables.muted, if it is true it is set to be false and the image on the button changes to UnMuted64.png, if it is false, it to be true and the image on the button is changed to Muted64Red.png.
HomeScreen	Since ShowWelcome is set to run when HomeScreen loads, the visual properties for the buttons are set in the default constructor. This will initialise the components of the form and set all of the buttons FlatStyle to Flat and set their border size to 0. The image on BackBtn is set to BackButton64.png and QuestionUI, MuteToggle, BackBtn and CheckAns are all hidden.
ShowCreate	This function creates a new instance of CreateAccount and configures it so it can be shown on the panel. It also sets up an EventHandler for the

	ShowLobby event, which will trigger the ShowLobby function in HomeScreen when invoked, it will also set the window text to "Create an account The Circuit Breaker Quiz" and set the target for the back button to "welcome" and change its image to BackButton64.png and then shows BackBtn. Whenever a form triggers an event that moves on to another form, the old form is closed to save on RAM usage and is no longer shown on the panel, and the next form is then put on the panel, in this case, CreateAccount is shown to the user. Finally, it sets up an event that will trigger the CloseForm function in CreateAccount whenever BackBtn is clicked.
ShowLogin	This function creates a new instance of Login and configures it so it can be shown on the panel. It also creates an EventHandler for the ShowLobby event, which calls the ShowLobby function in HomeScreen when invoked. It also sets the window text to "Login The Circuit Breaker Quiz" and sets the back button target to "welcome". It changes the image shown on BackBtn to BackButton64.png and shows it, before showing Login on the panel. Finally, it sets up an event that will trigger the CloseForm function in Login whenever BackBtn is clicked.
ShowLobby	This function creates an instance of Lobby and configures it so it can be shown on the panel. It also changes the windows text to "Lobby The Circuit Breaker Quiz" and sets the back button target to "welcome". It sets up EventHandlers for the ShowDifficulty, NextSection, ShowLeaderboard, ShowQuestionDesigner & ShowSettings events, which trigger functions of the same names in HomeScreen. It also the used question and section lists and resets the score and question number. It then sets the image displayed on BackBtn to Logout64.png and sets up the event to trigger the CloseForm function on Lobby. MuteToggle and BackBtn are unhidden, before showing Lobby on the panel. The current page for the tutorial is also set to "Multi".
ShowDifficulty	This function creates a new instance of DifficultySelect before configuring it so it can be shown on the panel. The window text is then set to "Select Difficulty The Circuit Breaker Quiz" and the target for the back button is set to "lobby". The image for BackBtn is set to BackButton64.png and an event is made for it to call the CloseForm function on DifficultySelect when it is clicked, TutorialBtn is also hidden. Finally, Difficulty is displayed on the panel.

ShowLeaderboard	When this function is called it will create a new instance of Leaderboard before configuring it so it will show up on the panel properly. Then the window text is changed to "Leaderboard The Circuit Breaker Quiz" and the back buttons target is set to "lobby", its image is also set to BackButton64.png and has an event created so that when it is clicked it will call the CloseForm function on Leaderboard. The TutorialBtn is also hidden. Finally, Leaderboard is shown on the panel.
ShowQuestion Designer	When this function is called it will create a new instance of QuestionDesigner before configuring it so it will show up on the panel properly. Then the window text is changed to "Question Designer The Circuit Breaker Quiz" and the starting page for Tutorial is set to "Designer". The back buttons target is set to "lobby" its image is also set to BackButton64.png and has an event created so that when it is clicked it will call the CloseForm function on QuestionDesigner. Finally, QuestionDesigner is shown on the panel.
ShowSettings	When this function is called it creates a new instance of Settings before configuring it so it will show up properly on the panel. The window text will then be set to "Settings The Circuit Breaker Quiz" and the back buttons target is changed to "lobby", its image is also set to BackButton64.png. The locations for CheckAns, BackBtn & MuteToggle are set to (1485, 770), (1370, 770) & (1255, 770) respectively. CheckAns is then unhidden, and setting is shown on the panel. EventHandlers are made for 2 events, ShowTextView and ShowWelcome, which call functions with the same name in HomeScreen. 2 events are then setup, one which will trigger the CloseForm function on Settings when the BackBtn is clicked, and another which will trigger the ApplySettings function on Settings whenever the CheckAns button is clicked, as it serves as the apply settings button for this form. Common_Variables.CurrentPage is also set to "CustomEnd".
NextSection	This function will first check if Common_Variables.q_number is less than 16, (since the questions come in batches of 5 per section if the number is over 16 when a new section is called for, all 20 questions have been answered). If it is less than 16 it will then generate a random number between 0 and the current length of UsedSection and check the value stored at that position in UsedSection using a switch case statement. It will then remove that value from UsedSection so that it cannot show up again in this play through of the Quiz, it will also set Common_Variables.CurrentSection to the full name of the current section of the Quiz, for use in the QuestionUI label, before calling the

	relevant function for the section of the Quiz that has been selected e.g. ShowCheckbox(). If it over 16, it will begin the process of ending the Quiz. It first deserialises the UserData dat file into a list and then deletes the file. It will then search the list for the current account the user is using and populate its statistic variables (e.g. LifetimeCorrect & LifetimeIncorrect) with the updated figures before serialising the updated list into a new UserData file and clearing the list afterwards. It will then check if the user has enabled the setting that skips the end screen, by checking if Common_Variables.SkipEnd is true. If it is true it will run the same code as ShowLobby, if it is not true, it will run the ShowEndScreen function.
ShowMultiChoice	This event gets triggered whenever the switch case in NextSection goes down the "MultiChoice" branch. It will first create a new instance of MultiChoice before configuring it to be displayed on the panel correctly. The locations for CheckAns, BackBtn & MuteToggle are set to (1485, 770), (1370, 770) & (1255, 770) respectively, the target for the back button is set to "lobby" and the image on the BackBtn is set to BackButton64.png. The text on the window is set to "Multiple Choice The Circuit Breaker Quiz" and CheckAns is unhidden. (Each of the question sections' functions set the UI buttons positions and unhides CheckAns & QuestionUI in case they are the first question sections to be shown to the user.) QuestionUI.Text is then updated to show the current question number, current section of the Quiz (Multiple Choice) and the current score the user has. EventHandlers are then setup for the UpdateQNumber & NextSection events, which call functions of the same names in HomeScreen. QuestionUI is then unhidden, and events are made to call the CloseForm function in MultiChoice when the BackBtn is clicked and to also call the AnsCheck function in MultiChoice whenever CheckAns is clicked. Finally, MultiChoice is shown on the panel.
ShowCheckbox	This event is triggered whenever the switch case in NextSection goes down the "Checkbox" branch. It will first create a new instance of Checkbox, before configuring it so it can show up on the panel properly. The locations for CheckAns, BackBtn & MuteToggle are set to (1485,

	770), (1370, 770) & (1255, 770) respectively and the target for the back button is set to "lobby". The image for BackBtn is also set to BackButton64.png and the text on the Quiz window is set to "Checkbox The Circuit Breaker Quiz". QuestionUI.Text is then updated to show that the current section the Quiz is on is now Checkbox, it is also updated to show the current score of the user and the current question number, after this CheckAns is unhidden. EventHandlers are then setup for the UpdateQNumber & NextSection events, which trigger functions of the same name in HomeScreen when invoked. After this QuestionUI is unhidden and events are set up that call check.CloseForm when the BackBtn is clicked, and one that calls check.AnsCheck whenever CheckAns is clicked. Finally, Checkbox is shown on the panel.
ShowTrueOrFalse	This event is triggered whenever the switch case in NextSection goes down the "TrueOrFalse" branch. It will first create a new instance of TrueOrFalse, before configuring it so it can show up on the panel properly. The locations for CheckAns, BackBtn & MuteToggle are set to (1485, 770), (1370, 770) & (1255, 770) respectively and the target for the back button is set to "lobby". The image for BackBtn is also set to BackButton64.png and the text on the Quiz window is set to "True or False The Circuit Breaker Quiz". QuestionUI.Text is then updated to show that the current section the Quiz is on is now True or False, it is also updated to show the current score of the user and the current question number, after this CheckAns is unhidden. EventHandlers are then setup for the UpdateQNumber & NextSection events, which trigger functions of the same name in HomeScreen when invoked. After this QuestionUI is unhidden and events are set up that call torf.CloseForm when the BackBtn is clicked, and one that calls torf.AnsCheck whenever CheckAns is clicked. Finally, TrueOrFalse is shown on the panel.
ShowDragDrop	This event gets triggered whenever the switch case in NextSection goes down the "DragAndDrop" branch. It will first create a new DragAndDrop instance before configuring it to be displayed on the panel correctly.

	The locations for CheckAns, BackBtn & MuteToggle are set to (1485, 770), (1370, 770) & (1255, 770) respectively, the target for the back button is set to "lobby" and the image on the BackBtn is set to BackButton64.png. The text on the window is set to "Drag and Drop The Circuit Breaker Quiz" and CheckAns is unhidden. QuestionUI.Text is then updated to show the current question number, current section of the Quiz (Drag and Drop) and the current score the user has. EventHandlers are then setup for the UpdateQNumber & NextSection events, which call functions of the same names in HomeScreen. QuestionUI is then unhidden, and events are made to call the CloseForm function in DragAndDrop when the BackBtn is clicked and to also call the AnsCheck function in DragAndDrop whenever CheckAns is clicked. Finally, DragAndDrop is unhidden on the panel.
ShowFillInBlank	This event is triggered whenever the switch case in NextSection goes down the "FillInBlank" branch. It will first create a new instance of FillInBlank, before configuring it so it can show up on the panel properly. The locations for CheckAns, BackBtn & MuteToggle are set to (1485, 770), (1370, 770) & (1255, 770) respectively and the target for the back button is set to "lobby". The image for BackBtn is also set to BackButton64.png and the text on the Quiz window is set to "Fill In The Blank The Circuit Breaker Quiz". QuestionUI.Text is then updated to show that the current section the Quiz is on is now True or False, it is also updated to show the current score of the user and the current question number, after this CheckAns is unhidden. EventHandlers are then setup for the UpdateQNumber & NextSection events, which trigger functions of the same name in HomeScreen when invoked. After this QuestionUI is unhidden and events are set up that call blank.CloseForm when the BackBtn is clicked, and one that calls blank.AnsCheck whenever CheckAns is clicked. Finally, FillInBlank is shown on the panel.

ShowEndScreen	If the user has not enabled the SkipEnd option in the settings form, this form will show up at the end of the Quiz. This function will create a new instance of EndScreen and configure it so it can show up on the form correctly. After this, BackBtn, MuteToggle & CheckAns are hidden and EndScreen is shown on the panel. EventHandlers are then set up for the ShowLobby and ShowLeaderboard events, which call events of the same names in HomeScreen.
ShowTextView	When called this function will create a new instance of TextView before configuring it so it can show up on the form correctly. It then moves BackBtn to 1485, 770 and MuteToggle to 1370, 770, before hiding CheckAns. An else if statement then checks if the user called this function from the Settings form or QuestionDesigner form by checking if Common_Variables.FileToBeViewed is equal to any of the 5 text file names for Settings, or the 4 text file names for QuestionDesigner, setting Target.Target to either “settings” or “designer” accordingly. An event is then setup that will run the CloseForm function on view whenever BackBtn is clicked, before showing TextView on the panel.
TutorialBtn_Click	This function is called when TutorialBtn is clicked. It checks the current state of the isLobby variable and calls the ShowTutorial function if it is visible.

WelcomeScreen

Welcome to The Circuit Breaker Quiz

Create Account

Login

Components	Function	Properties	Trigger
CreateAccountBtn	Loads the CreateAccount form onto the panel	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Create Account" <u>Size:</u> 349, 87 <u>Location:</u> 1172, 224	Click
LoginBtn	Loads the Login form onto the panel	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Login" <u>Size:</u> 349, 87 <u>Location:</u> 1172, 490	Click
WelcomeHeader	Welcomes the user to the Quiz	<u>Text:</u> "Welcome to The Circuit Breaker Quiz" <u>Font:</u> Lucida Sans, 48pt, Demibold Roman <u>Size:</u> 870, 153 <u>Location:</u> 20,30	N/A

Detailed Functionality

Component	Detailed Functionality
LoginBtn	Whenever this button is clicked by the user it triggers the LoginBtn_click event, which invokes the ShowLogin event on the HomeScreen, it first checks to see if the game is muted and plays click.wav if it isn't, it then loads the Login form on to the panel, after it is loaded the WelcomeScreen closes itself, showing Login form to the user.
CreateAccountBtn	Whenever this button is clicked by the user it triggers the CreateAccount_click event, which invokes the ShowCreate event on the HomeScreen, it first checks to see if the game is muted and plays click.wav if it isn't, it then loads the CreateAccount form on to the panel, after it is loads, the WelcomeScreen closes itself showing CreateAccount form to the user.

CreateAccount

Create an Account

To play the quiz, you must have an account.
Your Username cannot contain a special character.
Your Password must be longer than
3 characters and contain:

- An upper and lowercase letter
- A number
- A special character

Username:

Username too short.

Password:

Password too short.



Confirm Password: Passwords don't match.

Create Account



Components	Function	Properties	Trigger
CreateHeader	Tells the user they are on the account creation screen.	<u>Font:</u> Lucida Sans, 44.25pt, Demibold Roman <u>Text:</u> "Create an Account" <u>ForeColour:</u> White <u>Size:</u> 633, 80 <u>Location:</u> 20, 30	N/A
PassRequirements	Tells the user of the rules their password must follow for it to be valid.	<u>Font:</u> Lucida Sans, 18.25pt, Demibold Roman <u>Text:</u> "To play the Quiz, you must have an account. Your Username cannot contain a special character, or be shorter than 3 characters. Your Password must be longer than 3 characters and contain: • An upper and lowercase letter • A number • A special character <u>ForeColour:</u> White <u>Size:</u> 659, 260 <u>Location:</u> 27, 260	N/A
UsrHeader	Tells the user that the textbox below it is used to input your desired username. This turns red when the username is invalid, and back to white when the user	<u>Font:</u> Lucida Sans, 15pt, Demibold Roman <u>Text:</u> "Username:" <u>ForeColour:</u> White/Red <u>Size:</u> 119, 23	Username requirement not met/Clicking into UsrnameInput

	changes the username.	<u>Location:</u> 1162, 219	
PasswordHeader	Tells the user that the textbox below it is used to input your desired password. This turns red when the password is invalid, and back to white when the user changes the password.	<u>Font:</u> Lucida Sans, 15pt, Demibold Roman <u>Text:</u> "Password:" <u>ForeColour:</u> White/Red <u>Size:</u> 114, 23 <u>Location:</u> 1162, 305	Password requirement not met/Clicking into PasswordInput
ConfirmHeader	Tells the user that the textbox below it is used to input your desired password again, to confirm that its memorable. This turns red when the password isn't confirmed, and back to white when the user changes the password.	<u>Font:</u> Lucida Sans, 15pt, Demibold Roman <u>Text:</u> "Confirm Password:" <u>ForeColour:</u> White/Red <u>Size:</u> 204, 23 <u>Location:</u> 1162, 392	Password did not confirm/Clicking into ConfirmPassword
CreateBtn	Triggers the validation code that checks if the username is not already used, the password meets all of the requirements and that the text entered into ConfirmPassword matches the text in PasswordInput.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Create Account" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1166, 488	Click

PassShow	Toggles between showing the users password in the textboxes, or hiding them with asterisks, updating the image shown on the button accordingly.	<u>Image:</u> PassShow/PassHide64 <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Size:</u> 50, 50 <u>Location:</u> 1522, 350	Click
UsernameInput	This textbox allows the user to input their desired username.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>Size:</u> 340, 50 <u>Location:</u> 1166, 244	Clicking textbox & typing.
PasswordInput	This textbox allows the user to input their desired password.	<u>BorderStyle:</u> None <u>Font:</u> Microsoft Sans Serif, 30.75pt, Regular <u>PasswordChar:</u> * <u>Size:</u> 340, 50 <u>Location:</u> 1166, 244	Clicking textbox & typing.
ConfirmPassword	This textbox allows the user to input their password again, to ensure that they know it/typed the right password.	<u>BorderStyle:</u> None <u>Font:</u> Microsoft Sans Serif, 30.75pt, Regular <u>PasswordChar:</u> * <u>Size:</u> 340, 50 <u>Location:</u> 1166, 244	Clicking textbox & typing.

UsrTaken	This label appears whenever the username that the user wants is already taken.	<u>Text:</u> "Username is taken." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 135, 15 <u>Location:</u> 1375, 226	Requirement met (Username is already being used)
NoUsrname	This label appears when the user does not enter a username in the text box.	<u>Text:</u> "Please enter a username." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 135, 15 <u>Location:</u> 1375, 226	Requirement met (No Username provided)
TooShort	This label appears when the password does not meet the length requirement.	<u>Text:</u> "Password is too short." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 175, 15 <u>Location:</u> 1346, 313	Requirement not met (Password is not long enough)
NoNumber	This label appears when the user's password does not contain a number.	<u>Text:</u> "Password must contain a number." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 236, 15 <u>Location:</u> 1270, 313	Requirement not met (Password does not contain a number)

NotMixed	This label appears when the user's password is not in mixed case.	<u>Text:</u> "Password must be mixed case." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 214, 15 <u>Location:</u> 1292, 312	Requirement not met (Password is not in mixed case)
NoSymbol	This label appears when the user's password does not contain a symbol.	<u>Text:</u> "Password must contain a symbol." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 235, 15 <u>Location:</u> 1271, 312	Requirement not met (Password doesn't contain a symbol)
NoPassword	This label appears whenever the user has tried to make an account without a password.	<u>Text:</u> "Please enter a password." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 176, 15 <u>Location:</u> 1330, 313	Requirement met (No Password Entered.)
NoMatch	This label appears whenever the password entered in the ConfirmPassword text box does not match the password. in the PasswordInput textbox	<u>Text:</u> "Passwords don't match." <u>Font:</u> Lucida Sans, 9.75pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 169, 15 <u>Location:</u> 1360, 399	Requirement not met (Confirm Password does not match password)

Component	Detailed Functionality
UsrnameInput_Click	This event changes the UsrHeader colour to white and hides all of the error messages that may have appeared.
PasswordInput_Click	This event is triggered whenever PasswordInput is clicked, and it changes the PasswordHeader text colour to white and hides all of the error messages that may have appeared.
ConfirmPassword_Click	This event is triggerd whenever ConfirmPassword is clicked, and it sets the text colour of ConfirmHeader to white and hides NoMatch.
PassShow_Click	This function is triggered when PassShow is clicked. It first checks to see if the game is muted and plays click.wav if it isn't, it then checks the current state of PassShown. If it is true, the PasswordInput & ConfirmPassword PasswordChar properties are set to *, the PassShown variable is set to false and the image on the PassShow button is changed to PassHide64.png. If the variable is false, it sets the PasswordChar property for PasswordInput & ConfirmPassword to \0 which unhides the text and sets the image for the PassShow button to PassShow.png.
CreateBtn_Click	This event is triggered whenever CreateBtn is clicked by the user, which in turn calls the Validation function after playing click.wav if the game is not muted.
Validation	When this function is called if first creates 3 Boolean variables that are set to true if validation succeeds. The username is validated first, within a try catch statement to handle the exceptions thrown if the username if invalid. The username has any whitespace removed before entering the validation if statement, which checks if the username is not blank, contains only alphanumeric characters and if the username is longer than 3 characters. If any of these checks fail, a custom exception is thrown that will show an error message to the user, set the ForeColour of UsrnameHeader to red and write the error to the ErrorLog. If none of the checks fail, the valid username Boolean is set to true.

	The password is then validated also within a try catch statement to handle any thrown exceptions. After this, 5 Regex statements are created and assigned to variables, which contain checks for the 5 different requirements for the password requirements (contains upper & lowercase character, has a number, has a special character & is more than 3 characters) An if statement then checks if no password is inputted or if any of the Regex statements aren't true, triggering custom exceptions that shows an error message to the user, sets the ForeColour of PasswordHeader to red and writes the error to the ErrorLog accordingly. If none of the Regex statements are false and the password is not empty, the password Boolean is set to true. The password is then confirmed, again within a try catch statement to handle any thrown exceptions. Any whitespace is removed and an if statement checks if the text in PasswordInput & ConfirmPassword are the same, if it is the confirm Boolean is set to true. If they are not the same, a custom exception is triggered, and an error is written to the log. If all of the Booleans are true, then Common_Variables.username is set to Usrname.Text. After this is it checks to see if an UserData file already exists, if one already does it will deserialise the file into a list and check to see if the username is already used within the file, if it is already used it will throw an exception that will show an error to the user, change the ForeColour of ConfirmHeader to red and write an error to the log before leaving the if statement. If the username is not already used in the file, the existing file is deleted and then previously deserialised list is appended with the new user account and then reserialised into a new UserData file, before calling the LoadLobby event and closing the form. If the UserData file does not already exist a new file is created by serialising the users account before calling the LoadLobby event and closing the form.
ErrorLog	This function is used to write to the ErrorLog file. It writes a new line to the file that has the time of the error in UTC+0 followed by the error message provided by the exception thrown.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.

Login

Login to your Account

To play the Quiz, you must have an account.

Having an account lets you save your previous scores on the leaderboard.

If you don't have an account hit the back button and click the Create Account button.

Username:

Password:



Login

Username or password is incorrect.



Components	Function	Properties	Trigger
LoginHeader	Tells the user that they are on the Login from	<u>Font:</u> Lucida Sans, 44.25pt, Demibold Roman <u>Text:</u> "Login to your Account" <u>ForeColour:</u> White <u>Size:</u> 707, 80 <u>Location:</u> 20, 30	N/A
AccountInfo	Tells the user that they need an account, why they need one and where to make one if they don't have one	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>Text:</u> "To play the Quiz, you must have an account. Having an account lets you save your previous scores on the leaderboard. If you don't have an account hit the back button and click the Create Account button." <u>ForeColour:</u> White <u>Size:</u> 659, 202 <u>Location:</u> 27, 290	N/A
UsrnameHeader	Tells the user that the textbox below is used to input your username. Turns red to indicate when there is an error.	<u>Font:</u> Lucida Sans, 15pt, Demibold Roman <u>Text:</u> "Username:" <u>ForeColour:</u> White/Red <u>Size:</u> 119, 23 <u>Location:</u> 1162, 275	Validation error/Clicking into UsrnameInput

PasswordHeader	Tells the user that the textbox below is used to input their password. Turns red to indicate when there is an error.	<u>Font:</u> Lucida Sans, 15pt, Demibold Roman <u>Text:</u> "Password:" <u>ForeColour:</u> White/Red <u>Size:</u> 114, 23 <u>Location:</u> 1162, 361	Validation error/Clicking into PasswordInput
LoginBtn	Attempts to login to an account matching the credentials inputted by the user	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Login" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1166, 454	Click
UsrnameInput	Allows the user to input their username.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>Size:</u> 340, 50 <u>Location:</u> 1166, 300	Clicking textbox & typing
PasswordInput	Allows the user to input their password.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>PasswordChar:</u> * <u>Size:</u> 340, 50 <u>Location:</u> 1166, 300	Clicking textbox & typing

PassShow	Toggles between showing the users password in the textboxes, or hiding them with asterisks, updating the image shown on the button accordingly.	<u>Image:</u> PassShow/PassHide64 <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Size:</u> 50, 50 <u>Location:</u> 1522, 350	Click
Wrong	Tells the user that either the username or password is wrong, not specifying which is wrong to help prevent against brute force attacks.	<u>Font:</u> Lucida Sans, 12pt, Demibold Roman <u>Text:</u> "Username or password is incorrect" <u>ForeColour:</u> Red <u>Size:</u> 294, 18 <u>Location:</u> 1187, 557	Failed to find account matching username / Password entered does not match stored password

Detailed Functionality

Component	Detailed Functionality
PassShow_Click	This function checks the current state of the PassShown Boolean variable, if it is true, it sets PasswordInput.PasswordChar to an Asterix, sets the Boolean to false and changes the image shown on PassShow to PassHide64. If it is false, it sets PasswordInput.PasswordChar to \0, which will show the password again. It will also set the Boolean to true and set the image shown on PassShow to PassShow.
UsernameInput_Enter	This event triggers whenever you click into UsernameInput, when this event triggers the Wrong label hides and the ForeColour for UsernameHeader changes to white.
PasswordInput_Enter	This event triggers whenever you click into PasswordInput, when this event triggers the Wrong label hides and the ForeColour for PasswordHeader changes to white.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
Login_Load	This event is triggered when the form is loaded onto the panel. When the event is triggered, the Wrong label is hidden, the LoginBtn has its FlatStyle set to Flat, and its order size is set to 0.
LoginBtn_click	This event is triggered when the LoginBtn is clicked. It will play click.wav if the game is not muted and the checks if the UserData file exists and if it doesn't, a message box is show to the user that tells them to make an account. If the file does exist, it is deserialised into a list. It then runs a foreach loop that checks each object in the list to see if its User_id & PassCode properties match the username and password entered by the user. If it does match, Common_Variables.username is set to the username inputted by the user and the Quiz statistics variables are populated by the corresponding variables in the object. It then checks if the LoadLobby event is not null and invokes it if it isn't. It then closes the form. If it doesn't match, the Wrong label will show the .Text property for both UsernameInput and PasswordInput will be blanked and the colour properties for both the PasswordHeader and UsernameHeader are set to red.

Lobby

Hello, {insert username here}

Your lifetime correct answers:

Your lifetime incorrect answers:

=====

Lifetime Correct / Lifetime Incorrect

=====

Checkbox:

0/0

Drag and Drop:

0/0

Fill in the Blank:

0/0

Multiple Choice:

0/0

True or False:

0/0

Change Difficulty

Leaderboard

Play Quiz

Question Designer

Settings



Components	Function	Properties	Trigger
PlayGameBtn	Starts the Quiz	<u>Font:</u> Lucida Sans, 15.75pt, DemiboldRoman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Play Quiz" <u>Size:</u> 436, 108 <u>Location:</u> 1126, 332	Click
LeaderboardBtn	Loads the Leaderboard form onto the panel	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Leaderboard" <u>Size:</u> 349, 87 <u>Location:</u> 1172, 224	Click
DesignerBtn	Loads the QuestionDesigner form onto the panel	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White	Click

		<u>BackColour</u> : 54, 15, 90 <u>Text</u> : "Question Designer" <u>Size</u> : 349, 87 <u>Location</u> : 1172, 462	
SettingsBtn	Loads the Settings form onto the panel.	<u>Font</u> : Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle</u> : Flat <u>Cursor</u> : Hand <u>ForeColour</u> : White <u>BackColour</u> : 54, 15, 90 <u>Text</u> : "Settings" <u>Size</u> : 349, 87 <u>Location</u> : 1172, 565	Click
DifficultySelectBtn	Loads the DifficultySelect form onto the panel.	<u>Font</u> : Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle</u> : Flat <u>Cursor</u> : Hand <u>ForeColour</u> : White <u>BackColour</u> : 54, 15, 90 <u>Text</u> : "Change Difficulty" <u>Size</u> : 349, 87 <u>Location</u> : 1172, 121	Click
LobbyHeader	Greets the user and shows their username.	<u>Font</u> : Lucida Sans, 44.25pt, Demibold Roman	Lobby_Load

		<u>Text:</u> "Hello, {insert username here}" <u>ForeColour:</u> White <u>Size:</u> 775, 67 <u>Location:</u> 20, 30	
Stats	Shows the user some general statistics of their account, as well as the titles for the ExtendedStats,	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>Text:</u> "Your lifetime correct answers: Your lifetime incorrect answers: ===== Lifetime Correct / Lifetime Incorrect ===== Checkbox: Drag and Drop: Fill in the Blank: Multiple Choice: True or False: " <u>ForeColour:</u> White <u>Size:</u> 606, 407 <u>Location:</u> 25, 161	Lobby_Load
ExtendedStats	Shows the user their lifetime correct/incorrect answers, broken down into the 5 different question types.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>Text:</u> <pre> "{Common_Variables.CheckLifeCorrect} /{CheckLifeIncorrect} {Common_Variables.DropLifeCorrect} /{Common_Variables.DropLifeIncorrect} {Common_Variables.BlankLifeCorrect} /{Common_Variables.BlankLifeIncorrect} {Common_Variables.MultiLifeCorrect} /{Common_Variables.MultiLifeIncorrect} {Common_Variables.TorfLifeCorrect} /{Common_Variables.TorfLifeIncorrect} " </pre> <u>ForeColour:</u> White <u>Size:</u> 122, 188	Lobby_Load

		Location: 431, 380	
--	--	--------------------	--

Detailed Functionality

Component	Detailed Functionality
DifficultyBtn_click	Whenever DifficultyBtn is clicked it triggers this event which checks if the ShowDifficulty event is not null and then invokes it. After ShowDifficulty is invoked the Lobby form closes itself.
PlayBtn_click	Whenever PlayBtn is clicked it triggers this event which checks if the StartQuiz event is not null and then invokes it. After StartQuiz is invoked the Lobby form closes itself.
LeaderboardBtn_click	Whenever LeaderboardBtn is clicked it triggers this event which checks if the ShowLeaderboard event is not null and then invokes it. After ShowLeaderboard is invoked the Lobby form closes itself.
DesignerBtn_click	Whenever DesignerBtn is clicked it triggers this event which checks if the ShowQuestionDesigner event is not null and then invokes it. After ShowQuestionDesigner is invoked the Lobby form closes itself.
SettingsBtn_click	Whenever SettingsBtn is clicked it triggers this event which checks if the ShowSettings event is not null and then invokes it. After ShowSettings is invoked the Lobby form closes itself.
Lobby_Load	Whenever the Lobby form is loaded onto the panel this event triggers it sets LobbyHeader.Text to "Hello, (Common_Variables.username)" and sets the FlatStyle for all of the buttons to flat and sets the border size to 0. It also populates the variable fields for Stats.Text and ExtendedStats.Text using the variables stored in the Common_Variables global class.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.

DifficultySelect

Select Difficulty

Circuit Breaker has 3 difficulty settings:

Easy - The Quiz will only be made up of easy questions but correct answers will only be worth 0.5 points

Normal - The Quiz will be made up of a mix of easy and difficult questions with a small chance for random events to occur. Questions are worth 0.75 points

Hard - The Quiz will be made up of only difficult questions with a decent chance for random events to occur and points will be worth 1 point

Easy :D

Normal :/

Hard >:(



Components	Function	Properties	Trigger
DifficultyHeader	Tells the user that they are on the DifficultySelect form.	<u>Font:</u> Lucida Sans, 44.25pt, Demibold Roman <u>Text:</u> "Select Difficulty" <u>ForeColour:</u> White <u>Size:</u> 436, 108 <u>Location:</u> 20,30	N/A
DifficultyInfo	Tells the user information about the different difficulties.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>Text:</u> "Circuit Breaker has 3 difficulty settings: Easy - The Quiz will only be made up of easy questions but correct answers will only be worth 0.5 points Normal - The Quiz will be made up of a mix of easy and difficult questions with a small chance for random events to occur. Questions are worth 0.75 points Hard - The Quiz will be made up of only difficult questions with a decent chance for random events to occur and points will be worth 1 point " <u>ForeColour:</u> White <u>Size:</u> 1060, 387 <u>Location:</u> 26, 173	N/A

EasyBtn	Selects easy mode for the Quiz whenever it is clicked.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Easy :D" <u>Size:</u> 349, 87 <u>Location:</u> 1172, 142	Click
NormalBtn	Selects normal mode for the Quiz whenever it is clicked.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Normal :/" <u>Size:</u> 349, 87 <u>Location:</u> 1172, 332	Click
HardBtn	Selects hard mode for the Quiz when clicked.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Hard >:("	Click

		<u>Size:</u> 349, 87 <u>Location:</u> 1172, 522	
--	--	--	--

Detailed Functionality

Component	Detailed Functionality
Difficulty_Load	This function sets the visual properties of the 3 buttons (FlatStyle = Flat, BorderSize = 0) and then checks what the current difficulty is (normal by default), using a switch case, when it finds what the current difficulty is it will set its border size to 5 and then set the border colour according to the buttons difficulty,(LimeGreen for easy, orange for normal, and red for hard).
EasyBtn_click	Whenever the EasyBtn is clicked this event triggers, which sets the border size of the EasyBtn to 5 and sets its border colour to LimeGreen and sets the other 2 buttons border size to 0. It then sets Common_Variables.difficulty to "e" and sets the local difficulty variable to the current state of Common_Variables,difficulty.
NormalBtn_click	Whenever the NormalBtn is clicked this event triggers, which sets the border size of the NormalBtn to 5 and sets its border colour to Orange and sets the other 2 buttons border size to 0. It then sets Common_Variables.difficulty to "n" and sets the local difficulty variable to the current state of Common_Variables,difficulty.
HardBtn_click	Whenever the HardBtn is clicked this event triggers, which sets the border size of the HardBtn to 5 and sets its border colour to Red and sets the other 2 buttons border size to 0. It then sets Common_Variables.difficulty to "h" and sets the local difficulty variable to the current state of Common_Variables,difficulty.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.

Settings

Settings

☐ Admin Mode

☐ Read Out Questions

☐ Skip End Screen

☐ Dyslexia Mode

Volume:


Admin Settings:

Delete UserData

Reset Leaderboard

Custom End Message:

Select a message category ▼ See Existing Messages

Message: Please select a message category Font Size

Preview Save



Components	Function	Properties	Trigger
SettingsHeader	Tells the user that they are on the settings page or shows them the preview of their custom end message.	<u>Font:</u> Lucida Sans, 44.25pt / FontSizeInput.Value, Demibold Roman <u>Text:</u> "Settings"/MessageInput.Text <u>ForeColour:</u> White <u>Size:</u> 1256, 141 <u>Location:</u> 20,30	PreviewBtn Click
AdminHeader	Tells the user that the buttons below are for the Quiz's admin settings.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>Text:</u> "Admin Settings:" <u>ForeColour:</u> White <u>Size:</u> 277, 37 <u>Location:</u> 747, 171	Settings_Load / ApplySettings
EndMessageHeader	Tells the user that the UI elements below are used to add custom end screen messages	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>Text:</u> "Custom End Message:" <u>ForeColour:</u> White <u>Size:</u> 371, 37 <u>Location:</u> 747, 338	Settings_Load / ApplySettings
MessageHeader	Tells the user that the textbox below is used to input the end message	<u>Font:</u> Lucida Sans, 16pt, Demibold Roman <u>Text:</u> "Message:" <u>ForeColour:</u> White	Settings_Load / ApplySettings

		<u>Size:</u> 115, 25 <u>Location:</u> 749, 412	
FontSizeLabel	Tells the user that the NumericUpDown below is used to input the font size for the label that shows the message	<u>Font:</u> Lucida Sans, 16pt, Demibold Roman <u>Text:</u> "Font Size:" <u>ForeColour:</u> White <u>Size:</u> 117, 25 <u>Location:</u> 1309, 412	Settings_Load / ApplySettings
VolumeHeader	Tells the user that the trackbar below is used to adjust the volume.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>Text:</u> "Volume: {value}" <u>ForeColour:</u> White <u>Size:</u> 225, 40 <u>Location:</u> 35, 431	N/A
checkedListBox	Allows the user to toggle the 4 options that are on the box.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Items:</u> "Admin Mode Read Out Questions Skip End Screen Dyslexia Mode" <u>Size:</u> 392, 160 <u>Location:</u> 30, 215	Click

volumeBar	Allows the user to set the system volume to the number they want.	<u>BackColour:</u> 54, 15, 90 <u>Maximum:</u> 100 <u>LargeChange:</u> 10 <u>SmallChange:</u> 1 <u>TickFrequency:</u> 2 <u>Size:</u> 392, 160 <u>Location:</u> 30, 215	N/A
DeleteUsrDat	Allows the user to delete UserData.dat file.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Delete UserData" / "Are you sure?" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 743, 215	Click
ResetLeaderboard	Allows the user to delete Leaderboard.dat file.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Reset Leaderboard" / "Are you sure?" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 743, 215	Click

TextView	Allows the user to view the contents of the text file that contain	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "See Existing Messages" <u>ForeColour:</u> White <u>Size:</u> 266, 44 <u>Location:</u> 1141, 344	Click
PreviewBtn	Changes the text of SettingsHeader so the user can see what their message will look like.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Preview" <u>ForeColour:</u> White <u>Size:</u> 170, 44 <u>Location:</u> 745, 475	Click
SaveBtn	Saves the user's custom message.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Save" / "Are you sure?" <u>ForeColour:</u> White <u>Size:</u> 170, 44 <u>Location:</u> 745, 475	Click

MessageSelection	Allows the user to select the category of message that they want to add to.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Items:</u> "Very Bad (Score <= 3) Bad (Score 4 - 8) Normal (Score 9 - 12) Good (Score 12 -16) Very Good (Score >= 17)" <u>Size:</u> 393, 31 <u>Location:</u> 754, 378	Click
FontSizeInput	Allows the user to input the font size that they want their message to be.	<u>Font:</u> Lucida Sans, 30pt, Demibold Roman <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Maximum:</u> 44 <u>Size:</u> 68, 51 <u>Location:</u> 1323, 441	Click & Type
MessageInput	Allows the user to input their custom message	<u>Font:</u> Lucida Sans, 30.75pt, Regular <u>MaxLength:</u> 128 <u>ForeColour:</u> White <u>Size:</u> 552, 50 <u>Location:</u> 754, 441	Click & Type
NoOption	Error message that tells the user they have not selected a message	<u>Text:</u> "Please select a message category."	Requirement met (No option

	category.	<u>Font:</u> Lucida Sans, 16pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 393, 25 <u>Location:</u> 860, 412	selected)
NoMessage	Error message that tells the user that they have not inputted a message	<u>Text:</u> "Please enter a message." <u>Font:</u> Lucida Sans, 16pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 281, 25 <u>Location:</u> 860, 412	Requirement met (No message entered)
AlreadyExists	Error message that tells the user that the message they entered already exists.	<u>Text:</u> "Message already exists." <u>Font:</u> Lucida Sans, 16pt, Demibold Roman <u>ForeColour:</u> Red <u>Size:</u> 278, 25 <u>Location:</u> 860, 412	Requirement met (Message already exists in text file)

Detailed Functionality

Component	Detailed Functionality
Settings_Load	This function is called when the Settings form is loaded, and it first deserialises the UserData file and sets the settings Booleans in Common_Variables with the current state of the Booleans in UserData and then hides all of the UI elements related to the admin settings & custom end messages. It then sets the border sizes for all of the buttons to 0 and sets the FlatStyle for MessageSelection to flat.

	It then checks the current state of the settings variables in Common_Variables and checks that option in the checklistbox if the setting is true. In the case of the isAdmin setting, it will also unhide all of the UI relating to admin settings & custom end messages.
volumeBar_ValueChanged	This function is called whenever the value on volumeBar is changed. It first checks if the value is an odd number and subtracts 1 from the value if it is, as user32.dll only lets you change the volume in increments of 2. After this the text on VolumeHeader is changed to show the updated volume number.
VolDown	Sends a message to user32.dll to decrement the volume by 2.
VolUp	Sends a message to user32.dll to increment the volume by 2.
ApplySettings	This function is triggered when CheckAns is clicked, as it serves as the apply settings button in this form. It first checks to see if the quiz is muted and will play click.wav if it is not. It then checks if the current value of the volumeBar is the same as the previous value, if it isn't, it will call the VolDown function 50 times. It does this because there is no way to get the current system volume without external plugins, so it sets the volume to 0 before incrementing it to the correct value so that the value the user enters is the value that the volume is set to, regardless of the current volume. If the value is not set to 0, the code then enters into a for loop that will run the VolUp function as many times as is equal to the current value of volumeBar, divided by 2 as user32.dll can only increment the volume by 2 each time. After the for loop & if statement exit the current value of volumeBar is written to the previous volume variable. It then checks if the checkboxes in the checkListBox are checked or not, if one is checked its corresponding Boolean in Common_Variables is set to be true, and it is set to false if its checkbox is not checked. If the admin mode checkbox is checked it will also unhide the admin & custom end message UI and hide them if it is unchecked.

	It then deserialises the UserData file and updates the settings Booleans in that file with the current state of the Booleans in Common_Variables. It then deletes the old file and recreates it by reserialising the UserData list with the updated UserData object.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
PreviewBtn_Click	This function is triggered whenever PreviewBtn is clicked, and will change the font size of SettingsHeader to match the value of FontSizeInput, it also changes the text of the header to match the text inputted into MessageInput.
SaveBtn_Click	When the SaveBtn is first clicked, its text is changed to “Are you sure?” and SaveFirstClick is set to false. After 3 seconds, if SaveFirstClick is still false it will change the text back to “Save” and change SaveFirstClick to true. If the button is clicked again, confirming that the user intends to save the current message, and SaveFirstClick is still false it will validate that there is text that has been inputted into SaveFirstClick and that a message category has been selected, showing the appropriate error message and set the colour of the MessageHeader to red if needed It will also remove any / from the input to prevent any errors. It will then read the text file for the message category selected and check to see if the inputted message already exists and will show an error if it does, change the header colour to red and blank the textbox. If the message is not a duplicate it will save the message to the end of the file, along with the font size. It will then change the text on SaveBtn to “Saved!” for one second to indicate that the message has been saved and blank MessageInput. It will then set SaveFirstClick to true.
MessageInput_Click	This is triggered by clicking the MessageInput textbox. It sets the colour of MessageHeader to white and hides all of the error messages, except for NoOption.
FileView_Click	This function is triggered by clicking FileView and it first checks if there is a message category selected and shows an error & turns MessageHeader’s ForeColour to red. If there is a category selected,

	it will set Common_Variables.FileToBeViewed to the file that matches the message category selected, before triggering the ShowTextView event and closing the form.
MessageSelection_Click	This function triggers whenever MessageSelection is clicked, and it sets the ForeColour of MessageHeader to white and hides the NoOption error.
DeleteUserDat_Click	When clicked for the first time, the text on the button will change to “Are you sure?” and the user will be required to click the button again to confirm that they want to delete the UserData file. If the button isn’t clicked again after 3 seconds, it will revert to its normal text. If it is clicked again before the 3 seconds have elapsed, it will delete the UserData.dat file and change the text on DeleteUsrDat to “Deleted!” for 500ms before calling the ShowWelcome event to take the user back to the welcome screen and then closes the form.
ResetLeaderboard_Click	When clicked for the first time, the text on the button will change to “Are you sure?” and the user will be required to click the button again to confirm that they want to delete the Leaderboard file. If the button isn’t clicked again after 3 seconds, it will revert to its normal text. If it is clicked again before the 3 seconds have elapsed, it will delete the Leaderboard.dat file and change the text on ResetLeaderboard to “Deleted!” for 500ms before reverting back to its normal text.
MessageInput_KeyUp	Every time a character is typed into MessageInput this function is triggered which checks to see it is a /, since all text files in my game that store data use a / as an indicator for splitting the line in an array, adding an extra one would cause the program to crash, as data is assigned to the wrong place.

Leaderboard

Leaderboard

File not found.

Descending

Username

Score

Lifetime Correct

Lifetime Incorrect



Components	Function	Properties	Trigger
LeaderboardHeader	Tells the user that they are on the Leaderboard page or shows them the preview of their custom end message.	<u>Font:</u> Lucida Sans, 44.25pt, Demibold Roman <u>Text:</u> "Leaderboard" <u>ForeColour:</u> White <u>Size:</u> 418, 74 <u>Location:</u> 20,30	N/A
Error	Tells the user that there is no LeaderboardData file to delete and prevents the program from crashing if there isn't one.	<u>Font:</u> Lucida Sans, 40pt, Demibold Roman <u>Text:</u> "File not found." <u>ForeColour:</u> White <u>Size:</u> 418, 74 <u>Location:</u> 20,30	No LeaderboardData file
DataGrid	Shows the data in LeaderboardData as a table that the users can sort	<u>ForeColour:</u> White <u>Size:</u> 1443, 601 <u>Location:</u> 30, 110	Leaderboard_Load
SortOrderBtn	Toggles between sorting the Leaderboard ascendingly or descendingly	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Ascending" / "Descending" <u>ForeColour:</u> White	Click

		<u>Size:</u> 340, 87 <u>Location:</u> 1207, 152	
SortNameBtn	Sorts the leaderboard by username when clicked	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Username" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1207, 265	Click
SortScoreBtn	Sorts the leaderboard by score when clicked	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Username" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1207, 378	Click
SortLCBtn	Sorts the leaderboard by lifetime correct when clicked	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Lifetime Correct"	Click

		<u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1207, 491	
SortLICBtn	Sorts the leaderboard by lifetime correct when clicked	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>Text:</u> "Lifetime Incorrect" <u>ForeColour:</u> White <u>Size:</u> 340, 87 <u>Location:</u> 1207, 604	Click

Detailed Functionality

Component	Detailed Functionality
Leaderboard_Load	This function is called when the form is loaded onto the panel and it first hides the error message, sets the row height to 35 and set the border size for SortScoreBtn to 5 to show that the leaderboard is sorted by score by default. It then checks if the LeaderboardData.dat file is present and unhides the error label if it isn't. If the file is present it will call LoadData.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
LoadData	When this function is called it will load LeaderboardData.dat into a list and then sort the list ascending or descending, by the option that the user selects (e.g. Username or score). It then sets the source for the DataGrid to the now sorted list and sets the headertext for the columns.
SortNameBtn_Click	When SortNameBtn is clicked it will set SortBy to Name, and set its own border size to 5, while setting the other options buttons to 0, to show that it is currently being sorted by username. It will then call LoadData.
SortScoreBtn_Click	When SortNameBtn is clicked it will set SortBy to Score, and set its own border size to 5, while setting the other options buttons to 0, to show that it is currently being sorted by score. It will then call LoadData.
SortLCBtn_Click	When SortNameBtn is clicked it will set SortBy to LC, and set its own border size to 5, while setting the other options buttons to 0, to show that it is currently being sorted by LifetimeCorrect. It will then call LoadData.
SortLICBtn_Click	When SortNameBtn is clicked it will set SortBy to LIC, and set its own border size to 5, while setting the other options buttons to 0, to show that it is currently being sorted by LifetimeIncorrect. It will then call LoadData.

SortOrderBtn_Click	If the quiz is ordered by ascending values, it will set the Leaderboard to order by descending values and change the buttons text to say “Descending”. If the quiz is ordered by descending values it will instead set the leaderboard to set by ascending values and set the buttons text to “Ascending” After this it will then call LoadData.
---------------------------	--

QuestionDesigner

Question Designer

Checkbox

▼

See Existing Questions

Preview

Save

Easy Question

Hard Question

Question

4

▲▼

First Option

☒ Answer

Second Option

☐

Third Option

☐

Fourth Option

☐

?

←

Question Designer

Multiple Choice

See Existing Questions

Preview

Save

Easy Question

Hard Question

Question

4

First Option

Second Option

Third Option

Fourth Option

Answer

?

←

Question Designer

True of False

▼

See Existing Questions

Preview

Save

Easy Question

Hard Question

Question

4

True

False

?

←

Question Designer

Fill in the Blank

See Existing Questions

Easy Question

Hard Question

Question

Answer

Alternative Answer 1

Alternative Answer 2

Alternative Answer 3

4

☒ Add answer

☒ Add answer

☒ Add answer



Components	Function	Properties	Trigger
Header	Tells the user that they are on the question designer page or shows them the question that they typed in.	<u>Font:</u> Lucida Sans, 44.25pt / SizeInput.Value, Demibold Roman <u>Text:</u> "Question Designer" / QuestionInput.Text <u>ForeColour:</u> White <u>Size:</u> 1274, 139 <u>Location:</u> 20,30	PreviewBtn_Click
questionTypeSelector	Allows the user to select the type of question that they want to make	<u>Font:</u> Lucida Sans, 21.75pt, Demibold Roman <u>Text:</u> "Please select a question type."/ "Checkbox"/ "Multiple Choice"/ "Fill in the Blank" / "True or False" <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Size:</u> 469, 41 <u>Location:</u> 42, 124	N/A
EasyBtn	Sets the difficulty for the question to easy.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White	EasyBtn_Click

		<u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Easy Question" <u>Size:</u> 349, 87 <u>Location:</u> 42, 250	
HardBtn	Sets the difficulty for the question to hard.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Hard Question" <u>Size:</u> 349, 87 <u>Location:</u> 42, 473	HardBtn_Click
ReturnBtn	Leaves the preview when clicked.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Return" <u>Size:</u> 349, 87 <u>Location:</u> 42, 250	ReturnBtn_Click

PreviewBtn	Allows the user to preview what their question will look like in game.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Preview" <u>Size:</u> 170, 44 <u>Location:</u> 32, 151	PreviewBtn_Click
TextView	Takes the user to the TextView form	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "See Existing Questions" <u>Size:</u> 272, 41 <u>Location:</u> 511, 104	TextView_Click
SaveBtn	Allows the user to save their custom question to the corresponding question type text file.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White	SaveBtn_Click

		<u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Save" / "Are you sure?" / "No Difficulty" / "No Question" / "No Answer" <u>Size:</u> 170, 44 <u>Location:</u> 211, 151	
TrueBtn	Sets the answer for a custom true or false question to true.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "True" <u>Size:</u> 330, 241 <u>Location:</u> 730, 319	TrueBtn_Click
FalseBtn	Sets the answer for a custom true or false question to false.	<u>Font:</u> Lucida Sans, 24pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "False" <u>Size:</u> 330, 241 <u>Location:</u> 1157, 319	FalseBtn_Click

QuestionInput	Allows the user to input their question.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>ForeColour:</u> Gainsboro/Black <u>Text:</u> "Question" <u>Size:</u> 680, 50 <u>Location:</u> 730, 199	Click textbox & typing
SizeInput	Allows the user to set the font size that their question should use.	<u>Font:</u> Lucida Sans, 28.75pt, Demibold Roman <u>ForeColour:</u> Black <u>Maximum:</u> 44 <u>Size:</u> 72, 53 <u>Location:</u> 1415, 196	Clicking & typing
FirstInput	Allows the user to input the first option / answer for their question.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>ForeColour:</u> Gainsboro/Black <u>Text:</u> "First Option" / "Answer" <u>Size:</u> 680, 50 <u>Location:</u> 730, 319	Click textbox & typing
SecondInput	Allows the user to input the first option / answer for their question.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular	Click textbox & typing

		<u>Text:</u> "Second Option" / "Alternative Answer 1" <u>ForeColour:</u> Gainsboro/Black <u>Size:</u> 680, 50 <u>Location:</u> 730, 383	
ThirdInput	Allows the user to input the first option / answer for their question.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>ForeColour:</u> Gainsboro/Black <u>Text:</u> ""First Option" / "Alternative Answer 2" <u>Size:</u> 680, 50 <u>Location:</u> 730, 447	Click textbox & typing
FourthInput	Allows the user to input the first option / answer for their question.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>ForeColour:</u> Gainsboro/Black <u>Text:</u> "False" <u>Size:</u> 680, 50 <u>Location:</u> 730, 510	Click textbox & typing
checkBox1	Allows the user to set the first option as an answer for the question or add an alternative answer.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>FlatStyle:</u> Standard <u>Cursor:</u> Hand	checkBox1_Check Changed

		<u>ForeColour:</u> Lime <u>Text:</u> "Answer" / "Add Answer" <u>Size:</u> 152, 50 <u>Location:</u> 1427, 319	
checkBox2	Allows the user to set the second option as an answer for the question or add an alternative answer.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>FlatStyle:</u> Standard <u>Cursor:</u> Hand <u>ForeColour:</u> Lime <u>Text:</u> "Answer" / "Add Answer" <u>Size:</u> 152, 50 <u>Location:</u> 1427, 383	checkBox2_Check Changed
checkBox3	Allows the user to set the third option as an answer for the question or add an alternative answer.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>FlatStyle:</u> Standard <u>Cursor:</u> Hand <u>ForeColour:</u> Lime <u>Text:</u> "Answer" / "Add Answer" <u>Size:</u> 152, 50 <u>Location:</u> 1427, 447	checkBox3_Check Changed
checkBox4	Allows the user to set the fourth	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman	checkBox4_Check Changed

	option as an answer for the question.	<u>FlatStyle</u> : Standard <u>Cursor</u> : Hand <u>ForeColour</u> : Lime <u>Text</u> : "Answer" <u>Size</u> : 152, 50 <u>Location</u> : 1427, 510	
radioButton1	Allows the user to set the first option as the answer for the Multiple Choice section.	<u>Font</u> : Lucida Sans, 18pt, Demibold Roman <u>FlatStyle</u> : Standard <u>Cursor</u> : Hand <u>ForeColour</u> : Lime <u>Text</u> : "Answer" <u>Size</u> : 152, 50 <u>Location</u> : 1427, 319	radioButton1_Check Changed
radioButton2	Allows the user to set the second option as the answer for the Multiple Choice section.	<u>Font</u> : Lucida Sans, 18pt, Demibold Roman <u>FlatStyle</u> : Standard <u>Cursor</u> : Hand <u>ForeColour</u> : Lime <u>Text</u> : "Answer" <u>Size</u> : 152, 50 <u>Location</u> : 1427, 383	radioButton2_Check Changed

radioButton3	Allows the user to set the third option as the answer for the Multiple Choice section.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>FlatStyle:</u> Standard <u>Cursor:</u> Hand <u>ForeColour:</u> Lime <u>Text:</u> "Answer" <u>Size:</u> 152, 50 <u>Location:</u> 1427, 447	radioButton3_Check Changed
radioButton4	Allows the user to set the fourth option as the answer for the Multiple Choice section.	<u>Font:</u> Lucida Sans, 18pt, Demibold Roman <u>FlatStyle:</u> Standard <u>Cursor:</u> Hand <u>ForeColour:</u> Lime <u>Text:</u> "Answer" <u>Size:</u> 152, 50 <u>Location:</u> 1427, 510	radioButton4_Check Changed
DummyTxtbox	Placeholder textbox that looks like the one in the Fill in the Blank section, shown when previewing Fill in the Blank questions	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans, 30.75pt, Regular <u>Size:</u> 1259, 164 <u>Location:</u> 32, 264	Previewing a Fill in the Blank question

Detailed Functionality

Component	Detailed Functionality
QuestionDesigner_Load	This function is called whenever the QuestionDesigner form is loaded. It first sets the visual properties for all of the UI buttons (e.g. setting FlatStyle to flat, setting the border colours for the Easy & HardBtn's to LimeGreen and Red respectively and setting all border sizes to 0). It then hides all of the question making UI, such as the radio buttons, checkboxes and textboxes.
TypeChanged	This function is called whenever the type of question being made is changed, it will check the current text of questionTypeSelector and configure the question making UI elements to fit the type of question selected, such as hiding the checkboxes if it is a Multiple Choice question, it also resets the text boxes to their default states. It also sets Common_Variables.CurrentPage to Designer + the current question type, so that the tutorial starts on the correct page. It also sets the _clear Booleans for all of the textboxes to true so the text that tells you which textbox is for what field is cleared when the textbox is cleared.
QuestionInput_Click	This function clears the QuestionInput and sets its ForeColour to black and sets question_clear to false.
FirstInput_Click	This function clears the FirstInput and sets its ForeColour to black and sets first_clear to false.
SecondInput_Click	This function clears SecondInput and sets its ForeColour to black and sets second_clear to false.
ThirdInput_Click	This function clears ThirdInput and sets its ForeColour to black and sets third_clear to false.
FourthInput_Click	This function clears FourthInput and sets its ForeColour to black and sets fourth_clear to false.
radioButton4_Check Changed	This function is triggered when radioButton4 is clicked and it checks if selected does not equal 4 & if the program is not currently previewing a question, if these requirements are fulfilled, it clears the text on radioButton1 – 3 and sets the text for radioButton4 to "Answer" and sets the answer variable to FourthInput.Text.

radioButton3_Check Changed	This function is triggered when radioButton3 is clicked and it checks if selected does not equal 3 & if the program is not currently previewing a question, if these requirements are fulfilled, it clears the text on radioButton1, 2 & 4 and sets the text for radioButton3 to "Answer" and sets the answer variable to ThirdInput.Text.
radioButton2_Check Changed	This function is triggered when radioButton2 is clicked and it checks if selected does not equal 2 & if the program is not currently previewing a question, if these requirements are fulfilled, it clears the text on radioButton1, 3 & 4 and sets the text for radioButton2 to "Answer" and sets the answer variable to SecondInput.Text.
radioButton1_Check Changed	This function is triggered when radioButton1 is clicked and it checks if selected does not equal 1 & if the program is not currently previewing a question, if these requirements are fulfilled, it clears the text on radioButton2 - 4 and sets the text for radioButton1 to "Answer" and sets the answer variable to FirstInput.Text.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
EasyBtn_Click	This function is triggered when EasyBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then sets the border size of EasyBtn to 5 and the border size of HardBtn to 0, before setting the difficulty variable to e.
HardBtn_Click	This function is triggered when HardBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then sets the border size of HardBtn to 5 and the border size of EasyBtn to 0, before setting the difficulty variable to h.
TextView_Click	This function is triggered when TextView is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks if questionTypeSelector has not changed from its default text and turns its ForeColour to red if it has not changed to tell the user that they need to select a question type.

	If questionTypeSelector has changed from its default text, its SelectedIndex is checked and Common_Variables.FileToBeViewed is changed to the text file for the question type selected. It then checks to see if the ShowTextView event is not null, and invokes the event if it is not, before closing the form.
SaveBtn_Click	This function is triggered when SaveBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. To save a question users are required to click this button twice within relatively quick succession to confirm that they want to save the question to the file, to minimise the chance of a user adding in a broken question. To make sure that the question is only saved on the second click & that it is clear to the user that they must click the button twice, when the button is clicked it checks if SaveFirstClick is set to true and if it is the buttons text is set to "Are you sure?" and its colour is set to white, after this a 3 second delay is used to set the button back to its default state and sets SaveFirstClick to true, if SaveFirstClick is not true and if the button is not currently displaying an error. If the button is displaying an error when the 3 seconds elapse, SaveFirstClick is set to true, but the buttons text is not changed, so the user can see what went wrong with their previous attempt to save a question. If the button is clicked for a second time during those 3 seconds it sets SaveFirstClick to false, which disables the reset after the 3 seconds are up. It then removes any / from the inputs and then checks the inputs to see if there are any errors in the input, such as not selecting a difficulty, not changing the default text/leaving a textbox blank or not providing an answer. If any of these errors are triggered SaveBtn.Text is changed to error text, such as "No Question", "No Answer" or "No Difficulty", and its ForeColour is set to red. If none of these errors have been triggered it saves the question to the text file for the appropriate question type, generating a 3 letter ID using IDGen so the question will not show up twice, checking all used IDs in the file and generating a new one for the question being saved if the previous ID is already used. If this does not succeed after 1.5 seconds, the program assumes that all possible IDs have been used, and quits trying to save the program, showing a MessageBox to the user telling them that there are too many questions in the file.

	After the question is saved to the text file, it sets SaveBtn.Text to "Saved!", clears the answer variable, waits one second and then sets SaveBtn.Text back to "Save". This function is set to be async so that the program does not freeze during the times that it is waiting for 3 or 1 seconds.
questionType Selector_Click	This function is triggered when SaveBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then sets its ForeColour to white.
IDGen	This function returns a 3-character string of random letters.
PreviewBtn_Click	This function is triggered when PreviewBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then hides all of the UI and Shows ReturnBtn and sets the preview Boolean to true, before changing the font size of Header to the number in SizeInput. It then checks the current text of questionTypeSelector and then rearranges the UI elements to make them resemble that question's UI, such as changing the text, size and position of the checkboxes to the text inputted into the First - FourthInputs.
ReturnBtn_Click	This function is triggered when PreviewBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then sets the preview Boolean to false before unhiding the question making UI, before hiding the ReturnBtn & DummyTxtbox UI elements. It resets the checkboxes and radioButtons to their normal settings and clears their text properties and moves the TrueBtns and FalseBtns to their normal location and changes their sizes back to their normal sizes, and resets Header back to its normal properties. After it resets all of the UI to its normal properties it then checks the text on questionTypeSelector and then configures the UI for making questions of that type.
checkBox1_Check Changed	This function is triggered when checkBox1 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks if the box is checked and that the current question type is not fill in the blank, if these requirements are met cb1ans is set to 1 and checkBox1's text to "Answer", if the box is

	unchecked but the question type is not Fill in the Blank, it sets both of these to string.Empty. If the box is checked and the question type is Fill in the Blank then it will show checkBox2 and show SecondInput, if it is unchecked and the question type is Fill in the Blank then it hides all of the checkboxes, unchecks them and hides SecondInput, ThirdInput & FourthInput.
checkBox2_Check Changed	This function is triggered when checkBox2 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks if the box is checked and that the current question type is not fill in the blank, if these requirements are met cb2ans is set to 2 and checkBox2's text to "Answer", if the box is unchecked but the question type is not Fill in the Blank, it sets both of these to string.Empty. If the box is checked and the question type is Fill in the Blank then it will show checkBox3 and show ThirdInput, if it is unchecked and the question type is Fill in the Blank then it hides checkBox3 & 4, unchecks them and hides ThirdInput & FourthInput.
checkBox3_Check Changed	This function is triggered when checkBox3 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks if the box is checked and that the current question type is not fill in the blank, if these requirements are met cb3ans is set to 3 and checkBox3's text to "Answer", if the box is unchecked but the question type is not Fill in the Blank, it sets both of these to string.Empty. If the box is checked and the question type is Fill in the Blank then it will show FourthInput, if it is unchecked and the question type is Fill in the Blank then it hides FourthInput.
checkBox4_Check Changed	This function is triggered when checkBox3 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks if the box is checked and that the current question type is not fill in the blank, if these requirements are met cb4ans is set to 4 and checkBox4's text to "Answer", if the box is unchecked but the question type is not Fill in the Blank, it sets both of these to string.Empty.

TrueBtn_Click	This function is triggered when TrueBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks the current state of the preview Boolean, if it is true the border colour is set to white, if it is false then the border colour is set to lime. It then sets the border size for TrueBtn to 5 and the FalseBtn's border size to 0.
FalseBtn_Click	This function is triggered when FalseBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks the current state of the preview Boolean, if it is true the border colour is set to white, if it is false then the border colour is set to red, after this if statement then sets the border size for TrueBtn to 5 and the FalseBtn's border size to 0.
ReplaceWhitespace	This function uses a Regex statement to detect any whitespace in the entered string and removes it.
FirstInput_KeyUp	When a character is typed into FirstInput this function is triggered which will remove any / in the textbox.
SecondInput_KeyUp	When a character is typed into SecondInput this function is triggered which will remove any / in the textbox.
ThirdInput_KeyUp	When a character is typed into ThirdInput this function is triggered which will remove any / in the textbox.
FourthInput_KeyUp	When a character is typed into FourthInput this function is triggered which will remove any / in the textbox.
QuestionInput_KeyUp	When a character is typed into QuestionInput this function is triggered which will remove any / in the textbox.

MultiChoice

{placeholder}

15

● {placeholder}



{placeholder}



● {placeholder}



● {placeholder}

Admin Mode: {answer}

Please select an answer.

Question 1 – Multiple Choice
Score: 0/20



Components	Function	Properties	Trigger
QuestionHeader	Shows the user the question that they have to answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, questions.Split[2], Demibold Roman / Regular <u>Text:</u> questions.Split[4] <u>ForeColour:</u> White <u>Size:</u> 1274, 165 <u>Location:</u> 20, 30	NewQuestion
radioButton1	Tells the user what the first option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Either of the 4 possible answers to the question; random each time. <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 173 <u>Location:</u> 43, 245	radioButton1_CheckChanged
radioButton2	Tells the user what the second option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Either of the 4 possible answers to the question; random each time.	radioButton2_CheckChanged

		<u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 705, 173 <u>Location:</u> 741, 245	
radioButton3	Tells the user what the third option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Either of the 4 possible answers to the question; random each time. <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 705, 173 <u>Location:</u> 741, 439	radioButton3_CheckChanged
radioButton4	Tells the user what the fourth option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Either of the 4 possible answers to the question; random each time. <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 173 <u>Location:</u> 43, 439	radioButton4_CheckChanged

AdminText	Shows the answer to the question if admin mode is enabled	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Admin Mode: " + questions.Split[1] <u>ForeColour:</u> White <u>Size:</u> 232, 37 <u>Location:</u> 20, 675	Common_Variables.isAdmin
NoAns	Tells the user that they have not provided an answer	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Please select an answer." <u>ForeColour:</u> Red <u>Size:</u> 405, 37 <u>Location:</u> 1167, 675	Requirement met (No answer given)
TimerLabel	Shows the time that the player has left to answer a timed question	<u>Font:</u> Lucida Sans / Comic Sans MS, 32, Demibold Roman / Regular <u>Text:</u> TimeLeft <u>ForeColour:</u> White <u>Size:</u> 76, 49 <u>Location:</u> 1496, 30	Requirement met (Timed Question)
TextBlock1	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock1.png <u>BackColour:</u> 82, 32, 129	Requirement met (ObstructedAnswer)

		<u>Size:</u> 193, 153 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	
TextBlock2	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock2.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 200, 317 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)
TextBlock3	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock3.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 185, 277 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)
TextBlock4	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock4.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 193, 153 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)

Detailed Functionality

Component	Detailed Functionality
MultiChoice_Load	This function is called whenever the MultiChoice form is loaded, and it hides all of the special event UI elements, such as the TimerLabel and TextBlockers. It then sets questions.guess to an empty string and hides NoAns before disabling timer1. It then checks to see if Common_Variables.isAdmin is false and if it is, it will hide AdminText. After this if statement, it will then call the NewQuestion function.
NewQuestion	This function is called at the end of MultiChoice_Load & AnsCheck, its job is to load the next question. It firsts checks to see if MultiChoice is currently visible on the form, to prevent forms from accidentally loading questions when they are not visible. If the form is currently visible, it will hide/disable all of the special event UI elements, hide NoAns and uncheck all of the radio buttons before setting the isReverse Boolean to false. It will then check the Common_Variables.dyslexia Boolean and set the font for all of the text on this form to Comic Sans MS if it is true, and Lucida Sans if it is false. It will then read a random line from multichoice.txt, in the ./assets/text/ folder, and split it into an array (questions.Split) along the / character. This file contains all of the questions and metadata for the Multiple Choice questions. It will then check to see if the selected question is valid, first checking if its difficulty matches the one selected at the start of the quiz and then checking to see if it has already been used by checking if its ID is in the list of used questions. If either of these checks fail, a new random line is chosen and the process repeats, if both of these checks are ok, the questions ID is added to the used questions list and exits the loop. Now that a question has been chosen, the question text is set as QuestionHeader's text, and its font is changed to either Lucida Sans or Comic Sans MS, depending on the current state of Common_Variables.dyslexia, with its font size being set to the font size stored in the question array.

	The options for the question will be randomly assigned to each radioButton, with each button checking the text of the ones before it to ensure that no option shows up twice. It will then call the ReadQuestion async function, if the user enabled the read questions option in Settings. AdminText is then updated to show the answer for the current answer, for use by an administrator with admin mode enabled, before running the SpecialEvent function and returning questions.Split.
ReadQuestion	This function will wait 1 second to allow for any sounds that are currently playing to finish, before playing the sound file that matches the current questions ID, stored in ./assets/audio/multi-tts, if one exists & the game isn't muted.
SpecialEvent	This function will pick random numbers, between 0 and 20, and 0 and 5 respectively. If the quiz difficulty is normal, a special event will be triggered if the first number is equal to 1 or 2, if its difficulty is hard, an event will trigger if the number is between 1 and 3. The second random number is used to decide which event will trigger. If the chosen event is SmallText, all of the radioButtons font sizes will be set to 8, and QuestionHeaders font size will be set to 12. If the event is LargeText, then the font size for the radioButtons will be set to 64 and 84 for QuestionHeader. If the selected event is TimedQuestion, it will show TimerLabel and call the TimedQuestion function. If the selected event is ObstructedAnswer, it will randomise the position of all of the TextBlockers and unhide them. If the selected event is ReverseText, it will reverse the text of all the radioButtons and the QuestionHeader and set the isReverse Boolean to true. Finally, if the selected event is Wingdings it will set QuestionHeaders font to Wingdings, with a small chance for the radioButtons font to be changed as well, which is slightly more likely if the quiz is in hard mode.
radioButton1_Check Changed	This function is triggered when radioButton1 is clicked it checks Common_Variables.muted is false and plays click.wav if it false, it will then set questions.guess to the current text of the radioButton.
radioButton2_Check Changed	This function is triggered when radioButton2 is clicked it checks Common_Variables.muted is false and plays click.wav if it false, it will then set questions.guess to the current text of the radioButton.

radioButton3_Check Changed	This function is triggered when radioButton3 is clicked it checks Common_Variables.muted is false and plays click.wav if it false, it will then set questions.guess to the current text of the radioButton.
radioButton4_Check Changed	This function is triggered when radioButton4 is clicked it checks Common_Variables.muted is false and plays click.wav if it false, it will then set questions.guess to the current text of the radioButton.
AnsCheck	When this function is called, it will first check to see if the MultiChoice form is currently visible on the panel, it does this to prevent AnsCheck from changing anything if it is erroneously called. It starts by hiding the TimerLabel and then reverses the guess stored in questions.guess if isReverse is true, so the special event will not affect scoring. If questions.guess equals the answer stored in questions.Split, it will add 1 to Common_Variables.LifetimeCorrect and Common_Variables.MultiLifeCorrect and then add either 0.5, 0.75 or 1 to Common_Variables.score, depending on the difficulty selected, playing ding.wav if the game is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will increase q_number by one and then invoke the UpdateQNum event, to update the UI with the current score and question number, before calling NewQuestion. If questions.guess is a blank string it will show NoAns. If the questions.guess does not equal the answer stored in questions.Split, it will add 1 to Common_Variables.LifetimeIncorrect and Common_Variables.MultiLifeIncorrect and then play wrong.wav if the game is not muted. It will check if Common_Variables.q_number modulus 5 is equal

	to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will increase q_number by one and then invoke the UpdateQNum event, to update the UI with the current score and question number, before calling NewQuestion.
TimedQuestion	When this function is called it will set TimeLeft to 15 if the game is in normal mode, and 10 if the game is in hard mode. It will then enable timer1.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
timer1_Tick	This function triggers every tick of timer1, it will first decrease TimeLeft by 1, before changing TimerLable.Text to show the updated time left, if TimeLeft is less than or equal to 0 it will run the same code as AnsCheck.

{placeholder}

15

☒ {placeholder}



☐ {placeholder}



☐ {placeholder}



☐ {placeholder}



Admin Mode: {answer}

Please select an answer.

Question 6 – Checkbox
Score: 0/20



Components	Function	Properties	Trigger
QuestionHeader	Shows the user the question that they have to answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, questions.Split[2], Demibold Roman / Regular <u>Text:</u> questions.Split[4] <u>ForeColour:</u> White <u>Size:</u> 1192, 165 <u>Location:</u> 20, 30	NewQuestion
checkBox1	Tells the user what the first option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> questions.Split[5] <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 48 <u>Location:</u> 43, 245	checkBox1_ CheckChanged
checkBox2	Tells the user what the second option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> questions.Split[6] <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 48	checkBox2_ CheckChanged

		<u>Location:</u> 741, 245	
checkBox3	Tells the user what the third option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> questions.Split[6] <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 48 <u>Location:</u> 741, 439	checkBox3_ CheckChanged
checkBox4	Tells the user what the fourth option is and allows them to select it as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> questions.Split[7] <u>ForeColour:</u> White <u>Cursor:</u> Hand <u>Size:</u> 670, 48 <u>Location:</u> 741, 439	checkBox4_ CheckChanged
AdminText	Shows the answer to the question if admin mode is enabled	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Admin Mode: " + questions.Split[1] <u>ForeColour:</u> White <u>Size:</u> 232, 37 <u>Location:</u> 20, 660	Common_ Variables.isAdmin

NoAns	Tells the user that they have not provided an answer	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Please select an answer." <u>ForeColour:</u> Red <u>Size:</u> 475, 42 <u>Location:</u> 1097, 660	Requirement met (No answer given)
TimerLabel	Shows the time that the player has left to answer a timed question	<u>Font:</u> Lucida Sans / Comic Sans MS, 32, Demibold Roman / Regular <u>Text:</u> TimeLeft <u>ForeColour:</u> White <u>Size:</u> 76, 49 <u>Location:</u> 1496, 30	Requirement met (Timed Question)
TextBlock1	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock1.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 193, 153 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)
TextBlock2	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock2.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 200, 317	Requirement met (ObstructedAnswer)

		<u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	
TextBlock3	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock3.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 185, 277 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)
TextBlock4	Blocks parts of the screen during an ObstructedAnswer special event	<u>Image:</u> TextBlock4.png <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 193, 153 <u>Location:</u> Random location anywhere from 42, 163 to 1038, 600.	Requirement met (ObstructedAnswer)

Component	Detailed Functionality
Checkbox_Load	This function is triggered when the form loads, and it hides all of the special event UI and hides AdminText if the user did not turn on admin mode in Settings and then calls the NewQuestion function.
checkBox1_Check Changed	This function is triggered when checkBox1 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks to see if checkBox1 is checked, and sets cb1ans to 1 if it is, and an empty string if it isn't.
checkBox2_Check Changed	This function is triggered when checkBox2 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks to see if checkBox2 is checked, and sets cb2ans to 1 if it is, and an empty string if it isn't.
checkBox3_Check Changed	This function is triggered when checkBox3 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks to see if checkBox3 is checked, and sets cb3ans to 1 if it is, and an empty string if it isn't.
checkBox4_Check Changed	This function is triggered when checkBox4 is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It then checks to see if checkBox4 is checked, and sets cb4ans to 1 if it is, and an empty string if it isn't.
NewQuestion	When this function is called it unchecks all of the checkboxes and hides all of the special event UI and disables the timer. It will then set all fonts for the UI elements in this form to Comic Sans MS if the user enabled the dyslexia accessibility mode in settings, if the user did not enable this setting, the font will be set to Lucida Sans instead. It will then read the checkbox.txt file stored in ./assests/txt/checkbox.txt and save it to an array, a random line

	is chosen from this array and is then split along the / character and saved to questions.Split. The difficulty for the chosen question is then checked against the chosen difficulty for the quiz to make sure it is appropriate for the difficulty selected and a new question is chosen if it is not. If the difficulty for the question is ok it then checks to see if the ID for the question is in the list of used questions, and a new question is chosen if it is. If the question has not been used its ID is added to the list of used questions. The function then sets the text of QuestionHeader to the question stored in questions.Split and sets the font to either Comic Sans MS or Lucida Sans, depending on whether or not the user enabled the dyslexia mode. The font size for the header is also set to the font size stored in questions.Split The checkboxes are then populated with the options for the question and UpdateAdmin is ran, followed by SpecialEvent and ReadQuestion if the user has enabled the setting to read out questions. After this it returns questions.Split.
AnsCheck	When this function is called, it will first check to see if the Checkbox form is currently visible on the panel, it does this to prevent AnsCheck from changing anything if it is erroneously called. It will then combine all of the cbans string together and then check if none of the checkBoxes are checked and it will show NoAns if none are checked. If any of the checkBoxes are checked, it will compare cbcombined to the answer stored in questions.Split and if it is a match, it will add either 0.5, 0.75 or 1 to Common_Variables.score, depending on the difficulty and will play ding.wav if the game is not muted and add 1 to Common_Variables.Lifetimecorrect & Common_Variables.CheckLifeCorrect . It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will increase q_number by one and then invoke the UpdateQNum event, to update the UI with the current score and question number, before calling NewQuestion.

	If the users guess is wrong then it will add one to <code>Common_Variables.CheckLifeIncorrect</code> and <code>Common_Variables.LifetimeIncorrect</code> before playing <code>wrong.wav</code> if the game is not muted. It will check if <code>Common_Variables.q_number</code> modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the <code>NextSection</code> event and then close the form. If that check fails, it will increase <code>q_number</code> by one and then invoke the <code>UpdateQNum</code> event, to update the UI with the current score and question number, before calling <code>NewQuestion</code>.
CloseForm	When the <code>BackBtn</code> is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
SpecialEvent	This function will pick random numbers, between 0 and 20, and 0 and 5 respectively. If the quiz difficulty is normal, a special event will be triggered if the first number is equal to 1 or 2, if its difficulty is hard, an event will trigger if the number is between 1 and 3. The second random number is used to decide which event will trigger. If the chosen event is <code>SmallText</code>, all of the <code>checkBoxes</code> font sizes will be set to 8, and <code>QuestionHeaders</code> font size will be set to 12. If the event is <code>LargeText</code>, then the font size for the <code>checkBoxes</code> will be set to 64 and 84 for <code>QuestionHeader</code>. If the selected event is <code>TimedQuestion</code>, it will show <code>TimerLabel</code> and call the <code>TimedQuestion</code> function. If the selected event is <code>ObstructedAnswer</code>, it will randomise the position of all of the <code>TextBlockers</code> and unhide them. If the selected event is <code>ReverseText</code>, it will reverse the text of all the <code>checkBoxes</code> and the <code>QuestionHeader</code> and set the <code>isReverse</code> Boolean to true. Finally, if the selected event is <code>Wingdings</code> it will set <code>QuestionHeaders</code> font to <code>Wingdings</code>, with a small chance for the <code>checkBoxes</code> font to be changed aswell, which is slightly more likely if the quiz is in hard mode.
timer1_Tick	This function triggers every tick of <code>timer1</code> , it will first decrease <code>TimeLeft</code> by 1, before changing <code>TimerLabel.Text</code> to show the updated time left, if <code>TimeLeft</code> is less than or equal to 0 it will run the same code as <code>AnsCheck</code> .
TimedQuestion	When this function is called it will set <code>TimeLeft</code> to 15 if the game

	is in normal mode, and 10 if the game is in hard mode. It will then enable timer1.
ReadQuestion	This function will wait 1 second to allow for any sounds that are currently playing to finish, before playing the sound file that matches the current questions ID, stored in ./assets/audio/check-tts, if one exists & the game isn't muted.
UpdateAdmin	This function updates the text shown on AdminText looking at the answer for the question and adding a checkboxes text to AdminText, if its number show up in the answer, e.g. if a 3 is present in the answer, checkBox3.Text will be added to AdminText.

TrueOrFalse

{placeholder}

15

True

False

Admin Mode: {answer}

Please select an answer.

Question 11 - True or False
Score: 0/20



Components	Function	Properties	Trigger
QuestionHeader	Shows the user the question that they have to answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, questions.Split[2], Demibold Roman / Regular <u>Text:</u> questions.Split[4] <u>ForeColour:</u> White <u>Size:</u> 1274, 165 <u>Location:</u> 20, 30	NewQuestion
TrueBtn	Allows the user to select True as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 44.25pt, Demibold Roman / Regular <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "True" <u>Size:</u> 700, 400 <u>Location:</u> 30, 226	TrueBtn_Click
FalseBtn	Allows the user to select False as their answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, 44.25pt, Demibold Roman / Regular <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White	FalseBtn_Click

		<u>BackColour</u> : 54, 15, 90 <u>Text</u> : "False" <u>Size</u> : 700, 400 <u>Location</u> : 860, 226	
AdminText	Shows the answer to the question if admin mode is enabled	<u>Font</u> : Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text</u> : "Admin Mode: " + questions.Split[1] <u>ForeColour</u> : White <u>Size</u> : 232, 37 <u>Location</u> : 20, 670	Common_Variables.isAdmin
NoAns	Tells the user that they have not provided an answer	<u>Font</u> : Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text</u> : "Please select an answer." <u>ForeColour</u> : Red <u>Size</u> : 475, 42 <u>Location</u> : 1097, 670	Requirement met (No answer given)
TimerLabel	Shows the time that the player has left to answer a timed question	<u>Font</u> : Lucida Sans / Comic Sans MS, 32, Demibold Roman / Regular <u>Text</u> : TimeLeft <u>ForeColour</u> : White <u>Size</u> : 76, 49	Requirement met (Timed Question)

		<u>Location:</u> 1484, 19	
--	--	---------------------------	--

Detailed Functionality

Component	Detailed Functionality
TrueOrFalse_Load	This function is triggered when the form loads, it configures the visual properties for the True and False buttons, such as setting border size to 0 and border colour to white. It then hides Admin text if the user has not enabled admin mode, hides NoAns and TimerLabel. It then calls NewQuestion.
NewQuestion	When this function is called it checks to see if the form is currently visible and if it is, it configures the visual properties for the True and False buttons, such as setting border size to 0 and border colour to white and then hides NoAns and TimerLabel. It will then read the trueorfalse.txt file stored in ./assests/txt/trueorfalse.txt and save it to an array, a random line is chosen from this array and is then split along the / character and saved to questions.Split. The difficulty for the chosen question is then checked against the chosen difficulty for the quiz to make sure it is appropriate for the difficulty selected and a new question is chosen if it is not. If the difficulty for the question is ok it then checks to see if the ID for the question is in the list of used questions, and a new question is chosen if it is. If the question has not been used its ID is added to the list of used questions. The function then sets the text of QuestionHeader to the question stored in questions.Split and sets the font to either Comic Sans MS or Lucida Sans, depending on whether or not the user enabled the dyslexia mode, this also applies to the answer buttons. The font size for the header is also set to the font size stored in questions.Split. It will then call the ReadQuestion async function, if the user enabled the read questions option in Settings. AdminText is then updated to show the answer for the current answer, for use by an administrator with admin mode enabled, before running the SpecialEvent function and returning questions.Split.

AnsCheck	When this function is called, it will first check to see if the form is currently visible, if it is it will then check to see if an answer has been provided by the user, showing NoAns if not. If the form is currently visible and an answer is given, it will then check if that answer matches the answer in questions.Split and add a score of either 0.5, 0.75 or 1, depending on the difficulty selected and add 1 to the lifetime correct counter for the entire quiz and True or False, if the answer was right. It then plays ding.wav if the quiz is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will increase q_number by one and then invoke the UpdateQNum event, to update the UI with the current score and question number, before calling NewQuestion. If the users guess was wrong, it adds one to the lifetime incorrect for the entire quiz and True or False and then plays wrong.wav if the game is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will increase q_number by one and then invoke the UpdateQNum event, to update the UI with the current score and question number, before calling NewQuestion.
SpecialEvent	This function will pick random numbers, between 0 and 20, and 0 and 5 respectively. If the quiz difficulty is normal, a special event will be triggered if the first number is equal to 1 or 2, if its difficulty is hard, an event will trigger if the number is between 1 and 3. The second random number is used to decide which event will trigger. If the chosen event is SmallText, QuestionHeader's font size will be set to 12. If the event is LargeText, QuestionHeader's font sizer will be set to 84. If the selected event is TimedQuestion, it will show TimerLabel and call the TimedQuestion function. If the selected event is ReverseText, it will reverse the text on QuestionHeader. Finally, if the selected event is Wingdings it will set QuestionHeaders font to Wingdings.

TrueBtn_Click	This function is triggered when TrueBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then set question.guess to "true", set its own border size to 5 and then set TrueBtns border size to 0.
FalseBtn_Click	This function is triggered when FalseBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then set question.guess to "false", set its own border size to 5 and then set TrueBtns border size to 0.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
ReadQuestion	This function will wait 1 second to allow for any sounds that are currently playing to finish, before playing the sound file that matches the current questions ID, stored in ./assets/audio/torf-tts, if one exists & the game isn't muted.
TimedQuestion	When this function is called it will set TimeLeft to 15 if the game is in normal mode, and 10 if the game is in hard mode. It will then enable timer1.
timer1_Tick	This function triggers every tick of timer1, it will first decrease TimeLeft by 1, before changing TimerLable.Text to show the updated time left, if TimeLeft is less than or equal to 0 it will run the same code as AnsCheck.

FillInTheBlank

{placeholder}

15

Enter Answer Here

Admin Mode: {answer}

Please select an answer.

Question 16 - Fill in the Blank

Score: 0/20



Components	Function	Properties	Trigger
QuestionHeader	Shows the user the question that they have to answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, questions.Split[2], Demibold Roman / Regular <u>Text:</u> questions.Split[2] <u>ForeColour:</u> White <u>Size:</u> 1274, 165 <u>Location:</u> 20, 30	NewQuestion
AnswerInput	Allows the user to answer the question by typing in their answer.	<u>BorderStyle:</u> None <u>Font:</u> Lucida Sans / Comic Sans MS, 30.75pt, Demibold Roman / Regular <u>Size:</u> 1259, 164 <u>Location:</u> 32, 264	Clicking & Typing
AdminText	Shows all possible answers to the question if the user has Admin mode enabled	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Admin Mode: " + questions.Split[1] <u>ForeColour:</u> White <u>Size:</u> 232, 37 <u>Location:</u> 32, 453	Common_Variables .isAdmin

NoAns	Tells the user that they have not provided an answer to the question	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Please type in an answer" <u>ForeColour:</u> Red <u>Size:</u> 475, 42 <u>Location:</u> 1078, 660	Requirement met (No answer given)
TimerLabel	Shows the time that the player has left to answer a timed question	<u>Font:</u> Lucida Sans / Comic Sans MS, 32, Demibold Roman / Regular <u>Text:</u> TimeLeft <u>ForeColour:</u> White <u>Size:</u> 76, 49 <u>Location:</u> 1484, 30	Requirement met (Timed Question)

Component	Detailed Functionality
FillInTheBlank_Load	This function is triggered when the form loads, it first hides AdminText if you have not enabled admin mode and then updates the question number UI before calling NewQuestion.
NewQuestion	When this function is called it hides the TimerLabel and NoAns before disabling the timer, it will then read the trueorfalse.txt file stored in ./assests/txt/trueorfalse.txt and save it to an array, a random line is chosen from this array and is then split along the / character and saved to questions.Split, it will also set RemoveTxt to true. The difficulty for the chosen question is then checked against the chosen difficulty for the quiz to make sure it is appropriate for the difficulty selected and a new question is chosen if it is not. If the difficulty for the question is ok it then checks to see if the ID for the question is in the list of used questions, and a new question is chosen if it is. If the question has not been used its ID is added to the list of used questions. The question text is then displayed on QuestionHeader and sets the font to either Comic Sans MS or Lucida Sans, depending on whether or not the user enabled the dyslexia mode, this also applies to the AnswerInput. The font size for the header is also set to the font size stored in questions.Split. The AdminText is then updated to show all possible answers for this question, AnswerInputs text is changed to read "Enter Answer Here" and ReadQuestion is ran if the user had enabled the option to read questions, it will then run SpecialEvent before returning question.Split.
SpecialEvent	This function will pick random numbers, between 0 and 20, and 0 and 3 respectively. If the quiz difficulty is normal, a special event will be triggered if the first number is equal to 1 or 2, if its difficulty is hard, an event will trigger if the number is between 1 and 3. The second random number is used to decide which event will trigger. If the chosen event is SmallText,

	QuestionHeader's font size will be set to 12 and AnswerInput's will be set to 8. If the event is LargeText, QuestionHeader's font size will be set to 84 and AnswerInputs will be set to 64. If the selected event is TimedQuestion, it will show TimerLabel and call the TimedQuestion function. If the selected event is ReverseText, it will reverse the text on QuestionHeader.
AnsCheck	When this function is called and the form is visible, it will set the correct Boolean to false, before saving all possible answers to an array, splitting them along the character. It will then run AnswerInput.Text through ReplaceWhitespace and convert it to all lowercase. It will then check all values stored on the answer array to see if it matches the updated answer given in AnswerInput.Text, if the answer given matches any of the stored answers, correct is set to true. If correct is true it will then add a score of either 0.5, 0.75 or 1, depending on the difficulty selected and add 1 to the lifetime correct counter for the entire quiz and Fill in the Blank, if the answer was right. It then plays ding.wav if the quiz is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will add 1 to the question number and invoke the UpdateQNum event before calling NewQuestion. If correct is false, If correct is true it will then add a score of either 0.5, 0.75 or 1, depending on the difficulty selected and add 1 to the lifetime correct counter for the entire quiz and Fill in the Blank, if the answer was right. It then plays ding.wav if the quiz is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will add 1 to the question number and invoke the UpdateQNum event before calling NewQuestion.
ReadQuestion	This function will wait 1 second to allow for any sounds that are currently playing to finish, before playing the sound file that matches the current questions ID, stored in ./assets/audio/blank-tts, if one exists & the game isn't muted.

CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
ReplaceWhitespace	This function uses a Regex statement to detect any whitespace in the entered string and removes it.
TimedQuestion	When this function is called it will set TimeLeft to 15 if the game is in normal mode, and 10 if the game is in hard mode. It will then enable timer1.
timer1_Tick	This function triggers every tick of timer1, it will first decrease TimeLeft by 1, before changing TimerLabel.Text to show the updated time left, if TimeLeft is less than or equal to 0 it will run the same code as AnsCheck.
AnswerInput_Click	This function will trigger when AnswerInput is clicked, it will check the current state of RemoveTxt, and blank the textbox if it is true, to remove the instructional text from the input when the user goes to type their answer in.

DragAndDrop

Components	Function	Properties	Trigger
QuestionHeader	Shows the user the question that they have to answer.	<u>Font:</u> Lucida Sans / Comic Sans MS, questions.Split[1], Demibold Roman / Regular <u>Text:</u> questions.Split[3] <u>ForeColour:</u> White <u>Size:</u> 1274, 165 <u>Location:</u> 20, 30	NewQuestion
AnsHeader1	Tells the user of the image that is required to go in the picturebox below.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Text the corresponds to required answer <u>ForeColour:</u> White <u>Size:</u> 270, 49 <u>Location:</u> 869, 266	NewQuestion
AnsHeader2	Tells the user of the image that is required to go in the picturebox below.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Text the corresponds to required answer <u>ForeColour:</u> White <u>Size:</u> 270, 49	NewQuestion

		<u>Location:</u> 317, 266	
AnsHeader3	Tells the user of the image that is required to go in the picturebox below.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Text the corresponds to required answer <u>ForeColour:</u> White <u>Size:</u> 270, 49 <u>Location:</u> 23, 266	NewQuestion
AnsHeader4	Tells the user of the image that is required to go in the picturebox below.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> Text the corresponds to required answer <u>ForeColour:</u> White <u>Size:</u> 270, 49 <u>Location:</u> 592, 266	NewQuestion
TimerLabel	Shows the time that the player has left to answer a timed question	<u>Font:</u> Lucida Sans / Comic Sans MS, 32, Demibold Roman / Regular <u>Text:</u> TimeLeft <u>ForeColour:</u> White <u>Size:</u> 76, 49 <u>Location:</u> 12, 650	Requirement met (Timed Question)

NoAns	Tells the user that they have not provided an answer to the question	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "Please answer the question" <u>ForeColour:</u> Red <u>Size:</u> 467, 37 <u>Location:</u> 790, 660	Requirement met (No answer given)
pictureBox1	Shows the user one of the images to answer the question with.	<u>Image:</u> One of the 4 images for the question; randomised each time <u>Cursor:</u> Hand <u>BackColour:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox <u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> StretchImage <u>Tag:</u> Tag stored in question file that corresponds to image on pictureBox	NewQuestion
pictureBox2	Shows the user one of the images to answer the question with.	<u>Image:</u> One of the 4 images for the question; randomised each time	NewQuestion

		<u>Cursor:</u> Hand <u>BackColour:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox <u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> StretchImage <u>Tag:</u> Tag stored in question file that corresponds to image on pictureBox	
pictureBox3	Shows the user one of the images to answer the question with.	<u>Image:</u> One of the 4 images for the question; randomised each time <u>Cursor:</u> Hand <u>BackColour:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox <u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> StretchImage	NewQuestion

		<u>Tag</u> : Tag stored in question file that corresponds to image on pictureBox	
pictureBox4	Shows the user one of the images to answer the question with.	<u>Image</u> : One of the 4 images for the question; randomised each time <u>Cursor</u> : Hand <u>BackColour</u> : 54, 15, 90 <u>Size</u> : Set to size stored in the question text file which corresponds to image for pictureBox <u>Location</u> : Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode</u> : StretchImage <u>Tag</u> : Tag stored in question file that corresponds to image on pictureBox	NewQuestion
pictureBox5	Allows the user to drop an image on to this box to answer the question	<u>Image</u> : One of the 4 images for the question; randomised each time <u>Cursor</u> : Hand <u>BackColour</u> : 54, 15, 90 <u>Size</u> : Set to size stored in the question text file which corresponds to image for pictureBox	NewQuestion

		<u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> Zoom	
pictureBox6	Allows the user to drop an image on to this box to answer the question	<u>Image:</u> One of the 4 images for the question; randomised each time <u>Cursor:</u> Hand <u>BackColor:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox <u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> Zoom	NewQuestion
pictureBox7	Allows the user to drop an image on to this box to answer the question	<u>Image:</u> One of the 4 images for the question; randomised each time <u>Cursor:</u> Hand <u>BackColor:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox	NewQuestion

		<u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> Zoom	
pictureBox8	Allows the user to drop an image on to this box to answer the question	<u>Image:</u> One of the 4 images for the question; randomised each time <u>Cursor:</u> Hand <u>BackColour:</u> 54, 15, 90 <u>Size:</u> Set to size stored in the question text file which corresponds to image for pictureBox <u>Location:</u> Set to location stored in the question text file which corresponds to image for pictureBox <u>SizeMode:</u> Zoom	NewQuestion
AdminTxt1	Tells the user what image needs to go above it if the user has Admin mode enabled.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "1", "2", "3", "4" <u>ForeColour:</u> White <u>Size:</u> 256, 37 <u>Location:</u> 876, 577	Common_Variables .isAdmin

AdminTxt2	Tells the user what image needs to go above it if the user has Admin mode enabled.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "1", "2", "3", "4" <u>ForeColour:</u> White <u>Size:</u> 256, 37 <u>Location:</u> 317, 577	Common_Variables .isAdmin
AdminTxt3	Tells the user what image needs to go above it if the user has Admin mode enabled.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "1", "2", "3", "4" <u>ForeColour:</u> White <u>Size:</u> 256, 37 <u>Location:</u> 31, 577	Common_Variables .isAdmin
AdminTxt4	Tells the user what image needs to go above it if the user has Admin mode enabled.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "1", "2", "3", "4" <u>ForeColour:</u> White <u>Size:</u> 256, 37 <u>Location:</u> 599, 577	Common_Variables .isAdmin
One	Tells a user that is in Admin mode that the image next to it goes in the box marked 1.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "1"	Common_Variables .isAdmin

		<u>ForeColour:</u> White <u>Size:</u> 37, 37 <u>Location:</u> 1235, 9	
Two	Tells a user that is in Admin mode that the image next to it goes in the box marked 2.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "2" <u>ForeColour:</u> White <u>Size:</u> 37, 37 <u>Location:</u> 1235, 162	Common_Variables .isAdmin
Three	Tells a user that is in Admin mode that the image next to it goes in the box marked 3.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "3" <u>ForeColour:</u> White <u>Size:</u> 37, 37 <u>Location:</u> 1235, 330	Common_Variables .isAdmin
Four	Tells a user that is in Admin mode that the image next to it goes in the box marked 4.	<u>Font:</u> Lucida Sans / Comic Sans MS, 24, Demibold Roman / Regular <u>Text:</u> "4" <u>ForeColour:</u> White <u>Size:</u> 37, 37 <u>Location:</u> 1235, 507	Common_Variables .isAdmin

Component	Detailed Functionality
DragAndDrop_Load	This function is triggered when the form loads, it first hides AdminText if you have not enabled admin mode and then updates the question number UI before calling NewQuestion.
NewQuestion	When this function is called it will first hide NoAns & TimerLabel and then disables the timer. It will then read the draganddrop.txt file stored in ./assests/txt/draganddrop.txt and save it to an array, a random line is chosen from this array and is then split along the / character and saved to questions.Split. All of the pictureBoxes are then set to allow DragDrop, the first 4 have their images set to null and the last 4 have their images set to UI_Purple.png. Then all of the pictureBoxes tags are set to null. After this all of the text on this form has its font set to Comic Sans MS if Common_Variables.isAdmin is true. The difficulty for the chosen question is then checked against the chosen difficulty for the quiz to make sure it is appropriate for the difficulty selected and a new question is chosen if it is not. If the difficulty for the question is ok it then checks to see if the ID for the question is in the list of used questions, and a new question is chosen if it is. If the question has not been used its ID is added to the list of used questions. The options for the question are then randomly distributed between the 4 pictureBoxes, each option has Location and Size metadata that is applied to the pictureBox that it is being shown on, so each option will be guaranteed to show up correctly. The header text for the option will be applied to the AnsHeader above the pictureBox and the number of the option selected (1, 2, 3 or 4) will be set as the AdminTxt text. The tag for the picture box will also be set to the file name for the image selected. This will repeat for all 4 pictureBoxes, with each pictureBox checking the ones before it to make sure that no option shows up twice. The answer variable is then set to the 4 pictureBox tags, and the text for QuestionHeader is set to questions.Split[3] and its font is changed to either Lucida Sans or Comic Sans MS, depending on the current state of Common_Variables.dyslexia,

	with its font size being set to the font size stored in the question array. It will then call the ReadQuestion event if the read question setting is enabled in the settings form. It will then call the SpecialEvent function.
pictureBox1_Mouse Down	This function allows the image stored in pictureBox1 to be copied to any pictureBox that it dragged onto.
pictureBox2_Mouse Down	This function allows the image stored in pictureBox2 to be copied to any pictureBox that it dragged onto.
pictureBox3_Mouse Down	This function allows the image stored in pictureBox3 to be copied to any pictureBox that it dragged onto.
pictureBox4_Mouse Down	This function allows the image stored in pictureBox4 to be copied to any pictureBox that it dragged onto.
pictureBox5_DragDrop	This function is called when an image is dragged onto pictureBox5. It will set the image that has been dragged onto the pictureBox as its image. It will also set the tag of the pictureBox that has been dragged from as the tag for the pictureBox being dragged to.
pictureBox6_DragDrop	This function is called when an image is dragged onto pictureBox6. It will set the image that has been dragged onto the pictureBox as its image. It will also set the tag of the pictureBox that has been dragged from as the tag for the pictureBox being dragged to.
pictureBox7_DragDrop	This function is called when an image is dragged onto pictureBox7. It will set the image that has been dragged onto the pictureBox as its image. It will also set the tag of the pictureBox that has been dragged from as the tag for the pictureBox being dragged to.
pictureBox8_DragDrop	This function is called when an image is dragged onto pictureBox8. It will set the image that has been dragged onto the pictureBox as its image. It will also set the tag of the pictureBox that has been dragged from as the tag for the

	pictureBox being dragged to.
pictureBox5_DragEnter	This will change the mouse cursor to a copy cursor when the user enters the pictureBox while dragging a picture.
pictureBox6_DragEnter	This will change the mouse cursor to a copy cursor when the user enters the pictureBox while dragging a picture.
pictureBox7_DragEnter	This will change the mouse cursor to a copy cursor when the user enters the pictureBox while dragging a picture.
pictureBox8_DragEnter	This will change the mouse cursor to a copy cursor when the user enters the pictureBox while dragging a picture.
AnsCheck	If the current form is visible it will hide the TimerLabel. It will then check to see if the tags for pictureBoxes 5-8 combined are the same as the answer variable. If it does equal the answer variable it will then add a score of either 0.5, 0.75 or 1, depending on the difficulty selected and add 1 to the lifetime correct counter for the entire quiz and Drag and Drop, if the answer was right. It then plays ding.wav if the quiz is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will add 1 to the question number and invoke the UpdateQNum event before calling NewQuestion. If any of the tags are null then it will show NoAns. If it does not equal answer, If correct is true it will then add a score of either 0.5, 0.75 or 1, depending on the difficulty selected and add 1 to the lifetime correct counter for the entire quiz and Drag and Drop, if the answer was right. It then plays ding.wav if the quiz is not muted. It will check if Common_Variables.q_number modulus 5 is equal to 0, or if it is over 19, in which case it adds 1 to the question number, invoke the NextSection event and then close the form. If that check fails, it will add 1 to the question number and invoke the UpdateQNum event before calling NewQuestion.

CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
SpecialEvent	This function will pick random numbers, between 1 and 20, and 0 and 4 respectively. If the quiz difficulty is normal, a special event will be triggered if the first number is equal to 1 or 2, if its difficulty is hard, an event will trigger if the number is between 1 and 3. The second random number is used to decide which event will trigger. If the chosen event is SmallText, QuestionHeader's font size will be set to 12 and the AnswerHeader's will be set to 8. If the event is LargeText, QuestionHeader's font size will be set to 84 and AnswerHeader's will be set to 64. If the selected event is TimedQuestion, it will show TimerLabel and call the TimedQuestion function. If the selected event is ReverseText, it will reverse the text on QuestionHeader and the AnswerHeaders. If the selected event is Wingdings then the font for QuestionHeader is set to Wingdings.
TimedQuestion	When this function is called it will set TimeLeft to 15 if the game is in normal mode, and 10 if the game is in hard mode. It will then enable timer1.
timer1_Tick	This function triggers every tick of timer1, it will first decrease TimeLeft by 1, before changing TimerLabel.Text to show the updated time left, if TimeLeft is less than or equal to 0 it will run the same code as AnsCheck.
ReadQuestion	This function will wait 1 second to allow for any sounds that are currently playing to finish, before playing the sound file that matches the current questions ID, stored in ./assets/audio/drop-tts, if one exists & the game isn't muted.

EndScreen

{placeholder}



Your final score:

15

out of 20!

Leaderboard

Return to Lobby

Components	Function	Properties	Trigger
EndMessage	Shows the user an end message, depending on their score.	<u>Font:</u> Lucida Sans, Font size for selected message, Demibold Roman <u>Text:</u> Random message depending on final score. <u>ForeColour:</u> White <u>Size:</u> 1256, 165 <u>Location:</u> 20, 30	EndScreen_Load
ScoreMessage1	Shows the first part of the score message.	<u>Font:</u> Lucida Sans, 32.25, Demibold Roman <u>Text:</u> "Your final score:" <u>ForeColour:</u> White <u>Size:</u> 389, 49 <u>Location:</u> 1045, 181	N/A
ScoreMessage2	Shows the second part of the score message.	<u>Font:</u> Lucida Sans, 32.25, Demibold Roman <u>Text:</u> "our of 20!" <u>ForeColour:</u> White <u>Size:</u> 236, 49 <u>Location:</u> 1113, 489	N/A
EndScoreTxt	Shows the user their final score	<u>Font:</u> Lucida Sans, 72, Demibold Roman <u>Text:</u> Common_Variables.score	EndScreen_Load

		<u>ForeColour:</u> DarkRed, Red, Orange, Lime, LimeGreen <u>Size:</u> 434, 259 <u>Location:</u> 1016, 230 <u>TextAlign:</u> MiddleCentre	
Gold	Shows if the user achieved a gold trophy.	<u>Image:</u> GoldTrophyBlack.png / GoldTrophy <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 178, 256 <u>Location:</u> 336, 205 <u>SizeMode:</u> StretchImage	Common_Variables .score > 16
Silver	Shows if the user achieved a silver trophy.	<u>Image:</u> SilverTrophyBlack.png / SilverTrophy <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 121, 244 <u>Location:</u> 606, 217 <u>SizeMode:</u> StretchImage	Common_Variables .score > 12
Bronze	Shows if the user achieved a bronze trophy.	<u>Image:</u> CopperTrophyBlack.png / CopperTrophy <u>BackColour:</u> 82, 32, 129 <u>Size:</u> 143, 220 <u>Location:</u> 106, 241 <u>SizeMode:</u> StretchImage	Common_Variables .score > 8

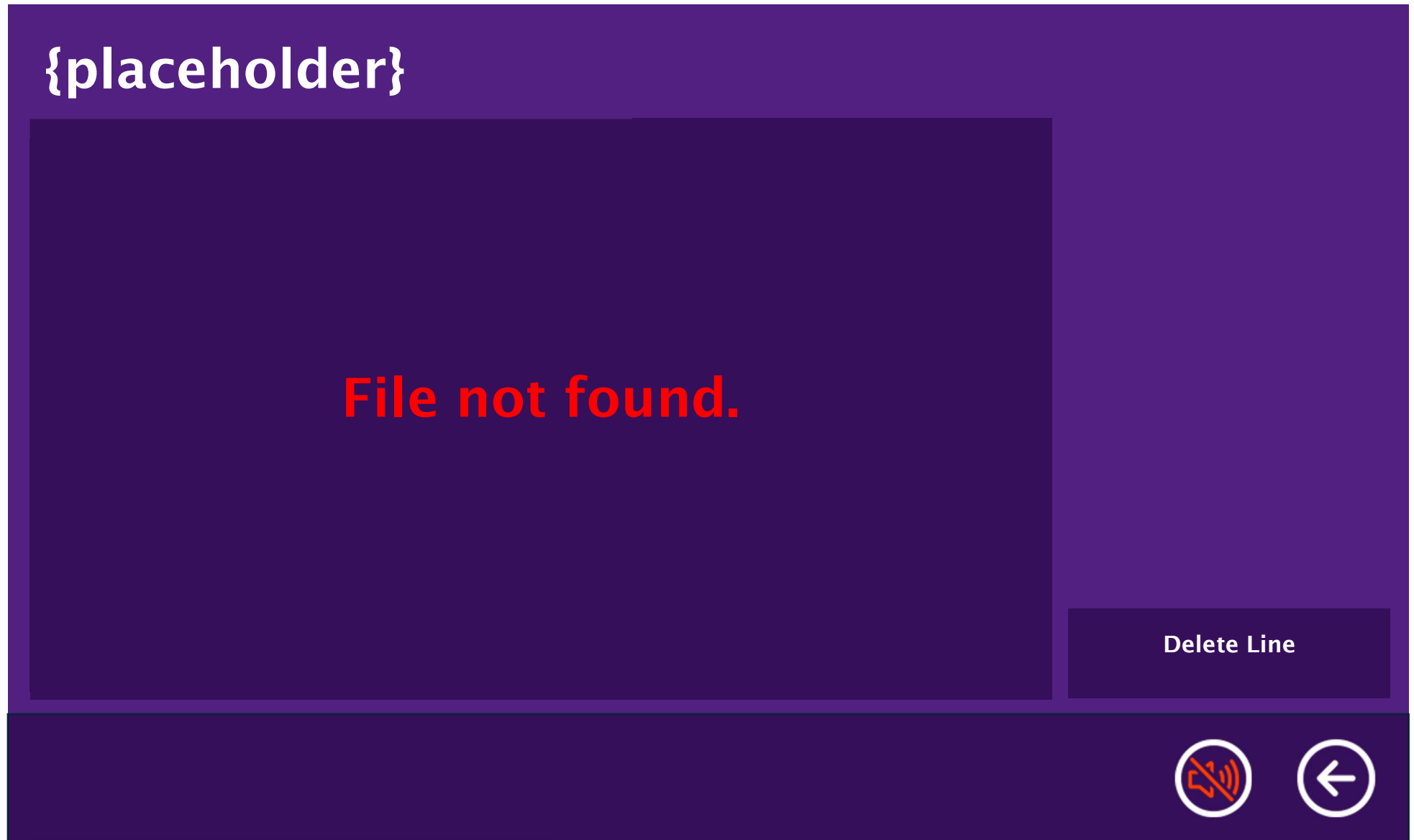
progressBar1	Shows the users the score they got out of 20, on a progressbar	<u>Style:</u> Continuous <u>Maximum:</u> 80 <u>Size:</u> 621, 23 <u>Location:</u> 106, 513	EndScreen_Load
Zero	Shows the user that the left of the progressBar is 0.	<u>Font:</u> Lucida Sans, 15.75, Demibold Roman <u>Text:</u> "0" <u>ForeColour:</u> DarkRed <u>Size:</u> 23, 23 <u>Location:</u> 100, 548	N/A
Ten	Shows the user that the middle of the progressBar is 10.	<u>Font:</u> Lucida Sans, 15.75, Demibold Roman <u>Text:</u> "10" <u>ForeColour:</u> Orange <u>Size:</u> 36, 23 <u>Location:</u> 398, 548	N/A
Twenty	Shows the user that the end of the progressBar is 20.	<u>Font:</u> Lucida Sans, 15.75, Demibold Roman <u>Text:</u> "20" <u>ForeColour:</u> LimeGreen <u>Size:</u> 36, 23 <u>Location:</u> 710, 548	N/A
GoToLeaderboard	Allows the user to view the leaderboard.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat	Click

		<u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Leaderboard" <u>Size:</u> 340, 87 <u>Location:</u> 889, 627	
ReturnLobby	Allows the user to view the leaderboard.	<u>Font:</u> Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle:</u> Flat <u>Cursor:</u> Hand <u>ForeColour:</u> White <u>BackColour:</u> 54, 15, 90 <u>Text:</u> "Return to Lobby" <u>Size:</u> 340, 87 <u>Location:</u> 1233, 627	Click

Component	Detailed Functionality
EndScreen_Load	This function is triggered when the form loads, it first checks to see if a LeaderboardData file exists, and creates one with the current users LifetimeCorrect, LifetimeIncorrect, score and username. If a LeaderboardData file already exists it will load the current data into a list and add the new data to the end of it, before saving that to a new LeaderboardData file. It will then update the users game statistics in the UserData file. It will then set the value of progressBar1 to 4 times the current score, as there are 80 possible scores and the progressBar's max value is 80, so to map the score to the progressBar, the score is multiplied by 4. If the score is below 3, it will then read a line from VeryBadEnd.txt and set EndMessage.Text to that value and set its font size to the font size for that message. It will then set EndScoreTxt's colour to Red, unless the score is 0, then it is set to DarkRed. If the score is above 3 but below 9, it will then read a line from BadEnd.txt and set EndMessage.Text to that value and set its font size to the font size for that message. It will then set EndScoreTxt's colour to OrangeRed. If the score is above 8 but below 13, it will then read a line from NormalEnd.txt and set EndMessage.Text to that value and set its font size to the font size for that message. It will then set EndScoreTxt's colour to Orange. If the score is above 12 but below 17, it will then read a line from GoodEnd.txt and set EndMessage.Text to that value and set its font size to the font size for that message. It will then set EndScoreTxt's colour to Lime. If the score is above 16, it will then read a line from VeryGoodEnd.txt and set EndMessage.Text to that value and set its font size to the font size for that message. It will then set EndScoreTxt's colour to LimeGreen.

GoToLeaderboard_Click	This function is triggered when GoToLeaderboard is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then reset the score and question number variables and then show the leaderboard before closing the form.
ReturnLobby_Click	This function is triggered when ReturnLobby is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then reset the score and question number variables and then show the lobby before closing the form.

TextViewer



Components	Function	Properties	Trigger
dataGridView1	Displays the contents of the text file being viewed	<u>ForeColor</u> : White <u>BackColor</u> : 54, 15, 90 <u>ReadOnly</u> : White <u>Size</u> : 1443, 601 <u>Location</u> : 30, 110	LoadGrid
ViewingHeader	Tells the user the name of the text file that is being viewed	<u>Font</u> : Lucida Sans, 44.25pt, Demibold Roman <u>Text</u> : Common_Variables .FileToBeViewed <u>ForeColor</u> : White <u>Size</u> : 964, 77 <u>Location</u> : 20, 30	TextViewer_Load
DeleteBtn	Allows the user to delete a line from the file they are viewing	<u>Font</u> : Lucida Sans, 15.75pt, Demibold Roman <u>FlatStyle</u> : Flat <u>Cursor</u> : Hand <u>ForeColor</u> : White <u>BackColor</u> : 54, 15, 90 <u>Text</u> : "Delete Line" <u>Size</u> : 340, 87 <u>Location</u> : 1223, 624	Click

Component	Detailed Functionality
EndScreen_Load	This function is called when the form is loaded, and it sets the text for ViewingHeader to the current string stored in Common_Variables.FileToBeViewed and then calls LoadGrid.
LoadGrid	This function will load the file that is in the location stored in Common_Variables.FileToBeViewed, it will change the header of the columns depending on the type of text file that is being loaded it (e.g. end message file or question file). It will also check if there is only one row left in the DataGrid, as this means all questions have been deleted, if there is only one row left it will hide the DeleteBtn to prevent the game from crashing.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
DeleteBtn_Click	When the DeleteBtn is first clicked, its text is changed to "Are you sure?" and DeleteFirstClick is set to false. After 3 seconds, if DeleteFirstClick is still false it will change the text back to "Delete Line" and change DeleteFirstClick to true. If the button is clicked for a second time in that 3 second interval DeleteFirstClick is set to true, and the current text file being shown is loaded into a temporary array and the file is deleted. The question that the user deleted is then removed from that list and the updated list is then saved as a text file with the same name in the same location as the old one. It will then call LoadGrid.

Tutorial

{placeholder}

{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}
{This is placeholder text}

{placeholder}



Question 5 – Drag and Drop
Score: 0/20

Components	Function	Properties	Trigger
pictureBox1	Shows an image of the item being described by the tutorial.	<u>Image</u> : Depends on tutorial selected <u>Size</u> : Depends on tutorial selected <u>Location</u> : Depends on tutorial selected <u>SizeMode</u> : Zoom	UpdateForm
GoBackBtn	Goes back to the previous tutorial page when clicked.	<u>Image</u> : BackButton64.png <u>Cursor</u> : Hand <u>BackColor</u> : 54, 15, 90 <u>Size</u> : 64, 64 <u>Location</u> : 1508, 187	GoBackBtn_Click
GoFowardBtn	Goes foward to the next tutorial page when clicked.	<u>Image</u> : GoFoward.png <u>Cursor</u> : Hand <u>BackColor</u> : 54, 15, 90 <u>Size</u> : 64, 64 <u>Location</u> : 1508, 538	GoFowardBtn_Click
TutorialTxt	Shows the user the text for the tutorial page they are on.	<u>Font</u> : Lucida Sans, 15.75pt, Demibold Roman <u>Text</u> : Depends on the tutorial page selected. <u>ForeColour</u> : White <u>Size</u> : 649, 256	UpdateForm

		<u>Location:</u> 30, 255	
Header	Shows the user the title for the tutorial page selected.	<u>Font:</u> Lucida Sans, 39.75pt Demibold Roman <u>Text:</u> Title for tutorial page selected <u>ForeColour:</u> White <u>Size:</u> 1274, 71 <u>Location:</u> 20,30	UpdateForm

Detailed Functionality

Component	Detailed Functionality
Tutorial_Load	This function is called when the form is loaded, and it calls the UpdateForm function.
GoBack_Click	This function is triggered when GoBackBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then find the current page that the tutorial is on and subtract 1 from the index of the array that stores what pages can be displayed and stores the value at that point of the array in Common_Variables.PageToBeViewed. If the subtracted index is less than 0, it will set the index to the length of the array, so that if you try to go back on the 1st page, it will loop back around to the last page.
GoFoward_Click	This function is triggered when GoFowardBtn is clicked it checks Common_Variables.muted is false and plays click.wav if it false. It will then find the current page that the tutorial is on and add 1 to the index of the array that stores what pages can be displayed and sets Common_Variables.PageToBeViewed to the value stored at the updated array index. If the added index is more than the length of the array it will set the index to 0, so that if you try to go forward on the last page, it

	will loop back around to the 1 st page.
CloseForm	When the BackBtn is clicked this event is triggered, which checks if the form is currently visible and if it is, it will close.
UpdateForm	When this form is called it will check the current state of Common_Variables.FileToBeViewed and change the size, position and text of TutorialTxt to match the size, position and text that the tutorial page needs. It will do the same for the image shown on pictureBox1 and the text of Header.

Requirements Met

Design	Requirement Code
HomeScreen	FGR13, FGR14, FIR3, FIR4, FIR6
WelcomeScreen	FGR2
CreateAccount	FGR3a, FGR3b, FGR3c, FGR3d, FGR3e, FGR3f, FGR3g, FGR4a, FGR4b, FGR4c
Login	FGR3, FGR5, FGR6, FGR7
Lobby	FGR8, FGR11, FGR18
DifficultySelect	FGR10
Settings	FGR3c, FGR13, FGR16, FGR20
Leaderboard	FGR12
QuestionDesigner	FGR9
MultiChoice	FIR1, FIR2, FIR5, FIR9

Checkbox	FIR1, FIR2, FIR5, FIR9
TrueOrFalse	FIR1, FIR2, FIR5, FIR9
FillInTheBlank	FIR1, FIR2, FIR5, FIR9
DragAndDrop	FIR1, FIR2, FIR5, FIR9
EndScreen	FGR14
TextViewer	FGR21
Tutorial	FGR17

Testing

Test plan key:

FGR: Functional General Requirement

FIR: Functional In-Game Requirement

NFR: Non-functional Requirement

HomeScreen

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionUI Common_Variables .score Common_Varaibles .q_number Common_Varaibles .CurrentSection	Ensure that QuestionUI can be updated with up to date information.	QuestionUI.Text = "Question 5/20 - True or False Score: 2/20" Common_Variables .score = 3 Common_Varaibles .q_number = 6 Common_Varaibles .CurrentSection = "Multiple Choice"	QuestionUI.Text = "Question 6/20 - Multiple Choice Score: 3/20"	Pass	FIR6

2	AnsCheck Checkbox	Ensure that clicking AnsCheck while Checkbox is visible will trigger its answer checking code.	AnsCheck clicked Checkbox visible	Checkbox.AnsCheck is triggered	Pass	FIR7
3	AnsCheck MultiChoice	Ensure that clicking AnsCheck while MultiChoice is visible will trigger its answer checking code.	AnsCheck clicked MultiChoice visible	MultiChoice .AnsCheck is triggered	Pass	FIR7
4	AnsCheck TrueOrFalse	Ensure that clicking AnsCheck while TrueOrFalse is visible will trigger its answer checking code.	AnsCheck clicked TrueOrFalse visible	TrueOrFalse .AnsCheck is triggered	Pass	FIR7
5	AnsCheck FillInTheBlank	Ensure that clicking AnsCheck while FillInTheBlank is visible will trigger its answer checking code.	AnsCheck clicked FillInTheBlank visible	FillInTheBlank .AnsCheck is triggered	Pass	FIR7

6	AnsCheck DragAndDrop	Ensure that clicking AnsCheck while DragAndDrop is visible will trigger its answer checking code.	AnsCheck clicked DragAndDrop visible	DragAnsDrop .AnsCheck is triggered	Pass	FIR7
7	UsedSection	Ensure that quiz sections are randomly chosen and cannot appear again.	Random value in UsedSection (MultiChoice)	MultiChoice is shown to the user. MultiChoice is removed from list so it cannot show up again.	Pass	FIR4
8	Common_Variables .q_number Common_Variables .SkipEnd	Ensure that quiz ends after 20 questions and goes to EndScreen	Common_Variables .q_number = 20 Common_Variables .SkipEnd = false	Instead of going to a new section of the quiz, the user will be taken to EndScreen	Pass	FGR14
9	Common_Variables .q_number Common_Variables .SkipEnd	Ensure that quiz ends after 20 questions and goes to the Lobby, LeaderboardData & UserData should be updated.	Common_Variables .q_number = 20 Common_Variables .SkipEnd = true	Instead of going to a new section of the quiz, the user will be taken to the Lobby. Their lifetime statistics will be saved to their	Pass	FGR14 FGR23 FGR3f

	UserData LeaderboardData		Common_Variables .username = "TestUser" Common_Variables .score = 15 Common_Variables .LifetimeCorrect = 20 Common_Variables .LifetimeIncorrect = 0 Common_Variables .MultiLifeCorrect = 5 Common_Variables .MultiLifeIncorrect = 0 Common_Variables .CheckLifeCorrect = 5 Common_Variables .CheckLifeIncorrect = 0 Common_Variables .DropLifeCorrect = 5 Common_Variables .DropLifeIncorrect = 0	account and they will be added to the leaderboard.		
--	---------------------------------	--	--	--	--	--

			Common_Variables .TorfLifeCorrect = 5 Common_Variables .TorfLifeIncorrect = 0			
10	MuteToggle Common_Variables .muted	Ensure that clicking MuteToggle while the game is unmuted will mute the game and change the image on MuteToggle to show that the game is muted.	Common_Variables .muted = false MuteToggle clicked	Common_Variables .muted will be set to true and MuteToggles button will be changed to Muted64Red.png	Pass	FGR13
11	MuteToggle Common_Variables .muted	Ensure that clicking MuteToggle while the game is muted will unmute the game and change the image on MuteToggle to show that the game is unmuted.	Common_Variables .muted = true MuteToggle clicked	Common_Variables .muted will be set to false and MuteToggles button will be changed to UnMuted64.png	Pass	FGR13
12	BackBtn Target.Target	Ensure that clicking the back button takes the user back to the	Target.Target = "welcome"	The user will be taken back to the WelcomeScreen	Pass	FGR1

		form they were on.	BackBtn clicked			
13	BackBtn Target.Target	Ensure that clicking the back button takes the user back to the form they were on.	Target.Target = "lobby" BackBtn clicked	The user will be taken back to the Lobby	Pass	FGR1
14	BackBtn Target.Target	Ensure that clicking the back button takes the user back to the form they were on.	Target.Target = "settings" BackBtn clicked	The user will be taken back to the Settings form	Pass	FGR1
15	BackBtn Target.Target	Ensure that clicking the back button takes the user back to the form they were on.	Target.Target = "designer" BackBtn clicked	The user will be taken back to the Question Designer	Pass	FGR1
16	TutorialBtn	Ensure that clicking the TutorialBtn will take the user to the Tutorial screen.	TutorialBtn clicked.	The user will be taken to the Tutorial screen	Pass	FGR17
	CheckAns MuteToggle	Ensure that all buttons play click.wav when	CheckAns clicked MuteToggle clicked	Click.wav is played	Pass	FGR13

	TutorialBtn BackBtn Common_Variables .muted	clicked.	TutorialBtn clicked BackBtn clicked Common_Variables .muted = false			
17	CheckAns MuteToggle TutorialBtn BackBtn Common_Variables .muted	Ensure that all buttons do not play click.wav when clicked and the quiz is muted.	CheckAns clicked MuteToggle clicked TutorialBtn clicked BackBtn clicked Common_Variables .muted = true	Click.wav is not played	Pass	FGR13

WelcomeScreen

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	CreateAccountBtn	Ensure that clicking the CreateAccountBtn takes you to the CreateAccount form	CreateAccountBtn clicked	The user should be taken to the CreateAccount form. Click.wav will play.	Pass	FGR2
2	LoginBtn	Ensure that clicking the LoginBtn takes you to the Login form	LoginBtn clicked	The user should be taken to the Login form. Click.wav will play.	Pass	FGR2

CreateAccount

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	UsrnameInput PasswordInput CreateBtn	Ensure that user cannot make an account without entering Username and Password	UsrnameInput & PasswordInput are blank & CreateBtn is clicked	The user should be shown 2 error labels that tell them to enter Username and Password. Headers for both textboxes will also turn red. Error will be written to error log.	Pass	FGR3a FGR3d FGR4a
2	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that Usernames cannot be used twice.	UsrnameInput.Text = "TestUsr" (name already used) PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!" CreateBtn clicked	The user should be told that their username is already taken by an error label that is shown and UsrnameInput.Text is blanked. UsrHeader will also turn red.	Pass	FGR3c FGR4a

				Error will be written to error log. Click.wav will play.		
3	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that Usernames only consist of alpha-numeric character.	UsrnameInput.Text = "ThisIsATest!" PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!" CreateBtn clicked	The user should be told that their username cannot contain symbols by an error label that is shown and UsrnameInput.Text is blanked. UsrHeader will also turn red. Error will be written to error log. Click.wav will play.	Pass	FGR3e FGR4a
4	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that Usernames are longer than 3 characters.	UsrnameInput.Text = "Bla" PasswordInput = "ABCabc123!"	The user should be told that their username is too short by an error label that is shown and	Pass	FGR3a FGR4a

			ConfirmPassword = "ABCabc123!" CreateBtn clicked	UsrnameInput.Text is blanked. UsrHeader will also turn red. Error will be written to error log. Click.wav will play.		
5	UsrnameInput	Ensure that usernames are not longer than 12 characters.	Typing 13 characters into UsrnameInput (abcdefghijklm)	The user will be unable type anymore characters after the 12th	Pass	FGR3a FGR4a
6	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that whitespace is removed from Usernames.	UsrnameInput.Text = "A B C D E F" PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!" CreateBtn clicked	Any whitespace should be removed from the username (A B C D E F => ABCDEF). Account creation proceeds. Click.wav will play.	Pass	FGR3e FGR4a

7	UsrnamelInput PasswordInput ConfirmPassword CreateBtn	Ensure that passwords that are 3 characters or less are rejected.	UsrnamelInput.Text = "TestUsr" PasswordInput = "Ab1" ConfirmPassword = "Ab1" CreateBtn clicked	User is told that their password is too short by error label and PasswordInput is blanked. PasswordHeader will turn red. Error will be written to error log. Click.wav will play.	Pass	FGR3d FGR4a
8	UsrnamelInput PasswordInput ConfirmPassword CreateBtn	Ensure that passwords that do not have numbers are rejected.	UsrnamelInput.Text = "TestUsr" PasswordInput = "ABCabc!" ConfirmPassword = "ABCabc!" CreateBtn clicked	User is told that their password does not have a number by error label and PasswordInput is blanked. PasswordHeader will turn red. Error will be written to error log. Click.wav will play.	Pass	FGR3d FGR4a
9	UsrnamelInput PasswordInput ConfirmPassword CreateBtn	Ensure that passwords with no uppercase letters are rejected.	UsrnamelInput.Text = "TestUsr" PasswordInput = "abc123!"	User is told that their password must be mixed case	Pass	FGR3d FGR4a

			ConfirmPassword = "abc123!" CreateBtn clicked	by error label and PasswordInput is blanked. PasswordHeader will turn red. Error will be written to error log. Click.wav will play.		
10	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that passwords with no lowercase letters are rejected.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABC123!" ConfirmPassword = "ABC123!" CreateBtn clicked	User is told that their password must be mixed case by error label and PasswordInput is blanked. PasswordHeader will turn red. Error will be written to error log. Click.wav will play.	Pass	FGR3d FGR4a
11	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that passwords with no symbols are rejected.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABCabc123"	User is told that their password must contain a symbol by error label and	Pass	FGR3d FGR4a

			ConfirmPassword = "ABCabc123" CreateBtn clicked	PasswordInput is blanked. PasswordHeader will turn red. Error will be written to error log. Click.wav will play.		
12	UsrnameInput PasswordInput ConfirmPassword CreateBtn UserData	Ensure that valid passwords are accepted.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!" CreateBtn clicked	Account creation proceeds. Account credentials are written to UserData.dat Lobby is shown to user. Click.wav will play.	Pass	FGR3d FGR3b FGR4a
13	UsrnameInput PasswordInput ConfirmPassword CreateBtn UserData	Ensure that UserData file is created if none exists.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABCabc123!" ConfirmPassword	Account creation proceeds. Account credentials are written to new UserData.dat	Fail	FGR3d FGR3b FGR4a

			= "ABCAbc123!" CreateBtn clicked UserData deleted	Account Statistics variables are blanked Lobby is shown to user. Click.wav will play.		
14	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that account creation cannot proceed without confirming password.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABCAbc123!" ConfirmPassword = "ABCAbc123" CreateBtn clicked	User is told that their confirmed password does not match their password by error label and PasswordInput is blanked. ConfirmHeader will turn red. Error will be written to error log. Click.wav will play.	Pass	FGR3d FGR4a
15	UsrnameInput PasswordInput ConfirmPassword CreateBtn	Ensure that account creation cannot proceed without confirming password.	UsrnameInput.Text = "TestUsr" PasswordInput = "ABCAbc123!"	User is told that their confirmed password does not match their password by error label and PasswordInput is blanked.	Pass	FGR3d FGR4a

			ConfirmPassword = "ABCabc123" CreateBtn clicked	ConfirmHeader will turn red. Error will be written to error log. Click.wav will play.		
16	PasswordInput ConfirmPassword PassShow	Ensure that user can unhide their password.	PassShown = false PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!"	Both textboxes have their PasswordChar variables set to '\0', which shows any text in the boxes as normal text PassShown set to true. Click.wav will play.	Pass	FGR4b
17	PasswordInput ConfirmPassword PassShow	Ensure that user can hide their password.	PassShown = true PasswordInput = "ABCabc123!" ConfirmPassword = "ABCabc123!"	Both textboxes have their PasswordChar variables set to '*', which shows text in the box as Asterix. PassShown set to false. Click.wav will play.	Pass	FGR4b

Login

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	LoginBtn	Make sure that quiz does not crash when there is no UserData file, and that user is told to make one.	There is no UserData.dat file & CreateBtn is clicked.	A messagebox will pop up to tell the user that there is no UserData file, and to go to the CreateAccount screen to make an account. Click.wav will play.	Pass	FGR17
2	UsrnameInput PasswordInput LoginBtn	Make sure that an account cannot be signed into if the password is wrong.	UsrnameInput = "TestUser" PasswordInput = "Aa123!AAAA" LoginBtn clicked	An error label will show up telling the user that either the username or password is wrong, both headers will turn red, and the textboxes will be blanked. Click.wav will play.	Pass	FGR6

3	UsrnameInput PasswordInput LoginBtn	Make sure that an account cannot be signed into if the password and username is wrong.	UsrnameInput = "UserTest" PasswordInput = "ABC123!" LoginBtn clicked	An error label will show up telling the user that either the username or password is wrong, both headers will turn red, and the textboxes will be blanked. Click.wav will play.	Pass	FGR6
4	UsrnameInput PasswordInput LoginBtn	Make sure that an account cannot be signed into if the username is wrong.	UsrnameInput = "UserTest" PasswordInput = "ABCabc123!" LoginBtn clicked	An error label will show up telling the user that either the username or password is wrong, both headers will turn red, and the textboxes will be blanked. Click.wav will play.	Pass	FGR6
5	UsrnameInput PasswordInput LoginBtn	Make sure that an account cannot be signed into if nothing is entered	UsrnameInput = "" PasswordInput = ""	An error label will show up telling the user that either the username or	Pass	FGR2

		into the textboxes	LoginBtn clicked	password is wrong, both headers will turn red, and the textboxes will be blanked. Click.wav will play.		
6	PasswordInput PassShow	Ensure that user can unhide their password.	PassShown = false PasswordInput = "ABCabc123!"	PasswordInput has its PasswordChar variable set to '\0', which shows any text in the box as normal text. PassShown set to true. Click.wav will play and PassShows image will change to PassShow.png	Pass	FGR7
7	PasswordInput PassShow	Ensure that user can hide their password.	PassShown = true PasswordInput = "ABCabc123!"	PasswordInput has its PasswordChar variable set to '*', which shows text in the box as Asterix. PassShown set to false.	Pass	FGR7

				Click.wav will play and PassShows image will change to PassHide64		
8	UsrnameInput PasswordInput LoginBtn	Ensure that a user can log into an account that already exists	UsrnameInput = "TestUser" PasswordInput = "ABCabc123!" LoginBtn clicked	User logs into account, settings variables and account statistics are loaded from account. Lobby is shown to the user. Click.wav will play.	Pass	FGR3 FGR3f FGR3g
9	UsrnameInput PasswordInput LoginBtn	Ensure that a user can log into a second account that already exists	UsrnameInput = "TestUser1" PasswordInput = "ABCabc123!" LoginBtn clicked	User logs into account, settings variables and account statistics are loaded from account. Lobby is shown to the user. Click.wav will play.	Fail	FGR3 FGR3f FGR3g FGR15

Lobby

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	LobbyHeader	Make sure that the LobbyHeader uses the user's username	Common_Variables .username = "TestUser1"	LobbyHeaders text will read "Hello, " + Common_Variables .username	Pass	FGR18
2	Stats	Make sure that the users lifetime corrects and lifetime incorrect are shown to the user.	Common_Variables .LifetimeCorrect = 0 Common_Variables .LifetimeIncorrect = 0	Users lifetime correct and lifetime incorrect are shown on the label.	Pass	FGR3f
4	ExtendedStats	Make sure that the lifetime correct and lifetime incorrect for each question type are shown to the user.	Common_Variables .CheckLifeCorrect = 0 Common_Variables .CheckLifeIncorrect = 0 Common_Variables	ExtendedStats should look like "0/0 0/0 0/0 0/0 0/0" As that is the current state of those	Pass	FGR3f

			.DropLifeCorrect = 0 Common_Variables .DropLifeIncorrect = 0 Common_Variables .BlankLifeCorrect = 0 Common_Variables .BlankLifeIncorrect = 0 Common_Variables .MultiLifeCorrect = 0 Common_Variables .MultiLifeIncorrect = 0 Common_Variables .TorfLifeCorrect = 0 Common_Variables .TorfLifeIncorrect = 0	variables.		
--	--	--	--	-----------------------	--	--

5	DifficultySelectBtn Common_Variables .muted	Make sure that the user is taken to the DifficultySelect screen when clicked and click sound is played.	DifficultySelectBtn clicked Common_Variables .muted = false	User is taken to the DifficultySelect screen and click.wav is played.	Pass	FGR8
6	DifficultySelectBtn Common_Variables .muted	Make sure that the user is taken to the DifficultySelect screen when clicked and click sound isn't played as game is muted.	DifficultySelectBtn clicked Common_Variables .muted = true	User is taken to the DifficultySelect screen and click.wav is not played.	Pass	FGR8 FGR13
7	LeaderboardBtn Common_Variables .muted	Make sure that the user is taken to the Leaderboard when clicked and click sound is played.	LeaderboardBtn clicked Common_Variables .muted = false	User is taken to the Leadeboard and click.wav is played.	Pass	FGR8
8	LeaderboardBtn Common_Variables .muted	Make sure that the user is taken to the DifficultySelect screen when clicked and click sound isn't played as game is muted.	LeaderboardBtn clicked Common_Variables .muted = true	User is taken to the Leaderboard and click.wav is not played.	Pass	FGR8 FGR13

9	PlayGameBtn Common_Variables .muted	Make sure that the quiz starts when clicked and click sound is played.	PlayGameBtn clicked Common_Variables .muted = false	The quiz starts and click.wav is played.	Pass	FGR8
10	PlayGameBtn Common_Variables .muted	Make sure that the quiz starts when clicked and click sound isn't played as game is muted.	PlayGameBtn clicked Common_Variables .muted = true	The quiz starts and click.wav is not played.	Pass	FGR8 FGR13
11	DesignerBtn Common_Variables .muted	Make sure that the Question Designer is shown when clicked and click sound is played.	DesignerBtn clicked Common_Variables .muted = false	The Question Designer is shown and click.wav is played.	Pass	FGR8
12	DesignerBtn Common_Variables .muted	Make sure that the Question Designer is shown when clicked and the click sound isn't played as game is muted.	DesignerBtn clicked Common_Variables .muted = true	The Question Designer is shown and click.wav is not played.	Pass	FGR8 FGR13

13	SettingsBtn Common_Variables .muted	Make sure that the Settings page is shown when clicked and the click sound is played.	SettingsBtn clicked Common_Variables .muted = false	The Settings page is shown and click.wav is played.	Pass	FGR8
14	SettingsBtn Common_Variables .muted	Make sure that the Settings page is shown when clicked and the click sound isn't played as game is muted.	SettingsBtn clicked Common_Variables .muted = true	The Question Designer is shown and click.wav is not played.	Pass	FGR8 FGR13

DifficultySelect

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	EasyBtn NormalBtn HardBtn Common_Variables .difficulty Common_Variables .muted	Ensure that the button corresponding to the games current difficulty is highlighted when the form is loaded	Common_Variables .difficulty = "e" Common_Variables .muted = false	EasyBtn should have a green border other buttons should not. Click.wav is played.	Pass	N/A
2	EasyBtn NormalBtn HardBtn Common_Variables .difficulty Common_Variables .muted	Ensure that the button corresponding to the games current difficulty is highlighted when the form is loaded	Common_Variables .difficulty = "n" Common_Variables .muted = false	NormalBtn should have an orange border other buttons should not. Click.wav is played.	Pass	N/A
3	EasyBtn NormalBtn HardBtn	Ensure that the button corresponding to the games current	Common_Variables .difficulty = "h" Common_Variables	HardBtn should have a red border other buttons should not.	Pass	N/A

	Common_Variables .difficulty Common_Variables .muted	difficulty is highlighted when the form is loaded	.muted = false	Click.wav is played.		
4	EasyBtn Common_Variables .difficulty	Ensure that the difficulty is set to easy when EasyBtn is clicked and that it is highlighted.	EasyBtn clicked	EasyBtn is highlighted by a green border, while the other 2 buttons have their borders removed. Click.wav is played.	Pass	FGR10
5	EasyBtn Common_Variables .difficulty Common_Variables .muted	Ensure that the difficulty is set to easy when EasyBtn is clicked and that it is highlighted. Ensure that click.wav is not played as the game is muted.	EasyBtn clicked Common_Variables .muted = true	EasyBtn is highlighted by a green border, while the other 2 buttons have their borders removed. Click.wav is not played.	Pass	FGR10 FGR13
6	NormalBtn Common_Variables	Ensure that the difficulty is set to	NormalBtn clicked	NormalBtn is highlighted by an	Pass	FGR10

	.difficulty	normal when NormalBtn is clicked and that it is highlighted.		orange border, while the other 2 buttons have their borders removed. Click.wav is played.		
7	NormalBtn Common_Variables .difficulty Common_Variables .muted	Ensure that the difficulty is set to normal when NormalBtn is clicked and that it is highlighted. Ensure that click.wav is not played as the game is muted.	NormalBtn clicked Common_Variables .muted = true	NormalBtn is highlighted by an orange border, while the other 2 buttons have their borders removed. Click.wav is not played.	Pass	FGR10 FGR13
8	HardBtn Common_Variables .difficulty	Ensure that the difficulty is set to hard when HardBtn is clicked and that it is highlighted.	HardBtn clicked	HardBtn is highlighted by a red border, while the other 2 buttons have their borders removed. Click.wav is played.	Pass	FGR10

9	HardBtn Common_Variables .difficulty Common_Variables .muted	Ensure that the difficulty is set to hard when HardBtn is clicked and that it is highlighted. Ensure that click.wav is not played as the game is muted.	HardBtn clicked Common_Variables .muted = true	HardBtn is highlighted by a red border, while the other 2 buttons have their borders removed. Click.wav is not played.	Pass	FGR10 FGR13
---	--	--	--	---	------	----------------

Leaderboard

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	Leaderboard Data.dat	Ensure that the quiz does not crash if there is no LeaderboardData file and shows an error label telling the user that the file is not found.	Leaderboard Data.dat does not exist	Error is shown, program does not crash.	Pass	FGR19
2	Leaderboard Data.dat SortNameBtn	Ensure that the quiz does not crash if there is no LeaderboardData file and user clicks one of the sorting buttons.	Leaderboard Data.dat does not exist. SortNameBtn clicked	Error is shown, program does not crash.	Fail	FGR19
3	Leaderboard Data.dat DataGrid	Test to see if a valid LeaderboardData file is loaded onto the DataGrid correctly, sorted by	LeaderboardData .dat	Contents of LeaderboardData .dat are loaded onto DataGrid, sorted by highest score.	Pass	FGR12

	SortScoreBtn	score descendingly by default.		SortScoreBtn has its border size set to 5 to highlight the button.		
4	Leaderboard Data.dat DataGrid SortOrderBtn	Ensure that clicking the SortOrderBtn while the DataGrid is ordered by descending will make the DataGrid order ascendingly and update the text on SortOrderBtn to say "Ascending".	OrderBy = "Descending"	Contents of DataGrid are ordered ascendingly. SortOrderBtn text is changed to say "Ascending" Click.wav is played.	Pass	FGR12
5	Leaderboard Data.dat DataGrid SortOrderBtn Common_Variables .muted	Ensure that clicking the SortOrderBtn while the DataGrid is ordered by descending will make the DataGrid order ascendingly and update the text on SortOrderBtn to say "Ascending". Ensure that click.wav does not	OrderBy = "Descending" Common_Variables .muted = true	Contents of DataGrid are ordered ascendingly. SortOrderBtn text is changed to say "Ascending" Click.wav is not played.	Pass	FGR12 FGR13

		play as the quiz is muted.				
6	Leaderboard Data.dat DataGrid SortOrderBtn	Ensure that clicking the SortOrderBtn while the DataGrid is ordered by ascending will make the DataGrid order descending and update the text on SortOrderBtn to say "Descending".	OrderBy = "Ascending"	Contents of DataGrid are ordered descendingly. SortOrderBtn text is changed to say "Descending" Click.wav is played.	Pass	FGR12
7	Leaderboard Data.dat DataGrid SortOrderBtn Common_Variables .muted	Ensure that clicking the SortOrderBtn while the DataGrid is ordered by ascending will make the DataGrid order descending and update the text on SortOrderBtn to say "Descending". Ensure that click.wav does not play as the quiz is muted.	OrderBy = "Ascending" Common_Variables .muted = true	Contents of DataGrid are ordered descendingly. SortOrderBtn text is changed to say "Descending" Click.wav is not played.	Pass	FGR12 FGR13

		play as the quiz is muted.				
8	SortNameBtn	Ensure that, when SortNameBtn is clicked, the Leaderboard is sorted by username and that SortNameBtn is highlighted. Also ensure that click.wav is played.	SortNameBtn is clicked.	DataGrid is sorted by username, click.wav is played. SortNameBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12
9	SortNameBtn Common_Variables .muted	Ensure that, when SortNameBtn is clicked, the Leaderboard is sorted by username and that SortNameBtn is highlighted. Also ensure that click.wav is not played, as the quiz is muted.	SortNameBtn is clicked. Common_Variables .muted = true	DataGrid is sorted by username, click.wav is not played. SortNameBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12 FGR13

10	SortScoreBtn	Ensure that, when SortScoreBtn is clicked, the Leaderboard is sorted by score and that SortScoreBtn is highlighted. Also ensure that click.wav is played.	SortScoreBtn is clicked.	DataGrid is sorted by score, click.wav is played. SortScoreBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12
11	SortScoreBtn Common_Variables.muted	Ensure that, when SortScoreBtn is clicked, the Leaderboard is sorted by score and that SortScoreBtn is highlighted. Also ensure that click.wav is not played, as the quiz is muted.	SortScoreBtn is clicked. Common_Variables.muted = true	DataGrid is sorted by score, click.wav is not played. SortScoreBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12 FGR13
12	SortLCBtn	Ensure that, when SortLCBtn is clicked, the Leaderboard is sorted by lifetime correct and that	SortLCBtn is clicked.	DataGrid is sorted by lifetime correct, click.wav is played.	Pass	FGR12

		SortLCBtn is highlighted. Also ensure that click.wav is played.		SortLCBtns border size is set to 5 and all other buttons is set to 0.		
13	SortLCBtn Common_Variables .muted	Ensure that, when SortLCBtn is clicked, the Leaderboard is sorted by lifetime correct and that SortLCBtn is highlighted. Also ensure that click.wav is not played, as the quiz is muted.	SortLCBtn is clicked. Common_Variables .muted = true	DataGrid is sorted by lifetime correct, click.wav is not played. SortLCBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12 FGR13
14	SortLICBtn	Ensure that, when SortLICBtn is clicked, the Leaderboard is sorted by lifetime incorrect and that SortLICBtn is highlighted.	SortLICBtn is clicked.	DataGrid is sorted by lifetime incorrect, click.wav is played. SortLCBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12

		Also ensure that click.wav is played.				
15	SortLICBtn Common_Variables .muted	Ensure that, when SortLICBtn is clicked, the Leaderboard is sorted by lifetime incorrect and that SortLICBtn is highlighted. Also ensure that click.wav is not played, as the quiz is muted.	SortLICBtn is clicked. Common_Variables .muted = true	DataGrid is sorted by lifetime incorrect, click.wav is not played. SortLICBtns border size is set to 5 and all other buttons is set to 0.	Pass	FGR12 FGR13

Settings

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	checkedListBox	Ensure that the checkListBox is updated to show the settings that are currently enabled when the form is loaded.	Common_Variables .isAdmin = true Common_Variables .ReadQuestions = true.	Admin Mode and Read Out Questions are checked.	Pass	N/A
2	checkedListBox Common_Variables .isAdmin AnsCheck	Ensure that clicking the Admin Mode checkbox while it is unchecked will enable Admin Mode and check the checkbox.	Common_Variables .isAdmin = false checkListBox[0] clicked. AnsCheck clicked.	Common_Variables .isAdmin is set to true. Click.wav played. Admin settings UI is unhidden.	Pass	FGR16 FGR20
3	checkedListBox Common_Variables .isAdmin	Ensure that clicking the Admin Mode checkbox while it is unchecked will enable Admin Mode	Common_Variables .isAdmin = false checkListBox[0] clicked.	Common_Variables .isAdmin is set to true.	Pass	FGR16 FGR13 FGR20

	Common_Variables .muted AnsCheck	and check the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .muted = true AnsCheck clicked.	Click.wav not played. Admin settings UI is unhidden.		
4	checkedListBox Common_Variables .isAdmin AnsCheck	Ensure that clicking the Admin Mode checkbox while it is checked will disable Admin Mode and uncheck the checkbox.	Common_Variables .isAdmin = true checkListBox[0] clicked. AnsCheck clicked.	Common_Variables .isAdmin is set to false. Click.wav played. Admin settings UI is hidden.	Pass	FGR16 FGR20
5	checkedListBox Common_Variables .isAdmin Common_Variables .muted AnsCheck	Ensure that clicking the Admin Mode checkbox while it is checked will disable Admin Mode and uncheck the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .isAdmin = false checkListBox[0] clicked. Common_Variables .muted = true AnsCheck clicked.	Common_Variables .isAdmin is set to false. Click.wav not played. Admin settings UI is hidden.	Pass	FGR16 FGR13 FGR20

6	checkedListBox Common_Variables .ReadQuestions AnsCheck	Ensure that clicking the ReadQuestions checkbox while it is unchecked will enable the reading out of questions and check the checkbox.	Common_Variables .ReadQuestions= false checkListBox[1] clicked. AnsCheck clicked.	Common_Variables .ReadQuestions is set to true. Click.wav played.	Pass	FGR20
7	checkedListBox Common_Variables .ReadQuestions Common_Variables .muted AnsCheck	Ensure that clicking the ReadQuestions checkbox while it is unchecked will enable the reading out of questions and check the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .ReadQuestions= false checkListBox[1] clicked. Common_Variables .muted = true AnsCheck clicked.	Common_Variables .ReadQuestions is set to true. Click.wav not played.	Pass	FGR20 FGR13
8	checkedListBox Common_Variables .ReadQuestions AnsCheck	Ensure that clicking the ReadQuestions checkbox while it is checked will disable the reading out of questions	Common_Variables .ReadQuestions = true checkListBox[1] clicked.	Common_Variables .ReadQuestions is set to false. Click.wav is played.	Pass	FGR20 FGR13

		and uncheck the checkbox.	AnsCheck clicked.			
9	checkedListBox Common_Variables .ReadQuestions Common_Variables .muted AnsCheck	Ensure that clicking the ReadQuestions checkbox while it is checked will disable the reading out of questions and uncheck the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .ReadQuestions = true checkListBox[1] clicked. Common_Variables .muted = true AnsCheck clicked.	Common_Variables .ReadQuestions is set to false. Click.wav not played.	Pass	FGR20 FGR13
10	checkedListBox Common_Variables .SkipEnd AnsCheck	Ensure that clicking the SkipEnd checkbox while it is unchecked will enable the skipping of the EndScreen and check the checkbox	Common_Variables .SkipEnd = false checkListBox[2] clicked. AnsCheck clicked.	Common_Variables .SkipEnd is set to true. Click.wav played.	Pass	FGR20
11	checkedListBox Common_Variables	Ensure that clicking the SkipEnd checkbox while it is	Common_Variables .SkipEnd = false	Common_Variables .SkipEnd is set to true.	Pass	FGR20 FGR13

	.SkipEnd Common_Variables .muted AnsCheck	unchecked will enable the skipping of the EndScreen and check the checkbox Click.wav will not be played as the quiz is muted.	checkBox[2] clicked. Common_Variables .muted = true AnsCheck clicked.	Click.wav not played.		
12	checkedListBox Common_Variables .SkipEnd AnsCheck	Ensure that clicking the SkipEnd checkbox while it is checked will disable the skipping of the EndScreen and uncheck the checkbox	Common_Variables .SkipEnd = true checkBox[2] clicked. AnsCheck clicked.	Common_Variables .SkipEnd is set to false. Click.wav played.	Pass	FGR20
13	checkedListBox Common_Variables .SkipEnd Common_Variables .muted AnsCheck	Ensure that clicking the SkipEnd checkbox while it is checked will disable the skipping of the EndScreen and uncheck the checkbox Ensure that click.wav does not play as the	Common_Variables .SkipEnd = true checkBox[2] clicked. Common_Variables .muted = true AnsCheck clicked.	Common_Variables .SkipEnd is set to false. Click.wav not played.	Pass	FGR20 FGR13

		quiz is muted.				
14	checkedListBox Common_Variables .dyslexia Common_Variables .muted AnsCheck	Ensure that clicking the dyslexia checkbox while it is unchecked will enable the dyslexia mode for the quiz and check the checkbox.	Common_Variables .dyslexia = false checkListBox[3] clicked. AnsCheck clicked.	Common_Variables .dyslexia is set to true. Click.wav played.	Pass	FGR20
15	checkedListBox Common_Variables .dyslexia Common_Variables .muted AnsCheck	Ensure that clicking the dyslexia checkbox while it is unchecked will enable the dyslexia mode for the quiz and check the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .dyslexia = false checkListBox[3] clicked. Common_Variables .muted = true AnsCheck clicked.	Common_Variables .dyslexia is set to true. Click.wav not played.	Pass	FGR20 FGR13
16	checkedListBox	Ensure that clicking the dyslexia	Common_Variables .dyslexia = true	Common_Variables .dyslexia is set to	Pass	FGR20

	Common_Variables .dyslexia AnsCheck	checkbox while it is checked will disable the dyslexia mode for the quiz and uncheck the checkbox.	checkBox[3] clicked. AnsCheck clicked.	false. Click.wav is played.		
17	checkedListBox Common_Variables .dyslexia Common_Variables .muted AnsCheck	Ensure that clicking the dyslexia checkbox while it is checked will disable the dyslexia mode for the quiz and uncheck the checkbox. Ensure that click.wav does not play as the quiz is muted.	Common_Variables .dyslexia = true checkBox[3] clicked. AnsCheck clicked.	Common_Variables .dyslexia is set to false. Click.wav not played.	Pass	FGR20 FGR13
18	volumeBar VolumeHeader	Ensure that changing the value of the volume bar changes the value of VolumeHeader	volumeBar value changed from 0 to 40.	VolumeHeader should change from "Volume:" to "Volume: 40"	Pass	FGR13
19	volumeBar AnsCheck	Ensure that raising the volume on	volumeBar value changed from 0 to	System volume should change from	Pass	FGR13

		volumeBar will set the system volume to that value when settings are applied.	40. System volume is 20. AnsCheck clicked.	20 to 0 and then from 0 to 40.		
20	volumeBar AnsCheck	Ensure that lowering the volume on volumeBar will set the system volume to that value when settings are applied.	volumeBar value changed from 40 to 0. System volume is 40. AnsCheck clicked.	System volume should change from 40 to 0.	Pass	FGR13
21	volumeBar AnsCheck	Ensure that not changing the volume on volumeBar will not change the system volume	volumeBar value set to 40. System volume is 40. AnsCheck clicked.	Volume changing functions should not be called.	Pass	FGR13
22	DeleteUsrDat	Ensure that the user has to click this button twice to	DeleteUsrDat clicked.	Text on button should change to "Are you sure?" and	Pass	FGR16

		actually delete UserData	UserFirstClick = true.	UserFirstClick should be set to false, both reverting to their normal values if the button isn't clicked again in 3 seconds.		
23	DeleteUsrDat	Ensure that when the button is clicked for a second time within 3 seconds that the UserData file is deleted.	DeleteUsrDat clicked. UserFirstClick = false	Text on button should change to "Deleted!" and then send the user to the WelcomeScreen after 500ms. UserData.dat deleted. Click.wav will play.	Pass	FGR16
24	DeleteUsrDat Common_Variables .muted	Ensure that the user has to click this button twice to actually delete UserData Ensure that click.wav does not play as the quiz is muted.	DeleteUsrDat clicked. UserFirstClick = true. Common_Variables .muted = true	Text on button should change to "Are you sure?" and UserFirstClick should be set to false, both reverting to their normal values if the button isn't clicked again in	Pass	FGR16 FGR13

				3 seconds. Click.wav does not play.		
25	DeleteUsrDat	Ensure that when the button is clicked for a second time within 3 seconds that the UserData file is deleted. Ensure that click.wav does not play as the quiz is muted.	DeleteUsrDat clicked. UserFirstClick = false	Text on button should change to "Deleted!" and then send the user to the WelcomeScreen after 500ms. UserData.dat deleted. Click.wav will not play.	Pass	FGR16 FGR13
26	ResetLeaderboard	Ensure that when the button is clicked for a second time within 3 seconds that the LeaderboardData file is deleted.	ResetLeaderboard clicked. BoardFirstClick = false.	Text on button should change to "Deleted!" and then change back to "Reset Leaderboard" after 500ms. LeaderboardData .dat deleted.	Pass	FGR16

				Click.wav will play.		
27	ResetLeaderboard Common_Variables .muted	Ensure that when the button is clicked for a second time within 3 seconds that the LeaderboardData file is deleted. Click.wav should not play as quiz is muted.	ResetLeaderboard clicked. BoardFirstClick = false. Common_Variables .muted = true	Text on button should change to "Deleted!" and then change back to "Reset Leaderboard" after 500ms. LeaderboardData .dat deleted. Click.wav will not play.	Pass	FGR16 FGR13
28	MessageSelection TextView NoOption MessageHeader	Ensure that the user is not taken to TextView without selection a message category to view to prevent a crash.	MessageHeader .Text = "Select a message category" TextView clicked.	The user should not be taken to TextView, instead NoOption should be shown to the user and MessageHeader should turn red to indicate that there has been an error.	Pass	FGR19
29	MessageSelection	Ensure that the user is taken to	MessageHeader .Text = "Very Bad	Common_Variables .FileToBeViewed	Pass	FGR21

	TextView	TextViewer is they have selected a message category.	(Score <= 3)" TextView clicked.	should be set to "VeryBadEnd.txt" and TextViewer is shown to the user.		
30	MessageSelection TextView	Ensure that the user is taken to TextViewer is they have selected a message category.	MessageHeader .Text = "Bad (Score 4 - 8)" TextView clicked.	Common_Variables .FileToBeViewed should be set to "BadEnd.txt" and TextViewer is shown to the user.	Pass	FGR21
31	MessageSelection TextView	Ensure that the user is taken to TextViewer is they have selected a message category.	MessageHeader .Text = "Normal (Score 9 - 12)" TextView clicked.	Common_Variables .FileToBeViewed should be set to "NormalEnd.txt" and TextViewer is shown to the user.	Pass	FGR21
32	MessageSelection TextView	Ensure that the user is taken to TextViewer is they have selected a message category.	MessageHeader .Text = "Good (Score 12 - 16)" TextView clicked.	Common_Variables .FileToBeViewed should be set to "GoodEnd.txt" and TextViewer is shown to the user.	Pass	FGR21

33	MessageSelection TextView	Ensure that the user is taken to TextView is they have selected a message category.	MessageHeader .Text = "Very Good (Score >= 17)" TextView clicked.	Common_Variables .FileToBeViewed should be set to "VeryGoodEnd.txt" and TextView is shown to the user.	Pass	FGR21
34	SaveBtn	Ensure that the user has to click this button twice to actually save their message.	SaveFirstClick = true. SaveBtn clicked.	Text on button should change to "Are you sure?" and SaveFirstClick should be set to false, both reverting to their normal values if the button isn't clicked again in 3 seconds.	Pass	N/A
35	SaveBtn MessageInput MessageHeader NoMessage	Ensure that the user has to click this button twice to actually save their message and that a blank message is rejected	SaveFirstClick = false. MessageInput = "" SaveBtn clicked.	NoMessage should be shown and MessageHeader should be turned red.	Pass	FGR19

36	SaveBtn MessageInput	Ensure that the user has to click this button twice to actually save their message and that and / are removed from the message to prevent errors.	SaveFirstClick = false. MessageInput = "T/E/S/T" SaveBtn clicked.	MessageInput.Text should be changed to "TEST"	Pass	FGR16
37	SaveBtn MessageInput MessageSelection MessageHeader AlreadyExists	Ensure that the user has to click this button twice to actually save their message and that duplicate messages are rejected	SaveFirstClick = false. MessageSelection.SelectedIndex = 1 MessageInput = "Swing and a miss!" SaveBtn clicked.	Already exists should be shown and MessageHeader should be turned red MessageInput should be blanked	Pass	FGR16 FGR19
38	SaveBtn MessageInput MessageSelection	Ensure that the user has to click this button twice to actually save their message and that it is saved in the right place.	SaveFirstClick = false. MessageSelection.SelectedIndex = 0 MessageInput = "Test"	Message is written to VeryBadEnd.txt alongside font size entered. MessageInput should be blanked.	Pass	FGR16

			SaveBtn clicked.			
39	SaveBtn MessageInput MessageSelection	Ensure that the user has to click this button twice to actually save their message and that it is saved in the right place.	SaveFirstClick = false. MessageSelection.SelectedIndex = 1 MessageInput = "Test" SaveBtn clicked.	Message is written to BadEnd.txt alongside font size entered. MessageInput should be blanked.	Pass	FGR16
40	SaveBtn MessageInput MessageSelection	Ensure that the user has to click this button twice to actually save their message and that it is saved in the right place.	SaveFirstClick = false. MessageSelection.SelectedIndex = 2 MessageInput = "Test" SaveBtn clicked.	Message is written to NormalEnd.txt alongside font size entered. MessageInput should be blanked.	Pass	FGR16
41	SaveBtn MessageInput	Ensure that the user has to click this button twice to	SaveFirstClick = false.	Message is written to GoodEnd.txt alongside font size entered.	Pass	FGR16

	MessageSelection	actually save their message and that it is saved in the right place.	MessageSelection .SelectedIndex = 3 MessageInput = "Test" SaveBtn clicked.	MessageInput should be blanked.		
42	SaveBtn MessageInput MessageSelection	Ensure that the user has to click this button twice to actually save their message and that it is saved in the right place.	SaveFirstClick = false. MessageSelection .SelectedIndex = 4 MessageInput = "Test" SaveBtn clicked.	Message is written to VeryGoodEnd.txt alongside font size entered. MessageInput should be blanked.	Pass	FGR16
43	PreviewBtn SettingsHeader FontSizeInput	Ensure that SettingsHeaders text it changed to the current text of MessageHeader and that its font size is set to the current value of FontSizeInput.	FontSizeInput .Value = 44 MessageInput .Text = "Test" PreviewBtn clicked	SettingsHeaders font size is set to 44 and its text is set to "Test"	Pass	FGR16

44	DeleteUserDat PreviewBtn SaveBtn ResetLeaderboard TextView	Ensure that all these buttons play click.wav when clicked.	DeleteUserDat clicked PreviewBtn clicked SaveBtn clicked ResetLeaderboard clicked TextView clicked	All buttons play click.wav when clicked.	Pass	N/A
45	DeleteUserDat PreviewBtn SaveBtn ResetLeaderboard TextView Common_Variables .muted	Ensure that all these buttons do not play click.wav when clicked when the game is muted.	DeleteUserDat clicked PreviewBtn clicked SaveBtn clicked ResetLeaderboard clicked TextView clicked Common_Variables. muted = true	All buttons do not play click.wav when clicked.	Pass	FGR13

Candidate Number: 2217

Centre Number: 71955

46	MessageInput NoOption AlreadyTaken NoMessage MessageHeader	Ensure all error messages are cleared when MessageInput is clicked.	MessageInput clicked	All error labels are hidden and MessageHeaders colour is set to white.	Pass	FGR19
----	--	---	----------------------	--	------	-------

QuestionDesigner

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	questionType Selector TextView	Ensure that the user cannot go to TextView without selecting a question type.	questionType Selector.Text = "Please select a question type." TextView clicked.	questionType Selector's forecolour is set to red.	Pass	FGR19
2	questionType Selector PreviewBtn	Ensure that the user cannot preview a question without selecting a question type.	questionType Selector.Text = "Please select a question type." PreviewBtn clicked.	questionType Selector's forecolour is set to red.	Pass	FGR19
3	questionType Selector EasyBtn HardBtn	Ensure that the proper UI is shown to the user when they select the Checkbox question type.	questionType Selector.Text = "Checkbox"	QuestionInput, FirstInput, SecondInput, ThirdInput, FourthInput, SizeInput, EasyBtn, HardBtn and checkBox1 - 4 are	Pass	FGR9

	QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput checkBox1 checkBox2 checkBox3 checkBox4			all shown to the user in the proper places.		
4	questionType Selector EasyBtn HardBtn QuestionInput	Ensure that the proper UI is shown to the user when they select the Fill in the Blank question type.	questionType Selector.Text = "Fill in the Blank"	QuestionInput, FirstInput, SizeInput, EasyBtn, HardBtn and checkBox1 are all shown to the user in the proper places and with the proper text.	Pass	FGR19

	SizeInput FirstInput checkBox1					
5	questionType Selector EasyBtn HardBtn QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput radioButton1 radioButton2	Ensure that the proper UI is shown to the user when they select the Multiple Choice question type.	questionType Selector.Text = "Multiple Choice"	QuestionInput, FirstInput, SecondInput, ThirdInput, FourthInput, SizeInput, EasyBtn, HardBtn and radioButton1 – 4 are all shown to the user in the proper places.	Pass	FGR9

	radioButton3 radioButton4					
6	questionType Selector EasyBtn HardBtn QuestionInput SizeInput TrueBtn FalseBtn	Ensure that the proper UI is shown to the user when they select the True or False question type.	questionType Selector.Text = "True or False"	QuestionInput, SizeInput, EasyBtn, HardBtn, TrueBtn and FalseBtn shown to the user in the proper places.	Pass	FGR19
7	questionType Selector EasyBtn HardBtn QuestionInput	Ensure that the user can preview the question they have made.	questionType Selector.Text = "Checkbox" PreviewBtn clicked.	QuestionInput, FirstInput, SecondInput, ThirdInput, FourthInput, are all hidden and checkboxes 1-4 are changed to resemble the	Pass	FGR9

	FirstInput SecondInput ThirdInput FourthInput SizeInput checkBox1 checkBox2 checkBox3 checkBox4 SaveBtn PreviewBtn TextView ReturnBtn			checkboxes used in the actual Checkbox section. The checkboxes are changed to show the text of First – FourthInput and Header has its text changed to the text contained in QuestionInput and its font size is changed to the value of SizeInput. ReturnBtn is unhidden. SaveBtn, questionType Selector, PreviewBtn, EastBtn, HardBtn and TextView are all hidden.		
8	questionType Selector	Ensure that the user can preview the	questionType Selector.Text = “Fill	All question making UI is hidden and	Pass	FGR9

	EasyBtn HardBtn QuestionInput FirstInput SizeInput checkBox1 ReturnBtn SaveBtn PreviewBtn TextView	question they have made.	in the Blank” PreviewBtn clicked.	DummyTxtbox is unhidden so that the form now resembles a Fill in the Blank question. Header has its text changed to the text contained in QuestionInput and its font size is changed to the value of SizeInput. ReturnBtn is unhidden.		
9	questionType Selector EasyBtn HardBtn QuestionInput	Ensure that the user can preview the question they have made.	questionType Selector.Text = “Multiple Choice” PreviewBtn clicked.	SaveBtn, questionType Selector, PreviewBtn, EastBtn, HardBtn and TextView are all hidden, aswell as all of the question making	Pass	FGR9

	FirstInput SecondInput ThirdInput FourthInput SizeInput radioButton1 radioButton2 radioButton3 radioButton4 ReturnBtn SaveBtn PreviewBtn TextView			UI, bar the radioButtons. radioButtons visual properties are changed to make the form resemble a Multiple Choice question. radioButtons text is set to the text of First-Fourth inputs. Header has its text changed to the text contained in QuestionInput and its font size is changed to the value of SizeInput. ReturnBtn is unhidden.		
10	questionType Selector	Ensure that the user can preview the	questionType Selector.Text =	SaveBtn,	Pass	FGR9

	EasyBtn HardBtn QuestionInput SizeInput ReturnBtn SaveBtn PreviewBtn TextView TrueBtn FalseBtn	question they have made.	“True or False” PreviewBtn clicked.	questionType Selector, PreviewBtn, EastBtn, HardBtn and TextView are all hidden, aswell as all of the question making UI, bar TrueBtn and FalseBtn. TrueBtn & FalseBtn visual properties are changed so that they resemble the buttons on an actual True of False question. Header has its text changed to the text contained in QuestionInput and its font size is changed to the value of SizeInput. ReturnBtn is unhidden.		
--	--	--------------------------	---	---	--	--

11	questionType Selector QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput checkBox1 checkBox2 checkBox3 checkBox4 EasyBtn SaveBtn	Ensure that the user can successfully save an easy question.	questionType Selector.Text = "Checkbox" QuestionInput.Text = "Which of these are real computer components?" FirstInput.Text = "CPU" SecondInput.Text = "Compiler" ThirdInput.Text = "GPU" FourthInput = "Ethernet" SizeInput = 40 checkbox 1 & 3 checked. EasyBtn clicked. SaveBtn clicked twice.	The question should be saved to checkbox.txt with a unique ID as an easy mode question.	Pass	FGR9
----	--	--	---	---	------	------

12	questionType Selector QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput checkBox1 checkBox2 checkBox3 checkBox4 HardBtn SaveBtn	Ensure that the user can successfully save a question.	questionType Selector.Text = "Checkbox" QuestionInput.Text = "Which of these are real CPU instruction sets?" FirstInput.Text = "Itanium" SecondInput.Text = "x86-64" ThirdInput.Text = "Arm64" FourthInput = "x32- 24" SizeInput = 44 checkbox 1, 2 & 3 checked. HardBtn clicked. SaveBtn clicked twice.	The question should be saved to checkbox.txt with a unique ID as a hard mode question.	Pass	FGR9
----	--	---	--	--	------	------

13	questionType Selector QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput checkBox1 checkBox2 checkBox3 EasyBtn SaveBtn	Ensure that the user can successfully save an easy question.	questionType Selector.Text = "Fill in the Blank" QuestionInput.Text = "What is Microsoft's code editor called?" FirstInput.Text = "Visual Studio" SecondInput.Text = "Visual Studio Code" ThirdInput.Text = "VS Code" FourthInput = "Notepad" SizeInput = 44 checkbox 1 & 2 checked. EasyBtn clicked. SaveBtn clicked twice.	The question should be saved to fillinblank.txt with a unique ID.	Pass	FGR9
----	---	---	--	--	------	------

14	questionType Selector QuestionInput FirstInput SecondInput ThirdInput FourthInput SizeInput radioButton1 radioButton2 radioButton3 radioButton4 EasyBtn SaveBtn	Ensure that the user can successfully save a question.	questionType Selector.Text = "Multiple Choice" QuestionInput.Text = "What is a GPU?" FirstInput.Text = "A chip that processes digital images" SecondInput.Text = "A chip that handles networking" ThirdInput.Text = "A device that displays digital images" FourthInput = "A storage device" SizeInput = 44 radioButton1 clicked. EasyBtn clicked.	The question should be saved to multichoice.txt with a unique ID.	Pass	FGR9
----	--	---	--	--	------	------

			SaveBtn clicked twice.			
15	questionType Selector QuestionInput FalseBtn EasyBtn SaveBtn	Ensure that the user can successfully save a question.	questionType Selector.Text = "Multiple Choice" QuestionInput.Text = "You can run Android on a desktop computer natively." SizeInput = 44 FalseBtn clicked. EasyBtn clicked. SaveBtn clicked twice.	The question should be saved to trueorfalse.txt with a unique ID.	Pass	FGR9
16	difficulty SaveBtn	Ensure that questions with no difficulty are rejected	difficulty = string.Empty	The question will not save and SaveBtns text will	Pass	FGR19

			SaveBtn clicked twice.	turn red and read "No Difficulty".		
17	questionType Selector QuestionInput SaveBtn	Ensure that if any of the textboxes are blank and the question type is not Fill in the Blank or True or False, the question will be rejected.	questionType Selector.Text = "Checkbox" QuestionInput.Text = string.Empty SaveBtn clicked twice.	The question will not save and SaveBtns text will turn red and read "No Question".	Pass	FGR19
18	questionType Selector FirstInput SaveBtn	Ensure that if any of the textboxes have not had their text changed from the default and the question type is not Fill in the Blank or True or False, the question will be rejected.	questionType Selector.Text = "Checkbox" FirstInput.Text = "First Option" SaveBtn clicked twice.	The question will not save and SaveBtns text will turn red and read "No Question".	Pass	FGR19
19	questionType Selector	Ensure that if the selected question type is True or False	questionType Selector.Text = "Checkbox"	The question will not save and SaveBtns text will	Pass	FGR19

	QuestionInput SaveBtn	or Fill in the Blank and QuestionInput is blank or the default text, the question will be rejected.	FirstInput.Text = "First Option" SaveBtn clicked twice.	turn red and read "No Question".		
20	questionType Selector answer SaveBtn	Ensure that the question is rejected if the user has not selected an answer for their question if the question type is not Checkbox or Fill in the Blank.	questionType Selector.Text = "Multiple Choice" answer = string.Empty SaveBtn clicked twice.	The question will not save and SaveBtns text will turn red and read "No Answer".	Pass	FGR19
21	questionType Selector checkBox1 checkBox2 checkBox3 checkBox4 SaveBtn	Ensure that the question is rejected if the user has not selected any answer for a Checkbox question	questionType Selector.Text = "Multiple Choice" checkBox1 unchecked. checkBox2 unchecked. checkBox3 unchecked.	The question will not save and SaveBtns text will turn red and read "No Answer".	Pass	FGR19

			checkbox4 unchecked. SaveBtn clicked twice.			
22	questionType Selector FirstInput SecondInput ThirdInput FourthInput checkbox1 checkbox2 checkbox3 SaveBtn	Ensure that the question is rejected if the user has not filled in all of the answer textboxes they wanted for a Fill in the Blank question.	questionType Selector.Text = "Multiple Choice" FirstInput.Text = "" SecondInput = "Alternative Answer 1" ThirdInput.Text = "" FourthInput.Text = "" checkbox1 checked. checkbox2 checked. checkbox3 checked. SaveBtn clicked twice.	The question will not save and SaveBtns text will turn red and read "No Answer".	Pass	FGR19

23	QuestionInput	Ensure that text is only cleared from the textbox when it is clicked for the first time after picking a question type.	QuestionInput clicked twice. “test” typed into textbox between clicks.	The textbox will be cleared the first time but will not the second time it is clicked.	Pass	N/A
24	FirstInput	Ensure that text is only cleared from the textbox when it is clicked for the first time after picking a question type.	FirstInput clicked twice. “test” typed into textbox between clicks.	The textbox will be cleared the first time but will not the second time it is clicked.	Pass	N/A
25	SecondInput	Ensure that text is only cleared from the textbox when it is clicked for the first time after picking a question type.	SecondInput clicked twice. “test” typed into textbox between clicks.	The textbox will be cleared the first time but will not the second time it is clicked.	Pass	N/A
26	ThirdInput	Ensure that text is only cleared from the textbox when it is clicked for the	ThirdInput clicked twice. “test” typed into textbox between	The textbox will be cleared the first time but will not the second time it is	Pass	N/A

		first time after picking a question type.	clicks.	clicked.		
27	FourthInput	Ensure that text is only cleared from the textbox when it is clicked for the first time after picking a question type.	FourthInput clicked twice. “test” typed into textbox between clicks.	The textbox will be cleared the first time but will not the second time it is clicked.	Pass	N/A
28	questionType Selector	Ensure that clicking the questionType Selector will set its forecolour to white.	questionType Selector clicked	questionType Selector’s forecolour will be set to white, clearing the error indicator.	Pass	FGR19
29	SaveBtn	Ensure that the user has to click this button twice to save the question	SaveBtn clicked. SaveFirstClick = true.	Text on button should change to “Are you sure?” and SaveFirstClick should be set to false, both reverting to their normal values if the button isn’t clicked again in 3 seconds.	Pass	FGR16

30	SaveBtn	Ensure that program will exit infinite loop if all question IDs are taken.	SaveBtn clicked twice. All IDs used.	MessageBox will show up telling the user that too many questions have been added to that file if the quiz is still generating IDs after 1.5s and will then exit the saving code.	Pass	FGR19
31	SaveBtn	Ensure that program fails gracefully if questions text file either does not exist or is corrupted, by showing the user an error message.	Attempt to save question to deleted file.	An error message will show up telling the user that the file is corrupted or missing.	Fail	FGR19
32	questionType Selector EasyBtn HardBtn PreviewBtn	Ensure that these buttons play click.wav when clicked.	Common_Variables .muted = false questionType Selector clicked. EasyBtn clicked. HardBtn clicked.	All of these will not play click.wav when clicked	Pass	N/A

	SaveBtn		PreviewBtn clicked.			
	TextView		SaveBtn clicked.			
	TrueBtn		TextView clicked.			
	FalseBtn		TrueBtn clicked.			
	ReturnBtn		FalseBtn clicked.			
	radioButton1		ReturnBtn clicked.			
	radioButton2		radioButton1 clicked.			
	radioButton3		radioButton2 clicked.			
	radioButton4		radioButton3 clicked.			
	checkBox1		radioButton4 clicked.			
	checkBox2		checkBox1 clicked.			
	checkBox3		checkBox2 clicked.			
	checkBox4		checkBox3 clicked.			
	Common_Variables .muted = false					

			checkBox4 clicked.			
33	questionType Selector EasyBtn HardBtn PreviewBtn SaveBtn TextView TrueBtn FalseBtn ReturnBtn radioButton1 radioButton2 radioButton3 radioButton4	Ensure that these buttons play click.wav when clicked.	Common_Variables .muted = false questionType Selector clicked. EasyBtn clicked. HardBtn clicked. PreviewBtn clicked. SaveBtn clicked. TextView clicked. TrueBtn clicked. FalseBtn clicked. ReturnBtn clicked. radioButton1 clicked. radioButton2	All of these will not play click.wav when clicked	Pass	FGR13

Candidate Number: 2217

Centre Number: 71955

	<code>checkBox1</code>		<code>clicked.</code>			
	<code>checkBox2</code>		<code>radioButton3</code> <code>clicked.</code>			
	<code>checkBox3</code>		<code>radioButton4</code> <code>clicked.</code>			
	<code>checkBox4</code>					
	<code>Common_Variables</code> <code>.muted = true</code>		<code>checkBox1 clicked.</code> <code>checkBox2 clicked.</code> <code>checkBox3 clicked.</code> <code>checkBox4 clicked.</code>			

MultiChoice

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionHeader Common_Variables .UsedQuestions	Ensure that a random question is chosen when NewQuestion is called.	Load the form multiple times	A different question will appear each time the form loads. Questions ID is added Common_Variables .UsedQuestions	Pass	FIR5
2	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "e"	Only easy questions should appear.	Pass	FGR10
3	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "h"	Only hard questions should appear.	Pass	FGR10

4	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "n"	Questions of all difficulties should appear.	Pass	FGR10
5	radioButton1 radioButton2 radioButton3 radioButton4	Ensure that options for the question are randomised and that no option is duplicated.	Load form 5 times.	No option should be in the same place all 5 times and no option should be duplicated	Pass	FIR5
6	questions.Split[1] radioButton3 questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect	Ensure than a question can be answered correctly.	radioButton3.Text = "Wi-Fi" questions.Split[1] = "Wi-Fi" questions.guess = radioButton3.Text Common_Variables .difficulty = "e" AnsCheck clicked	Ding.wav should be played and 0.5 should be added to Common_Variables .score as well as 1 to Common_Variables .MultiLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8

	Common_Variables .score AnsCheck					
7	questions.Split[1] radioButton3 questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure than a question can be answered correctly.	radioButton3.Text = "Wi-Fi" questions.Split[1] = "Wi- Fi" questions.guess = radioButton3.Text Common_Variables .difficulty = "n" AnsCheck clicked	Ding.wav should be played and 0.75 should be added to Common_Variables .score as well as 1 to Common_Variables .MultiLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8
8	questions.Split[1]	Ensure than a question can be	radioButton2.Text = "Android"	Ding.wav should be played and 1 should	Pass	FIR1 FIR7

	radioButton2 questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	answered correctly.	questions.Split[1] = "Android" questions.guess = radioButton2.Text Common_Variables .difficulty = "h" AnsCheck clicked	be added to Common_Variables .score as well as 1 to Common_Variables .MultiLifeCorrect & Common_Variables .LifetimeCorrect.		FIR8
9	questions.Split[1] radioButton3 questions.guess Common_Variables .difficulty	Ensure than a question can be answered incorrectly.	radioButton3.Text = "Wireless Ethernet" questions.Split[1] = "Wi- Fi" questions.guess = radioButton3.Text Common_Variables .difficulty = "e"	Wrong.wav should be played and 1 should be added to Common_Variables .MultiLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FIR1 FIR7

	Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck		AnsCheck clicked			
10	questions.Split[1] radioButton3 questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables	Ensure than a question can be answered incorrectly.	radioButton3.Text = "Wireless Ethernet" questions.Split[1] = "Wi-Fi" questions.guess = radioButton3.Text Common_Variables .difficulty = "n" AnsCheck clicked	Wrong.wav should be played and 1 should be added to Common_Variables .MultiLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FIR1 FIR7

	.score AnsCheck					
11	questions.Split[1] radioButton2 questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure than a question can be answered incorrectly.	radioButton2.Text = "iOS" questions.Split[1] = "Android" questions.guess = radioButton2.Text Common_Variables .difficulty = "h" AnsCheck clicked	Wrong.wav should be played and 1 should be added to Common_Variables .MultiLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FIR1 FIR7
12	questions.Split[1] radioButton2	Ensure than a question can be answered correctly.	radioButton2.Text = "iOS"	Wrong.wav should not be played and 1 should be added to	Pass	FIR1 FIR7 FGR13

	questions.guess Common_Variables .difficulty Common_Variables .MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score Common_Variables .muted AnsCheck		questions.Split[1] = "Android" questions.guess = radioButton2.Text Common_Variables .difficulty = "h" AnsCheck clicked Common_Variables .muted = true	Common_Variables .MultiLifeIncorrect & Common_Variables .LifetimeIncorrect.		
13	questions.Split[1] radioButton2 questions.guess Common_Variables .difficulty Common_Variables	Ensure than a question can be answered correctly.	radioButton2.Text = "Android" questions.Split[1] = "Android" questions.guess = radioButton2.Text Common_Variables	Ding.wav should not be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .MultiLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FGR13

	.MultiLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck Common_Variables .muted		.difficulty = "h" Common_Variables .muted = true AnsCheck clicked			
14	AdminText Common_Variables .isAdmin	Ensure that AdminText is not shown to the user if they are not an admin.	Common_Variables .isAdmin = false	AdminTxt is hidden.	Pass	FGR16
15	QuestionHeader radioButton1 radioButton2 radioButton3 radioButton4	Ensure that these items have their font set to <i>Comic Sans MS</i> is dyslexia mode is enabled in the settings.	Common_Variables .dyslexia = true.	All of these items should have their font set to <i>Comic Sans</i> <i>MS</i> .	Pass	NFR7

	AdminTxt NoAns TimerLabel Common_Variables .dyslexia					
16	QuestionHeader radioButton1 radioButton2 radioButton3 radioButton4 event_selected	Ensure that the font size for these items changes when SmallText is the selected event.	event_selected = 0	QuestionHeaders font size set to 12. radioButtons font size set to 8.	Pass	FIR2
17	QuestionHeader radioButton1 radioButton2	Ensure that the font size for these items changes when LargeText is the selected event.	event_selected = 1	QuestionHeaders font size set to 84. radioButtons font size set to 64.	Pass	FIR2

	radioButton3 radioButton4 event_selected					
18	QuestionHeader radioButton1 radioButton2 radioButton3 radioButton4 event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "n"	Timer is set to 15 seconds and ticks down 1 every second.	Pass	FIR2
19	QuestionHeader radioButton1 radioButton2	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "h"	Timer is set to 10 seconds and ticks down 1 every second.	Pass	FIR2

	radioButton3 radioButton4 event_selected Common_Variables .difficulty					
20	QuestionHeader TextBlock1 TextBlock2 TextBlock3 TextBlock4 event_selected	Ensure that TextBlocks are shown in random places on the screen, without blocking the question.	event_selected = 3	TextBlockers are randomly distributed between 43, 600 & 163, 1038	Pass	FIR2
21	QuestionHeader radioButton1 radioButton2	Ensure that the text on QuestionHeader and the radioButtons is reversed.	event_selected = 4	All text on QuestionHeader and the radioButtons is reversed. isReverse is set to	Pass	FIR2

	radioButton3 radioButton4 event_selected isReverse			true.		
22	QuestionHeader event_selected event_probability	Ensure that the font for QuestionHeader is set to Wingdings	event_selected = 5	All text on QuestionHeader is set to wingdings.	Pass	FIR2
23	QuestionHeader radioButton1 radioButton2 radioButton3 radioButton4 event_selected event_probability	Ensure that the font for QuestionHeader and the radioButtons is set to Wingdings	event_selected = 5 event_probability = 1	All text on QuestionHeader and the radioButtons is reversed.	Pass	FIR2

24	TimeLeft	Ensure that when the timer runs out, the answer checking code is ran.	TimeLeft = 0	The answer checking code is ran.	Pass	FIR2
25	radioButton1 radioButton2 radioButton3 radioButton4 Commonm_Variables .muted	Ensure that these buttons play click.wav if the game is not muted.	radioButton1 clicked radioButton2 clicked radioButton3 clicked radioButton4 clicked Commonm_Variables .muted = false	Click.wav is played.	Pass	N/A
26	radioButton1 radioButton2 radioButton3 radioButton4 Commonm_Variables	Ensure that these buttons will not play click.wav if the game is muted.	radioButton1 clicked radioButton2 clicked radioButton3 clicked radioButton4 clicked Commonm_Variables	Click.wav is not played.	Pass	FGR13

	.muted		.muted = true			
27	radioButton1 radioButton2 radioButton3 radioButton4 NoAns CheckAns	Ensure that the user has to give an answer to proceed.	radioButton1 unchecked radioButton2 unchecked radioButton3 unchecked radioButton4 unchecked CheckAns clicked	NoAns is shown to the user.	Pass	FGR19 FIR7 FIR9
28	Common_Variables .ReadQuestion Common_Variables .muted	Ensure that the correct question is read out when question reading is enabled and the game is not muted.	Common_Variables .ReadQuestion = true questions.Split[3] = "mA" Common_Variables .muted = false	mA.wav is played.	Pass	FGR13 NFR7

Checkbox

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionHeader Common_Variables .UsedQuestions	Ensure that a random question is chosen when NewQuestion is called.	Load the form multiple times	A different question will appear each time the form loads. Questions ID is added Common_Variables .UsedQuestions	Pass	FIR5
2	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "e"	Only easy questions should appear.	Pass	FGR10
3	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "h"	Only hard questions should appear.	Pass	FGR10

4	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "n"	Questions of all difficulties should appear.	Pass	FGR10
5	checkBox1 checkBox2 checkBox4 questions.Split[1] cbcombined Common_Variables .difficulty Common_Variables .CheckLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that a question can be answered and points are given according to difficulty.	checkBox1 checked checkBox2 checked checkBox4 checked cbcombined = "124" questions.Split[1] = "124" Common_Variables .difficulty = "e" AnsCheck clicked	Ding.wav should be played and 0.5 should be added to Common_Variables .score as well as 1 to Common_Variables .CheckLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8

6	checkbox1 checkbox2 checkbox4 questions.Split[1] cbcombined Common_Variables .difficulty Common_Variables .CheckLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that a question can be answered correctly and points are given according to difficulty.	checkbox1 checked checkbox2 checked checkbox4 checked cbcombined = "124" questions.Split[1] = "124" Common_Variables .difficulty = "n" AnsCheck clicked	Ding.wav should be played and 0.75 should be added to Common_Variables .score as well as 1 to Common_Variables .CheckLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8
7	checkbox1 checkbox2 checkbox3	Ensure that a question can be answered correctly and points are given according to	checkbox1 checked checkbox2 checked checkbox3 checked	Ding.wav should be played and 1 should be added to Common_Variables .CheckLifeCorrect	Pass	FIR1 FIR7 FIR8

	questions.Split[1] cbcombined Common_Variables .difficulty Common_Variables .CheckLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	difficulty.	cbcombined = "123" questions.Split[1] = "123" Common_Variables .difficulty = "h" AnsCheck clicked	& Common_Variables .LifetimeCorrect.		
8	checkBox1 checkBox2 checkBox4 questions.Split[1] cbcombined	Ensure that a question can be answered wrongly.	checkBox1 checked checkBox2 checked checkBox4 checked cbcombined = "124" questions.Split[1] = "123"	Ding.wav should be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .CheckLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FIR1 FIR7

	Common_Variables .difficulty Common_Variables .CheckLifeIncorrect Common_Variables .LifetimeIncorrect Common_Variables .score AnsCheck		Common_Variables .difficulty = "h" AnsCheck clicked			
9	checkBox1 checkBox2 checkBox3 questions.Split[1] cbcombined Common_Variables .difficulty Common_Variables	Ensure that when a question is answered correctly but the game is muted, ding.wav will not be played.	checkBox1 checked checkBox2 checked checkBox3 checked cbcombined = "123" questions.Split[1] = "123" Common_Variables .difficulty = "h"	Ding.wav should not be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .CheckLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FGR13 FIR7

	.CheckLifeIncorrect Common_Variables .LifetimeIncorrect Common_Variables .score Common_Variables .muted AnsCheck		AnsCheck clicked Common_Variables .muted = true			
10	checkBox1 checkBox2 checkBox4 questions.Split[1] cbcombined Common_Variables .difficulty Common_Variables .CheckLifeIncorrect	Ensure that when a question is answered correctly but the game is muted, wrong.wav will not be played.	checkBox1 checked checkBox2 checked checkBox4 checked cbcombined = "124" questions.Split[1] = "123" Common_Variables .difficulty = "h" AnsCheck clicked	Wrong.wav should not be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .CheckLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FGR13 FIR7

	Common_Variables .LifetimeIncorrect Common_Variables .score Common_Variables .muted AnsCheck		Common_Variables .muted = true			
11	AdminText Common_Variables .isAdmin	Ensure that AdminText is not shown to the user if they are not an admin.	Common_Variables .isAdmin = false	AdminTxt is hidden.	Pass	FGR16
12	QuestionHeader checkBox1 checkBox2 checkBox3 checkBox4	Ensure that these items have their font set to <i>Comic Sans MS</i> is dyslexia mode is enabled in the settings.	Common_Variables .dyslexia = true.	All of these items should have their font set to <i>Comic Sans</i> <i>MS</i> .	Pass	NFR7

	AdminTxt NoAns TimerLabel Common_Variables .dyslexia					
13	QuestionHeader checkBox1 checkBox2 checkBox3 checkBox4 event_selected	Ensure that the font size for these items changes when SmallText is the selected event.	event_selected = 0	QuestionHeaders font size set to 12. checkBoxes font size set to 8.	Pass	FIR2
14	QuestionHeader checkBox1 checkBox2	Ensure that the font size for these items changes when LargeText is the selected event.	event_selected = 1	QuestionHeaders font size set to 84. checkBoxes font size set to 64.	Pass	FIR2

	checkbox3 checkbox4 event_selected					
15	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "n"	Timer is set to 15 seconds and ticks down 1 every second.	Pass	FIR2
16	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "h"	Timer is set to 10 seconds and ticks down 1 every second.	Pass	FIR2
17	QuestionHeader TextBlock1 TextBlock2	Ensure that TextBlocks are shown in random places on the screen, without blocking the question.	event_selected = 3	TextBlockers are randomly distributed between 43, 600 & 163, 1038	Pass	FIR2

	TextBlock3 TextBlock4 event_selected					
18	QuestionHeader checkBox1 checkBox2 checkBox3 checkBox4 event_selected isReverse	Ensure that the text on QuestionHeader and the checkBoxes is reversed.	event_selected = 4	All text on QuestionHeader and the checkBoxes is reversed. isReverse is set to true.	Pass	FIR2
19	QuestionHeader event_selected event_probability	Ensure that the font for QuestionHeader is set to Wingdings	event_selected = 5	All text on QuestionHeader is set to wingdings.	Pass	FIR2

20	QuestionHeader checkBox1 checkBox2 checkBox3 checkBox4 event_selected event_probability	Ensure that the font for QuestionHeader and the checkBoxes is set to Wingdings	event_selected = 5 event_probability = 1	All text on QuestionHeader and the checkBoxes is reversed.	Pass	FIR2
21	TimeLeft	Ensure that when the timer runs out, the answer checking code is ran.	TimeLeft = 0	The answer checking code is ran.	Pass	FIR2
22	checkBox1 checkBox2 checkBox3 checkBox4	Ensure that these buttons play click.wav if the game is not muted.	checkBox1 clicked checkBox2 clicked checkBox3 clicked checkBox4 clicked	Click.wav is played.	Pass	N/A

	Common_Variables .muted		Common_Variables .muted = false			
23	checkBox1 checkBox2 checkBox3 checkBox4 Commonm_Variables .muted	Ensure that these buttons will not play click.wav if the game is muted.	checkBox1 clicked checkBox2 clicked checkBox3 clicked checkBox4 clicked Commonm_Variables .muted = true	Click.wav is not played.	Pass	FGR13
24	checkBox1 checkBox2 checkBox3 checkBox4 NoAns AnsCheck	Ensure that the user has to give an answer to proceed.	checkBox1 unchecked checkBox2 unchecked checkBox3 unchecked checkBox4 unchecked AnsCheck clicked	NoAns is shown to the user.	Pass	FIR7 FIR9 FGR19

25	Common_Variables .ReadQuestion Common_Variables .muted	Ensure that the correct question is read out when question reading is enabled and the game is not muted.	Common_Variables .ReadQuestion = true questions.Split[3] = "cA" Common_Variables .muted = false	cA.wav is played.	Pass	FGR13 NFR7
26	questions.Split[1] checkBox1 checkBox2 checkBox3 AdminText	Ensure that the correct answer is shown on AdminText if admin mode is enabled	checkBox1.Text = "Itanium" checkBox2.Text = "x86-64" checkBox3.Text = "Arm64"	AdminText is updated to read "Admin Mode: Itanium, x86-64, Arm64"	Pass	FGR16
27	AdminText Common_Variables .isAdmin	Ensure that AdminText is not shown to the user if they are not an admin.	Common_Variables .isAdmin = false	AdminTxt is hidden.	Pass	FGR16

TrueOrFalse

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionHeader Common_Variables .UsedQuestions	Ensure that a random question is chosen when NewQuestion is called.	Load the form multiple times	A different question will appear each time the form loads. Questions ID is added Common_Variables .UsedQuestions	Pass	FIR5
2	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "e"	Only easy questions should appear.	Pass	FGR10
3	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "h"	Only hard questions should appear.	Pass	FGR10
4	QuestionHeader	Ensure that only questions matching	Common_Variables .difficulty = "n"	Questions of all difficulties should	Pass	FGR10

	Common_Variables .difficulty	the users chosen difficulty are selected.		appear.		
5	questions.guess questions.Split[1] Common_Variables .difficulty Common_Variables .TorfLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that the user can answer a question correctly and will be rewarded with points according to their difficulty.	questions.guess = "true" questions.Split[1] = "true" Common_Variables .difficulty = "e" AnsCheck clicked	Ding.wav should be played and 0.5 should be added to Common_Variables .score as well as 1 to Common_Variables .TorfLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8
6	questions.guess questions.Split[1] Common_Variables .difficulty	Ensure that the user can answer a question correctly and will be rewarded with points according to their difficulty.	questions.guess = "true" questions.Split[1] = "true" Common_Variables	Ding.wav should be played and 0.75 should be added to Common_Variables .score as well as 1 to Common_Variables	Pass	FIR1 FIR7 FIR8

	Common_Variables .score Common_Variables .TorflifeCorrect Common_Variables .LifetimeCorrect AnsCheck		.difficulty = "n" AnsCheck clicked	.TorflifeCorrect & Common_Variables .LifetimeCorrect.		
7	questions.guess questions.Split[1] Common_Variables .difficulty Common_Variables .TorflifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that the user can answer a question correctly and will be rewarded with points according to their difficulty.	questions.guess = "true" questions.Split[1] = "true" Common_Variables .difficulty = "h" AnsCheck clicked	Ding.wav should be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .TorflifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FIR1 FIR7 FIR8

8	questions.guess questions.Split[1] Common_Variables .difficulty Common_Variables .TorfLifeCorrect Common_Variables .LifetimeCorrect AnsCheck Common_Variables .score	Ensure that the user can answer a question incorrectly.	questions.guess = "false" questions.Split[1] = "true" Common_Variables .difficulty = "h" AnsCheck clicked	Wrong.wav should be played and 1 should be added to Common_Variables .TorfLifeIncorrect & Common_Variables .LifetimeIncorrect.	Pass	FIR1 FIR7
9	questions.guess questions.Split[1] Common_Variables .difficulty Common_Variables .TorfLifeCorrect	Ensure that when a question is answered correctly but the game is muted, ding.wav will not be played.	questions.guess = "true" questions.Split[1] = "true" Common_Variables .difficulty = "h" AnsCheck clicked	Ding.wav should not be played and 1 should be added to Common_Variables .score as well as 1 to Common_Variables .TorfLifeCorrect & Common_Variables .LifetimeCorrect.	Pass	FGR13 FIR7 FIR8

	Common_Variables .LifetimeCorrect AnsCheck Common_Variables .muted Common_Variables .score		Common_Variables .muted = true			
10	questions.guess questions.Split[1] Common_Variables .difficulty Common_Variables .TorfLifeCorrect Common_Variables .LifetimeCorrect AnsCheck Common_Variables	Ensure that when a question is answered correctly but the game is muted, wrong.wav will not be played.	questions.guess = "false" questions.Split[1] = "true" Common_Variables .difficulty = "h" AnsCheck clicked	Wrong.wav should not be played and 1 should be added to Common_Variables.TorfLifeIncorrect & Common_Variables.LifetimeIncorrect.	Pass	FGR13 FIR7

	.score					
11	QuestionHeader event_selected	Ensure that the font size for these items changes when SmallText is the selected event.	event_selected = 0	QuestionHeaders font size set to 12.	Pass	FIR2
12	QuestionHeader event_selected	Ensure that the font size for these items changes when LargeText is the selected event.	event_selected = 1	QuestionHeaders font size set to 84.	Pass	FIR2
13	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "n"	Timer is set to 15 seconds and ticks down 1 every second.	Pass	FIR2
14	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and	event_selected = 2 Common_Variables .difficulty = "h"	Timer is set to 10 seconds and ticks down 1 every second.	Pass	FIR2

		that the timer label is shown and ticks down.				
15	QuestionHeader event_selected	Ensure that the text on QuestionHeader is reversed.	event_selected = 4	All text on QuestionHeader is reversed.	Pass	FIR2
16	TimeLeft	Ensure that when the timer runs out, the answer checking code is ran.	TimeLeft = 0	The answer checking code is ran.	Pass	FIR2
17	Common_Variables .ReadQuestion Common_Variables .muted	Ensure that the correct question is read out when question reading is enabled and the game is not muted.	Common_Variables .ReadQuestion = true questions.Split[3] = "tA" Common_Variables .muted = false	tA.wav is played.	Pass	FGR13 NFR7
18	TrueBtn FalseBtn questions.guess	Ensure that clicking the TrueBtn will make the users guess true and will	TrueBtn.BorderSize = 0 FalseBtn.BorderSize = 5	TrueBtns border size is set to 5 while FalseBtns is set to 0. questions.guess is set	Pass	N/A

		highlight the button.		to true		
19	TrueBtn FalseBtn questions.guess	Ensure that clicking the FalseBtn will make the users guess false and will highlight the button.	TrueBtn.BorderSize = 5 FalseBtn.BorderSize = 0	FalseBtns border size is set to 5 while TrueBtns is set to 0. questions.guess is set to false.	Pass	N/A
20	AdminText Common_Variables.isAdmin	Ensure that AdminText is hidden if the user is not in admin mode.	Common_Variables.isAdmin = false	AdminText is hidden.	Pass	FGR16
21	AdminText questions.Split[1]	Ensure that AdminText is updated to show the correct answer.	questions.Split[1] = "true"	AdminText reads = "Admin Mode: true"	Pass	FGR16
22	NoAns questions.guess AnsCheck	Ensure that the user is told to select an answer if they did not.	question.guess = "no ans" AnsCheck clicked	NoAns should be shown.	Pass	FIR7 FIR9 FGR19

23	TrueBtn FalseBtn Common_Variables .muted	Ensure that the buttons play click.wav when clicked if the game is not muted.	TrueBtn clicked FalseBtn clicked Common_Variables .muted = false.	Click.wav is played	Pass	N/A
24	TrueBtn FalseBtn Common_Variables .muted	Ensure that the buttons do not play click.wav when clicked if the game is muted.	TrueBtn clicked FalseBtn clicked Common_Variables .muted = true.	Click.wav is not played	Pass	FGR13
25	QuestionHeader TrueBtn FalseBtn AdminTxt NoAns TimerLabel Common_Variables	Ensure that these items have their font set to <i>Comic Sans MS</i> if dyslexia mode is enabled in the settings.	Common_Variables .dyslexia = true.	All of these items should have their font set to <i>Comic Sans MS</i> .	Pass	NFR7

	.dyslexia					
26	event_selected	Ensure that event_selected does not fall out of the index of possible events	Trigger special events 5 times.	event_selected should never be greater than the index of SpecialEventList.	Fail	FIR2

FillInTheBlank

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionHeader Common_Variables .UsedQuestions	Ensure that a random question is chosen when NewQuestion is called.	Load the form multiple times	A different question will appear each time the form loads. Questions ID is added Common_Variables .UsedQuestions	Pass	FIR5
2	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "e"	Only easy questions should appear.	Pass	FGR10
3	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "h"	Only hard questions should appear.	Pass	FGR10
4	QuestionHeader	Ensure that only questions matching	Common_Variables .difficulty = "n"	Questions of all difficulties should	Pass	FGR10

	Common_Variables .difficulty	the users chosen difficulty are selected.		appear.		
5	AnswerInput AnsCheck	Ensure that any whitespace is removed from the input and that it is set to lowercase.	AnswerInput.Text "a B c D" AnsCheck clicked.	AnswerInput.Text should be changed to "abcd"	Pass	N/A
6	AnswerInput AnsCheck Common_Variables .difficulty Common_Variables .score Common_Variables .BlankLifeCorrect Common_Variables .LifetimeCorrect answer	Ensure that a question can be answered correctly.	AnswerInput.Text = "displayport" answer = ["hdmi", "displayport", "dvi", "power"] Common_Variables .difficulty = "e"	Ding.wav should be played and 1 should be added to Common_Variables. BlankLifeCorrect and Common_Variables. LifetimeCorrect. 0.5 should be added to Common_Variables. score	Pass	FIR1 FIR7 FIR8

7	AnswerInput AnsCheck Common_Variables .difficulty Common_Variables .score Common_Variables .BlankLifeCorrect Common_Variables .LifetimeCorrect answer	Ensure that a question can be answered correctly.	AnswerInput.Text = "displayport" answer = ["hdmi", "displayport", "dvi", "power"] Common_Variables .difficulty = "n"	Ding.wav should be played and 1 should be added to Common_Variables.BlankLifeCorrect and Common_Variables.LifetimeCorrect. 0.75 should be added to Common_Variables.score	Pass	FIR1 FIR7 FIR8
8	AnswerInput AnsCheck Common_Variables .difficulty Common_Variables .score	Ensure that a question can be answered correctly.	AnswerInput.Text = "root" answer = ["root", "admin", "administrator"] Common_Variables .difficulty = "h"	Ding.wav should be played and 1 should be added to Common_Variables.BlankLifeCorrect and Common_Variables.LifetimeCorrect.	Pass	FIR1 FIR7 FIR8

	Common_Variables .BlankLifeCorrect Common_Variables .LifetimeCorrect answer			1 should be added to Common_Variables. score		
9	AnswerInput AnsCheck Common_Variables .difficulty Common_Variables .BlankLifeIncorrect Common_Variables .LifetimeIncorrect answer	Ensure that a question can be answered incorrectly.	AnswerInput.Text = "sardine tourism" answer = ["root", "admin", "administrator"] Common_Variables .difficulty = "h"	Wrong.wav should be played and 1 should be added to Common_Variables. BlankLifeIncorrect and Common_Variables. LifetimeIncorrect.	Pass	FIR1 FIR7
10	AnswerInput AnsCheck	Ensure that a question can be answered correctly and that ding.wav	AnswerInput.Text = "displayport"	Ding.wav should not be played and 1 should be added to Common_Variables.	Pass	FGR13 FIR7 FIR8

	Common_Variables .difficulty Common_Variables .score Common_Variables .BlankLifeCorrect Common_Variables .LifetimeCorrect Answer Common_Variables .muted	does not play as the quiz is muted.	answer = ["hdmi", "displayport", "dvi", "power"] Common_Variables .difficulty = "n" Common_Variables .muted = true	BlankLifeCorrect and Common_Variables. LifetimeCorrect. 0.75 should be added to Common_Variables. score		
11	AnswerInput AnsCheck Common_Variables .difficulty Common_Variables .score Common_Variables	Ensure that a question can be answered incorrectly, and that wrong.wav does not play as the quiz is muted.	AnswerInput.Text = "sardine tourism" answer = ["root", "admin", "administrator"] Common_Variables .difficulty = "h" Common_Variables	Wrong.wav should not be played and 1 should be added to Common_Variables. BlankLifeIncorrect and Common_Variables. LifetimeIncorrect.	Pass	FGR13 FIR7

	.BlankLifeCorrect Common_Variables .LifetimeCorrect answer Common_Variables .muted		.muted = true			
12	QuestionHeader event_selected	Ensure that the font size for these items changes when SmallText is the selected event.	event_selected = 0	QuestionHeaders font size set to 12.	Pass	FIR2
13	QuestionHeader event_selected	Ensure that the font size for these items changes when LargeText is the selected event.	event_selected = 1	QuestionHeaders font size set to 84.	Pass	FIR2
14	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and	event_selected = 2 Common_Variables .difficulty = "n"	Timer is set to 15 seconds and ticks down 1 every second.	Pass	FIR2

		ticks down.				
15	event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "h"	Timer is set to 10 seconds and ticks down 1 every second.	Pass	FIR2
16	QuestionHeader event_selected	Ensure that the text on QuestionHeader is reversed.	event_selected = 4	All text on QuestionHeader is reversed.	Pass	FIR2
17	TimeLeft	Ensure that when the timer runs out, the answer checking code is ran.	TimeLeft = 0	The answer checking code is ran.	Pass	FIR2
18	AnswerInput	Ensure that AnswerInput is cleared if it is clicked for the first time that question.	AnswerInput.Text = "Enter Answer Here" AnswerInput clicked	AnswerInput.Text is cleared.	Pass	N/A

19	AnswerInput NoAns AnsCheck	Ensure that the user is told to answer the question if they provide no answer.	AnswerInput.Text = "Enter Answer Here" AnswerInput.Text = "" AnsCheck clicked	NoAns is shown.	Pass	FIR7 FIR9 FGR19
20	Common_Variables .ReadQuestion Common_Variables .muted	Ensure that the correct question is read out when question reading is enabled and the game is not muted.	Common_Variables .ReadQuestion = true questions.Split[3] = "fFIN" Common_Variables .muted = false	fFIN.wav is played.	Pass	FGR13 NFR7
21	Common_Variables .isAdmin AdminText	Ensure that AdminText is hidden if the user is not in Admin mode	Common_Variables .isAdmin = false	AdminText is hidden.	Pass	FGR16
22	AdminText questions.Split	Ensure that AdminText is updated.	Common_Variables .isAdmin = false	AdminText is set to questions.Split[1].	Pass	FGR16

23	QuestionHeader AnswerInput AdminTxt NoAns TimerLabel Common_Variables .dyslexia	Ensure that these items have their font set to <i>Comic Sans MS</i> is dyslexia mode is enabled in the settings.	Common_Variables .dyslexia = true.	All of these items should have their font set to <i>Comic Sans MS</i> .	Pass	NFR7
24	event_selected	Ensure that event_selected does not fall out of the index of possible events	Trigger special events 5 times.	event_selected should never be greater than the index of SpecialEventList.	Fail	FIR2

DragAndDrop

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	QuestionHeader Common_Variables .UsedQuestions	Ensure that a random question is chosen when NewQuestion is called.	Load the form multiple times	A different question will appear each time the form loads. Questions ID is added Common_Variables .UsedQuestions	Pass	FIR5
2	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "e"	Only easy questions should appear.	Pass	FGR10
3	QuestionHeader Common_Variables .difficulty	Ensure that only questions matching the users chosen difficulty are selected.	Common_Variables .difficulty = "h"	Only hard questions should appear.	Pass	FGR10
4	QuestionHeader	Ensure that only questions matching	Common_Variables .difficulty = "n"	Questions off all difficulties should	Pass	FGR10

	Common_Variables .difficulty	the users chosen difficulty are selected .		appear.		
5	QuestionHeader pictureBox5 pictureBox6 pictureBox7 pictureBox8 Common_Variables .difficulty = "e" answer Common_Variables .DropLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that question can be answered correctly	QuestionHeader = "Match these logos to their standard" pictureBox5.Tag = "PCIe" pictureBox6.Tag = "SATA" pictureBox7.Tag = "DisplayPort" pictureBox8.Tag = "Bluetooth" Common_Variables .difficulty = "e" answer = "PCIe SATA DisplayPort Bluetooth" AnsCheck clicked	Ding.wav should be played and 0.5 should be added to the users score. 1 should be added to both DropLifeCorrect and LifetimeCorrect.	Pass	FIR1 FIR7 FIR8

6	QuestionHeader pictureBox5 pictureBox6 pictureBox7 pictureBox8 Common_Variables .difficulty = "n" answer Common_Variables .DropLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	Ensure that question can be answered correctly	QuestionHeader = "Match these logos to their standard" pictureBox5.Tag = "PCIe" pictureBox6.Tag = "SATA" pictureBox7.Tag = "DisplayPort" pictureBox8.Tag = "Bluetooth" Common_Variables .difficulty = "n" answer = "PCIe SATA DisplayPort Bluetooth" AnsCheck clicked	Ding.wav should be played and 0.75 should be added to the users score. 1 should be added to both DropLifeCorrect and LifetimeCorrect.	Pass	FIR1 FIR7 FIR8
7	QuestionHeader	Ensure that question can be answered	QuestionHeader = "Match these logos to	Ding.wav should be played and 1 should	Pass	FIR1 FIR7 FIR8

	pictureBox5 pictureBox6 pictureBox7 pictureBox8 Common_Variables .difficulty = "n" answer Common_Variables .DropLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score AnsCheck	correctly	their standard" pictureBox5.Tag = "Kotlin" pictureBox6.Tag = "Go" pictureBox7.Tag = "Rust" pictureBox8.Tag = "HolyC" Common_Variables .difficulty = "h" answer = "KotlinGoRustHolyC" AnsCheck clicked	be added to the users score. 1 should be added to both DropLifeCorrect and LifetimeCorrect.		
8	Common_Variables .muted AnsCheck	Ensure that question can be answered correctly but that ding.wav does not	QuestionHeader = "Match these logos to their standard"	Ding.wav should not be played and 1 should be added to the users score.	Pass	FGR13 FIR7 FIR8

	pictureBox5 pictureBox6 pictureBox7 pictureBox8 Common_Variables .difficulty = "n" answer Common_Variables .DropLifeCorrect Common_Variables .LifetimeCorrect Common_Variables .score	play as the quiz is muted.	pictureBox5.Tag = "Kotlin" pictureBox6.Tag = "Go" pictureBox7.Tag = "Rust" pictureBox8.Tag = "HolyC" Common_Variables .difficulty = "h" answer = "KotlinGoRustHolyC" AnsCheck clicked Common_Variables .muted = true	1 should be added to both DropLifeCorrect and LifetimeCorrect.		
9	QuestionHeader pictureBox5 pictureBox6	Ensure that question can be answered incorrectly	QuestionHeader = "Match these logos to their standard" pictureBox5.Tag = "Go"	wrong.wav should be played 1 should be added to both DropLifeIncorrect and LifetimeIncorrect.	Pass	FIR1 FIR7 FIR8

	pictureBox7 pictureBox8 Common_Variables .difficulty = "n" answer Common_Variables .DropLifeIncorrect Common_Variables .LifetimeIncorrect Common_Variables .score AnsCheck		pictureBox6.Tag = "Kotlin" pictureBox7.Tag = "Rust" pictureBox8.Tag = "HolyC" Common_Variables .difficulty = "h" answer = "KotlinGoRustHolyC" AnsCheck clicked			
10	QuestionHeader pictureBox5 pictureBox6 pictureBox7	Ensure that question can be answered incorrectly and that wrong.wav does not play as the quiz is muted.	QuestionHeader = "Match these logos to their standard" pictureBox5.Tag = "Go" pictureBox6.Tag = "Kotlin"	wrong.wav should not be played 1 should be added to both DropLifeIncorrect and LifetimeIncorrect.	Pass	FGR13 FIR7 FIR8

	pictureBox8 Common_Variables .difficulty = "n" answer Common_Variables .DropLifeIncorrect Common_Variables .LifetimeIncorrect Common_Variables .score AnsCheck Common_Variables .muted		pictureBox7.Tag = "Rust" pictureBox8.Tag = "HolyC" Common_Variables .difficulty = "h" answer = "KotlinGoRustHolyC" AnsCheck clicked Common_Variables .muted = true			
11	AnsCheck pictureBox5 pictureBox6 pictureBox7	Ensure that the user is told to answer the question if they have not populated all of the pictureBoxes.	pictureBox5.Tag = "" pictureBox6 Tag = "" pictureBox7 Tag = "" pictureBox8 Tag = ""	NoAns is shown.	Pass	FIR7 FIR9 FGR19

	pictureBox8 NoAns		AnsCheck clicked			
12	QuestionHeader AnsHeader1 AnsHeader2 AnsHeader3 AnsHeader4 AdminTxt1 AdminTxt2 AdminTxt3 AdminTxt4 One Two Three	Ensure that these items have their font set to <i>Comic Sans MS</i> is dyslexia mode is enabled in the settings.	Common_Variables .dyslexia = true.	All of these items should have their font set to <i>Comic Sans MS</i>.	Pass	NFR7

	Four NoAns TimerLabel Common_Variables .dyslexia					
13	QuestionHeader AnsHeader1 AnsHeader2 AnsHeader3 AnsHeader4 event_selected	Ensure that the font size for these items changes when SmallText is the selected event.	event_selected = 0	QuestionHeaders font size set to 12. AnsHeaders font size set to 8.	Pass	FIR2
14	QuestionHeader AnsHeader1 AnsHeader2	Ensure that the font size for these items changes when LargeText is the selected event.	event_selected = 1	QuestionHeaders font size set to 84. AnsHeaders font size set to 64.	Pass	FIR2

	AnsHeader3 AnsHeader4 event_selected					
15	QuestionHeader event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "n"	Timer is set to 15 seconds and ticks down 1 every second.	Pass	FIR2
16	QuestionHeader event_selected Common_Variables .difficulty	Ensure that the timer is set according to the games difficulty and that the timer label is shown and ticks down.	event_selected = 2 Common_Variables .difficulty = "h"	Timer is set to 10 seconds and ticks down 1 every second.	Pass	FIR2
17	QuestionHeader AnsHeader1 AnsHeader2	Ensure that the text on QuestionHeader and the AnsHeaders is reversed.	event_selected = 4	All text on QuestionHeader and the AnsHeaders is reversed.	Pass	FIR2

	AnsHeader3 AnsHeader4 event_selected					
18	QuestionHeader event_selected event_probability	Ensure that the font for QuestionHeader is set to Wingdings	event_selected = 5	All text on QuestionHeader is set to wingdings.	Pass	FIR2
19	TimeLeft	Ensure that when the timer runs out, the answer checking code is ran.	TimeLeft = 0	The answer checking code is ran.	Pass	FIR2
20	AdminText1 AdminText2 AdminText3 AdminText4 One	Ensure that all admin text is hidden if the user is not in admin mode.	Common_Variables.isAdmin = false.	AdminText1, AdminText2, AdminText3, AdminText4, One, Two, Three, Four are all hidden.	Pass	FGR16

	Two Three Four Common_Variables .isAdmin					
21	AdminText1 AdminText2 AdminText3 AdminText4	Ensure that AdminText is set to the correct answer for the pictureBox above it.	Generate a new question	Number on AdminText1- 4 should match the pictureBox of the answer.	Pass	FGR16
22	Common_Variables .ReadQuestion Common_Variables .muted	Ensure that the correct question is read out when question reading is enabled and the game is not muted.	Common_Variables .ReadQuestion = true questions.Split[3] = "dA" Common_Variables .muted = false	dA.wav is played.	Pass	FGR13 NFR7

EndScreen

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	LeaderboardData	Ensure that a new LeaderboardData file is created if there is not one already.	LeaderboardData.dat does not exist. Common_Variables .username = "TestUser" Common_Variables .score = 15 Common_Variables .LifetimeCorrect = 20 Common_Variables .LifetimeIncorrect = 0	LeaderboardData.dat File will be created and populated with current users Leaderboard statistics	Pass	FGR23
2	LeaderboardData	Ensure that the current LeaderboardData.dat file is added to if there is one.	LeaderboardData.dat exists. Common_Variables .username = "TestUser" Common_Variables	LeaderboardData.dat File will be added to with current users Leaderboard statistics	Fail	FGR23

			.score = 15 Common_Variables .LifetimeCorrect = 20 Common_Variables .LifetimeIncorrect = 0			
3	UserData	Ensure that the current users account statistics are updated when the quiz is over.	Common_Variables .LifetimeCorrect = 15 Common_Variables .LifetimeIncorrect = 0 Common_Variables .MultiLifeCorrect = 5 Common_Variables .MultiLifeIncorrect = 0 Common_Variables .CheckLifeCorrect = 5 Common_Variables .CheckLifeIncorrect = 0 Common_Variables .DropLifeCorrect = 5	Statistics stored in UserData will be updated to the current state of those variables	Fail	FGR3f

			Common_Variables .DropLifeIncorrect = 0 Common_Variables .TorfLifeCorrect = 5 Common_Variables .TorfLifeIncorrect = 0			
4	UserData	Program will fail gracefully if UserData file is not present.	UserData.dat deleted.	MessageBox will show up telling the user that the UserData file is missing.	Fail	FGR21
5	Common_Variables .score EndMessage EndScoreTxt	Ensure that appropriate end message and colour for EndScoreTxt is chosen according to the users final score.	Common_Variables .score = 0	EndScoreTxt has its colour set to DarkRed and its text set to the users current score. EndMessages text is set to a random line from VeryBadEnd.txt, with its font size being set to the one stored with the selected message.	Pass	N/A

6	Common_Variables .score EndMessage EndScoreTxt	Ensure that appropriate end message and colour for EndScoreTxt is chosen according to the users final score.	Common_Variables .score <= 3	EndScoreTxt has its colour set to Red and its text set to the users current score. EndMessages text is set to a random line from VeryBadEnd.txt, with its font size being set to the one stored with the selected message.	Pass	N/A
7	Common_Variables .score EndMessage EndScoreTxt	Ensure that appropriate end message and colour for EndScoreTxt is chosen according to the users final score.	Common_Variables .score > 3 && <= 8	EndScoreTxt has its colour set to OrangeRed and its text set to the users current score. EndMessages text is set to a random line from BadEnd.txt, with its font size being set to the one stored with the selected message.	Pass	N/A
8	Common_Variables .score	Ensure that appropriate end	Common_Variables .score > 8 && <= 12	EndScoreTxt has its colour set to Orange	Pass	N/A

	EndMessage EndScoreTxt Bronze	message and colour for EndScoreTxt is chosen according to the users final score. Ensure that appropriate trophy image is shown to the user.		and its text set to the users current score. EndMessages text is set to a random line from NormalEnd.txt, with its font size being set to the one stored with the selected message. Bronze has its image set to CopperTrophy.png.		
9	Common_Variables .score EndMessage EndScoreTxt Bronze Silver	Ensure that appropriate end message and colour for EndScoreTxt is chosen according to the users final score. Ensure that appropriate trophy image is shown to the user.	Common_Variables .score > 12 && <= 16	EndScoreTxt has its colour set to Lime and its text set to the users current score. EndMessages text is set to a random line from GoodEnd.txt, with its font size being set to the one stored with the selected message. Bronze has its image set to	Pass	N/A

				CopperTrophy.png, and Silver has its set to SilverTrophy.png.		
10	Common_Variables .score EndMessage EndScoreTxt Bronze Silver Gold	Ensure that appropriate end message and colour for EndScoreTxt is chosen according to the users final score. Ensure that appropriate trophy image is shown to the user.	Common_Variables .score > 17	EndScoreTxt has its colour set to LimeGreen and its text set to the users current score. EndMessages text is set to a random line from VeryGoodEnd.txt, with its font size being set to the one stored with the selected message. Bronze has its image set to CopperTrophy.png, Silver has its set to SilverTrophy.png and Gold has its set to GoldTrophy.png.	Pass	N/A
11	Random selection of end message.	Ensure that program fails gracefully if end		Program should show an error message	Fail	FGR21

		message file is deleted.	Temporarily move all End Message files out of ./assets/txt/EndScreen	explaining to the user that the end message file is not present and take them back to the lobby.		
12	GoToLeaderboard Common_Variables .score Common_Variables .q_number	Ensure that player can go to the Leaderboard from the EndScreen	GoToLeaderboard clicked	Program should take user to Leaderboard, reset Common_Variables .score and Common_Variables .q_number and then close the EndScreen form.	Fail	FGR11 FGR14
13	ReturnLobby Common_Variables .score Common_Variables .q_number	Ensure that player can go to the Lobby from the EndScreen.	ReturnLobby clicked	Program should take user to the Lobby, reset Common_Variables .score and Common_Variables .q_number and then close the EndScreen form.	Fail	FGR14
14	ReturnLobby	Ensure that both of these buttons play	ReturnLobby clicked	Click.wav is played	Pass	N/A

Candidate Number: 2217**Centre Number: 71955**

	GoToLeaderboard Common_Variables .muted	click.wav when clicked if the game is not muted.	GoToLeaderboard clicked Common_Variables .muted = false			
15	ReturnLobby GoToLeaderboard Common_Variables .muted	Ensure that both of these buttons don't play click.wav when clicked if the game is muted.	ReturnLobby clicked GoToLeaderboard clicked Common_Variables .muted = true	Click.wav is not played	Pass	N/A

TextViewer

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	ViewingHeader Common_Variables .FileToBeViewed	Ensure that ViewingHeader.Text is set to Common_Variables .FileToBeViewed when the form loads.	Common_Variables .FileToBeViewed = "BadEnd.txt"	ViewingHeader.Text will read "BadEnd.txt"	Pass	N/A
2	ViewingHeader dataGridView1 Common_Variables .FileToBeViewed (any End Message file)	Ensure that end message files are loaded onto the dataGridView1 properly and with appropriate header titles.	Common_Variables .FileToBeViewed = "VeryBadEnd.txt"	VeryBadEnd.txt should be loaded onto the dataGridView1 with the first columns header title being Font Size, and the second columns header title being Message. This will be the same for all end message text files.	Pass	FGR21

3	ViewingHeader dataGridView1 Common_Variables .FileToBeViewed (checkbox.txt or multichoice.txt)	Ensure that end message files are loaded onto the dataGridView1 properly and with appropriate header titles.	Common_Variables .FileToBeViewed = "multichoice.txt"	multichoice.txt should be loaded onto the dataGridView1 with the column headers of "Difficulty", "Answer", "Font Size", "ID", "Question", "Option 1", "Option 2", "Option 3", "Option 4" in that order. This will be the same for checkbox.txt.	Pass	FGR21
4	ViewingHeader dataGridView1 Common_Variables .FileToBeViewed	Ensure that end message files are loaded onto the dataGridView1 properly and with appropriate header titles.	Common_Variables .FileToBeViewed = "trueorfalse.txt"	trueorfalse.txt should be loaded onto the dataGridView1 with the column headers of "Difficulty", "Answer", "Font Size", "ID", "Question" in that order.	Pass	FGR21
5	ViewingHeader	Ensure that end message files are	Common_Variables .FileToBeViewed =	trueorfalse.txt should be loaded	Pass	FGR21

	dataGridView1 Common_Variables .FileToBeViewed	loaded onto the dataGridView1 properly and with appropriate header titles.	"fillinblank.txt"	onto the dataGridView1 with the column headers of "Difficulty", "Answer(s)", "Font Size", "ID", "Question" in that order.		
6	DeleteBtn DeleteFirstClick	Ensure that the user has to click the DeleteBtn twice before a question is deleted.	DeleteFirstClick = true DeleteBtn clicked.	DeleteBtn text should change to "Are you sure?" and DeleteFirstClick is set to false. If the button has not been clicked again after 3 seconds the buttons text will go back to normal and DeleteFirstClick will be set to true.	Pass	FGR21
7	DeleteBtn DeleteFirstClick dataGridView1 .CurrentCell.RowIndex	Ensure that when the user has clicked the DeleteBtn for a second time within the 3 second window, the row	DeleteFirstClick = false DeleteBtn clicked. Common_Variables .FileToBeViewed =	The 4 th line of text on multichoice.txt will be deleted and the dataGridView will reload to	Pass	FGR21

		they have selected on the dataGridView will be deleted.	"multichoice.txt" dataGridView1 .CurrentCell.RowIndex = 3	reflect the updated file. DeleteBtn's text will be set to "Delete Question".		
8	DeleteBtn dataGridView1	Ensure that DeleteBtn will be hidden if there is only 1 row left on the table, to prevent the user from crashing the program by deleting all the lines in a file.	dataGridView1.Rows .Count = 1	DeleteBtn hidden.	Pass	N/A
9	DeleteBtn Common_Variables .muted	Ensure that DeleteBtn plays click.wav when clicked if the game is not muted.	Common_Variables .muted = false DeleteBtn clicked.	Click.wav is played.	Pass	N/A
10	DeleteBtn Common_Variables .muted	Ensure that DeleteBtn will not play click.wav when clicked if the game is muted.	Common_Variables .muted = true DeleteBtn clicked.	Click.wav is not played.	Pass	FGR13

Candidate Number: 2217

Centre Number: 71955

11	Common_Variables .FileToBeViewed	Ensure that the program fails gracefully if the text file that is going to be read does not exist for whatever reason.	Common_Variables .FileToBeViewed = "multichoice.txt" (File deleted for test)	Program exits gracefully with an error message that tells the user what caused the program to exit.	Fail	FGR19
----	-------------------------------------	--	---	--	------	-------

Tutorial

Test No.	Being Tested	Test	Test Data	Expected Outcome	Result	Requirement Fulfilled
1	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "DesignerCheckbox"	The text for TutorialTxt and Header will be set to the text for DesignerCheckbox and the image on pictureBox1 will be set to DesignerCheck.png	Pass	FGR17
2	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "DesignerMulti"	The text for TutorialTxt and Header will be set to the text for DesignerMulti and the image on pictureBox1 will be set to DesignerMulti.png	Pass	FGR17

3	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "DesignerBlank"	The text for TutorialTxt and Header will be set to the text for DesignerBlank and the image on pictureBox1 will be set to DesignerBlank.png	Pass	FGR17
4	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "DesignerTorf"	The text for TutorialTxt and Header will be set to the text for DesignerTorf and the image on pictureBox1 will be set to DesignerTorf.png	Pass	FGR17
5	Common_Variables .CurrentPage TutorialTxt Header	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the	Common_Variables .CurrentPage = "Designer"	The text for TutorialTxt and Header will be set to the text for Designer and the image on	Pass	FGR17

	pictureBox1	tutorial page that is in Common_Variables .CurrentPage		pictureBox1 will be set to Designer.png		
6	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "CustomEnd"	The text for TutorialTxt and Header will be set to the text for CustomEnd and the image on pictureBox1 will be set to CustomEnd.png	Pass	FGR17
7	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "Multi"	The text for TutorialTxt and Header will be set to the text for Multi and the image on pictureBox1 will be set to Multichoice.png	Pass	FGR17

8	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "Checkbox"	The text for TutorialTxt and Header will be set to the text for Checkbox and the image on pictureBox1 will be set to Checkbox.png	Pass	FGR17
9	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "Torf"	The text for TutorialTxt and Header will be set to the text for Torf and the image on pictureBox1 will be set to TrueorFalse.png	Pass	FGR17
10	Common_Variables .CurrentPage TutorialTxt	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are	Common_Variables .CurrentPage = "Blank"	The text for TutorialTxt and Header will be set to the text for Blank and the image on	Pass	FGR17

	Header pictureBox1	appropriate for the tutorial page that is in Common_Variables .CurrentPage		pictureBox1 will be set to FillInTheBlank.png		
11	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables .CurrentPage	Common_Variables .CurrentPage = "Drop"	The text for TutorialTxt and Header will be set to the text for Drop and the image on pictureBox1 will be set to DragDrop.png	Pass	FGR17
12	Common_Variables .CurrentPage TutorialTxt Header pictureBox1	Ensure that the text for TutorialTxt and Header and the picture in pictureBox1 are appropriate for the tutorial page that is in Common_Variables	Common_Variables .CurrentPage = "Event"	The text for TutorialTxt and Header will be set to the text for Event and the image on pictureBox1 will be set to SpecialEvent.png	Pass	FGR17

		.CurrentPage				
13	GoFoward Common_Variables .CurrentPage	Ensure that clicking the GoFoward button will take the user to the next page of the Tutorial.	Common_Variables .CurrentPage = "Drop"	Common_Variables .CurrentPage is set to the value stored to the right of the current value. The form is updated to show the new page, in this case Event.	Pass	FGR17
14	GoBack Common_Variables .CurrentPage	Ensure that clicking the GoBack button will take the user to the previous page of the Tutorial.	Common_Variables .CurrentPage = "Drop"	Common_Variables .CurrentPage is set to the value stored to the left of the current value. The form is updated to show the new page, in this case Blank.	Pass	FGR17
15	GoFoward Common_Variables .CurrentPage	Ensure that clicking the GoFoward button on the last page of the Tutorial will send the user back to the first page.	Common_Variables .CurrentPage = "Event"	Common_Variables .CurrentPage is set to the first value in the array as the index of the current value + 1 is more than the length of	Pass	FGR17

				the array. The form is updated to show the new page, in this case Designer.		
16	GoBack Common_Variables .CurrentPage	Ensure that clicking the GoBack button on the first page of the Tutorial will take the user to the last page.	Common_Variables .CurrentPage = "Designer"	Common_Variables .CurrentPage is set to the last value in the array as the index of the current value - 1 is less than the length of the array. The form is updated to show the new page, in this case Event.	Pass	FGR17
17	GoBack GoFoward Common_Variables .muted	Ensure that clicking either of these buttons while the game is not muted will play click.wav	GoBack clicked GoFoward clicked Common_Variables .muted = false	Click.wav is played.	Pass	N/A

Candidate Number: 2217

Centre Number: 71955

18	GoBack GoFoward Common_Variables .muted	Ensure that clicking either of these buttons while the game is muted will not play click.wav	GoBack clicked GoFoward clicked Common_Variables .muted = false	Click.wav is not played.	Pass	FGR13
----	--	--	--	-----------------------------	------	-------

Failed Tests

CreateAccount

Test 13

This test failed as the account statistics variables stored in the program were not reset if the user logged out of an existing account before signing in with a newly created account. This is a problem as it could confuse the user into thinking that they signed into their old account or it could allow the user to see the data of another users account, which would make the program unsecure.

Code Before:

```
//Loads Lobby.cs when all checks on the users inputs are valid & then hides the CreateAccount screen
if (password_valid == true && username_valid == true && password_confirm == true)
{
    Common_Variables.username = UsernameInput.Text;
    if (!File.Exists(Common_Variables.UserData))
    {
        using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Create))
        {
            UserData newuser = new UserData(UsernameInput.Text, PasswordInput.Text);
            userlist.Add(newuser);
            serialise.Serialize(fileStream, userlist); // Serialise data using a list of user objects
            userlist.Clear();
            if (LoadLobby != null)
            {
                LoadLobby.Invoke(this, EventArgs.Empty);
            }
            this.Close();
        }
    }
    else // User file exists so check username not taken
    {
        //Get the list of users by deserializing to a list of user objects
        using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Open))
        {
            userlist = (List<UserData>)(deserialise.Deserialize(fileStream));
        }

        //Check the list to see if username already in use
        try
```

Code After:

```
//Loads Lobby.cs when all checks on the users inputs are valid & then hides the CreateAccount screen
if (password_valid == true && username_valid == true && password_confirm == true)
{
    Common_Variables.username = UsernameInput.Text;
    Common_Variables.BlankLifeCorrect = 0;
    Common_Variables.BlankLifeIncorrect = 0;
    Common_Variables.CheckLifeCorrect = 0;
    Common_Variables.CheckLifeIncorrect = 0;
    Common_Variables.DropLifeCorrect = 0;
    Common_Variables.DropLifeIncorrect = 0;
    Common_Variables.MultiLifeCorrect = 0;
    Common_Variables.MultiLifeIncorrect = 0;
    Common_Variables.TorLifeCorrect = 0;
    Common_Variables.TorLifeIncorrect = 0;
    Common_Variables.LifetimeCorrect = 0;
    Common_Variables.LifetimeIncorrect = 0;
    Common_Variables.isAdmin = false;
    Common_Variables.ReadQuestions = false;
    Common_Variables.dyslexia = false;
    Common_Variables.SkipEnd = false;
    if (!File.Exists(Common_Variables.UserData))
    {
        using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Create))
        {
            UserData newuser = new UserData(UsernameInput.Text, PasswordInput.Text);
            userlist.Add(newuser);
            serialise.Serialize(fileStream, userlist); // Serialise data using a list of user objects
            userlist.Clear();
            if (LoadLobby != null)
```

This error was fixed by setting the account statistics variables to 0 and all of the settings to false when an account is created, similar to how account statistics are loaded in in the Login script.

Login

Test 9

This test failed as any account made after the initial account created could not be signed into, as the UsernameInput field & PasswordInput field would be cleared too early. This is a catastrophic error as it means that the account system is fundamentally broken, as only one account can be used by users so the Leaderboard will contain inaccurate data and individual account statistics cannot be saved, as there is only one viable account.

Code Before:

```
else
{
    Wrong.Show();
    UsernameHeader.ForeColor = System.Drawing.Color.Red;
    PasswordHeader.ForeColor = System.Drawing.Color.Red;
    UsernameInput.Text = "";
    PasswordInput.Text = "";
}
```

Code After:

```
else
{
    Wrong.Show();
    UsernameHeader.ForeColor = System.Drawing.Color.Red;
    PasswordHeader.ForeColor = System.Drawing.Color.Red;
}
}
UsernameInput.Text = "";
PasswordInput.Text = "";
```

This error was fixed by moving the lines of code that blank the 2 textboxes outside of the foreach loop that checks if the account credentials are correct, meaning that that textboxes will not be blanked prematurely, restoring use of accounts other than the first account created.

Leaderboard

Test 2

This test failed as the game crashed when the user tried to sort the leaderboard by the LeaderboardData file which does not exist, resulting in the program crashing and throwing a System.IO.FileNotFoundException exception, which is nonrecoverable and requires a full restart of the program.

Code Before:

```
private void LoadData()
{
    using (Stream fileStream = File.OpenRead(Common_Variables.LeaderboardData))
    {
        Leaderlist = (List<LeaderboardData>)(deserialize.Deserialize(fileStream))
    }

    if (OrderBy == "Descending")
    {
        switch (SortBy)
        {
            case "Name":
                Leaderlist = Leaderlist.OrderBy(o => o.Username).ToList(); //OrderByDescending sorts from Z => A, which is not intuitive as most people
                break;
            case "Score":
                Leaderlist = Leaderlist.OrderByDescending(o => o.Score).ToList();
                break;
            case "LC":
                Leaderlist = Leaderlist.OrderByDescending(o => o.LifetimeCorrect).ToList();
                break;
            case "LIC":
                Leaderlist = Leaderlist.OrderByDescending(o => o.LifetimeIncorrect).ToList();
                break;
        }
    }
}
```

Exception Unhandled
System.IO.FileNotFoundException: 'Could not find file 'C:\Users\hello\OneDrive - C2k\School\Computing\Yr 13\Project\Yr-13-NathanMcGee-CA\assets\dat\LeaderboardData.dat'.'

Ask Copilot | Show Call Stack | View Details | Copy Details | Start Live Share session
Exception Settings

Code After:

```
private void Leaderboard_Load(object sender, EventArgs e)
{
    Error.Hide();
    DataGrid.RowTemplate.Height = 35;

    SortScoreBtn.FlatAppearance.BorderSize = 5;

    if (File.Exists(Common_Variables.LeaderboardData))
    {
        LoadData();
    }
    else
    {
        Error.Show();
        SortLCBtn.Hide();
        SortLICBtn.Hide();
        SortNameBtn.Hide();
        SortOrderBtn.Hide();
        SortScoreBtn.Hide();
    }
}
```

To fix this error I changed the code so that the sorting buttons are hidden when the error label is shown, which eliminated the change of the program crashing due to trying to sort a non-existent file, as the user cannot hit the buttons that would start the sorting process.

QuestionDesigner

Test 31

This test failed as the program crashes if the user attempts to save to a question file that no longer exists for whatever reason. Instead of failing gracefully with an error message that explains the error it will throw a `System.IO.FileNotFoundException` exception. This error would result in the program needing to be fully restarted without explanation.

Code Before:

```
case "Checkbox":
{
    if (checkBox1.Checked || checkBox2.Checked || checkBox3.Checked || checkBox4.Checked)
    {
        cbcombined = cb1ans + cb2ans + cb3ans + cb4ans;
        sw.Start();
        while (!valid)
        {
            ID = IDGen();
            string[] ReadFile = File.ReadAllLines(Common_Variables.path + @"assets\txt\checkbox.txt");
            for (int index = 0; index < ReadFile.Length; index++)
            {
                string[] SplitFile = ReadFile[index].Split('/');
                if (SplitFile[2] != ID.ToUpper())
                {
                    valid = true;
                }
                else
                {
                    ID = IDGen();
                }
            }
            if (sw.ElapsedMilliseconds > 1500)
            {
                MessageBox.Show("Too many questions in file.", "Warning", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                SaveBtn.Text = "Save";
                return;
            }
        }
    }
}
```

Exception User-Unhandled

System.IO.FileNotFoundException: 'Could not find file 'C:\Users\hello\OneDrive - C2k\School\Computing\Yr 13\Project\Yr-13-NathanMcGee-CA\assets\txt\checkbox.txt.''

Ask Copilot | Show Call Stack | View Details | Copy Details | Start Live Share session

Exception Settings

- ☐ Break when this exception type is thrown
- ☒ Break when this exception type is user-unhandled

Except when thrown from:

- ☐ Yr-13-NathanMcGee-CA.exe

Open Exception Settings | Edit Conditions

Code After:

```
case "Checkbox":
{
    if (checkBox1.Checked || checkBox2.Checked || checkBox3.Checked || checkBox4.Checked)
    {
        cbcombined = cb1ans + cb2ans + cb3ans + cb4ans;
        sw.Start();
        while (!valid)
        {
            ID = IDGen();
            try
            {
                string[] ReadFile = File.ReadAllLines(Common_Variables.path + @"assets\txt\checkbox.txt");
                for (int index = 0; index < ReadFile.Length; index++)
                {
                    string[] SplitFile = ReadFile[index].Split('/');
                    if (SplitFile[2] != ID.ToUpper())
                    {
                        valid = true;
                    }
                    else
                    {
                        ID = IDGen();
                    }
                }
                if (sw.ElapsedMilliseconds > 1500)
                {
                    MessageBox.Show("Too many questions in file.", "Warning", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                    SaveBtn.Text = "Save";
                    return;
                }
            }
            catch
            {
                MessageBox.Show("checkbox.txt is either corrupted or missing, to prevent further data loss, the game will now exit.", "Critical Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                Environment.Exit(Environment.ExitCode);
            }
        }
    }
}
```

This happened for every question type, so to prevent this I put the code that saves the question to the question type text file inside a try catch statement that will show a messagebox to the user that explains why the program encountered an error, and that why the program will close.

TextViewer

Test 11

This test failed as the game crashed when the user tried to view a text file that does not exist, resulting in the program crashing and throwing a System.IO.FileNotFoundException exception, which is nonrecoverable and requires a full restart of the program.

Code Before:

```
else if (Common_Variables.FileToBeViewed == "checkbox.txt" || Common_Variables.FileToBeViewed == "multichoice.txt")
{
    StreamReader file = new StreamReader(Common_Variables.path + @"\assets\txt\" + Common_Variables.FileToBeViewed);
    string[] columnnames = { "Difficulty", "Answer", "Font Size", "ID", "Question", "Option 1", "Option 2", "Option 3" };

    foreach (string c in columnnames)
    {
        dt.Columns.Add(c);
    }

    string newline;

    while ((newline = file.ReadLine()) != null)
    {
        DataRow dr = dt.NewRow();
        string[] values = newline.Split('/');
        for (int index = 0; index < values.Length; index++)
        {
            dr[index] = values[index];
        }
        dt.Rows.Add(dr);
    }
    file.Close();
    dataGridView1.DataSource = dt;
}
```

Exception User-Unhandled

System.IO.FileNotFoundException: 'Could not find file 'C:\Users\boldiegoldie\OneDrive - C2k_School\Computing\Yr 13\Project\Yr-13-NathanMcGee-CA\assets\txt\checkbox.txt'.'

This exception was originally thrown at this call stack:

- [External Code]
- Yr_13_NathanMcGee_CA.TextViewer.LoadGrid() in [TextViewer.cs](#)
- Yr_13_NathanMcGee_CA.TextViewer.TextViewer_Load(object, System.EventArgs) in [TextViewer.cs](#)

View Details | Copy Details | Start Live Share session...

▲ Exception Settings

- ☒ Break when this exception type is user-unhandled
- Except when thrown from:
 - ☐ Yr-13-NathanMcGee-CA.exe

[Open Exception Settings](#) | [Edit Conditions](#)

Code After:

```
catch
{
    MessageBox.Show(Common_Variables.FileToBeViewed + " is either corrupted or missing, to prevent further data loss, the game will now exit.", "Critical Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    Environment.Exit(Environment.ExitCode);
}
```

This bug was simple to fix, by putting the loading code inside a try catch which will tell the user what text file is bugged and why the game will close after the message is acknowledged.

EndScreen

Test 2

This bug would write the updated LeaderboardData file to UserData's position, meaning that the Leaderboard would not have its file in the right place and then all user accounts would be deleted. If the user tried to sign into their account with the bugged UserData file it would crash the game as it would load LeaderboardData as UserData.

Code Before:

```
File.Delete(Common_Variables.LeaderboardData);

using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Create))
{
    LeaderboardData newdata = new LeaderboardData(Common_Variables.username, Common_Variables.LifetimeCorrect, Common_Variables.LifetimeIncorrect, Common_Variables.score);
    leaderlist.Add(newdata);
    serialise.Serialize(fileStream, leaderlist);
    leaderlist.Clear();
}
```

Code After:

```
File.Delete(Common_Variables.LeaderboardData);

using (Stream fileStream = File.Open(Common_Variables.LeaderboardData, FileMode.Create))
{
    LeaderboardData newdata = new LeaderboardData(Common_Variables.username, Common_Variables.score, Common_Variables.LifetimeCorrect, Common_Variables.LifetimeIncorrect);
    leaderlist.Add(newdata);
    serialise.Serialize(fileStream, leaderlist);
    leaderlist.Clear();
}
```

This bug was fixed by telling the LeaderboardData file to save to the correct location, instead of the UserData file. This fixes the bug as the UserData file is not overwritten, so users accounts are not wiped, and the Leaderboard has its file saved in the correct location.

Test 3 & 4

This test failed because the users account statistics were not updated, so they would be lost when the program is closed or if the user logs out of their account. This is because there was no code to update the statistics variables in EndScreen, only in HomeScreen, so statistics would only be saved if you skipped the EndScreen. Test 4 also failed because the game would crash if the UserData file was deleted, throwing a System.IO.FileNotFoundException exception.

Code Before:

```
else if (File.Exists(Common_Variables.LeaderboardData))
{
    using (Stream fileStream = File.Open(Common_Variables.LeaderboardData, FileMode.Open))
    {
        leaderlist = (List<LeaderboardData>)(deserialize.Deserialize(fileStream));
    }

    File.Delete(Common_Variables.LeaderboardData);

    using (Stream fileStream = File.Open(Common_Variables.LeaderboardData, FileMode.Create))
    {
        LeaderboardData newdata = new LeaderboardData(Common_Variables.username, Common_Variables.score, Common_Variables.LifetimeCorrect, Common_Variables.LifetimeIncorrect);
        leaderlist.Add(newdata);
        serialise.Serialize(fileStream, leaderlist);
        leaderlist.Clear();
    }
}

string[] LabelArray;
string[] ChosenLabel;

int tempScore = (int)(Common_Variables.score * 4);
```

Code After:

```
if (File.Exists(Common_Variables.UserData))
{
    using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Open))
    {
        userlist = (List<UserData>)(deserialize.Deserialize(fileStream));
    }

    File.Delete(Common_Variables.UserData);

    using (Stream fileStream = File.Open(Common_Variables.UserData, FileMode.Create))
    {
        foreach (UserData searchuser in userlist)
        {
            if (searchuser.User_id == Common_Variables.username)
            {
                searchuser.BlankLifeCorrect = Common_Variables.BlankLifeCorrect;
                searchuser.BlankLifeIncorrect = Common_Variables.BlankLifeIncorrect;
                searchuser.CheckLifeCorrect = Common_Variables.CheckLifeCorrect;
                searchuser.CheckLifeIncorrect = Common_Variables.CheckLifeIncorrect;
                searchuser.DropLifeCorrect = Common_Variables.DropLifeCorrect;
                searchuser.DropLifeIncorrect = Common_Variables.DropLifeIncorrect;
                searchuser.MultilifeCorrect = Common_Variables.MultilifeCorrect;
                searchuser.MultilifeIncorrect = Common_Variables.MultilifeIncorrect;
                searchuser.TorflifeCorrect = Common_Variables.TorflifeCorrect;
                searchuser.TorflifeIncorrect = Common_Variables.TorflifeIncorrect;
                searchuser.LifetimeCorrect = Common_Variables.LifetimeCorrect;
                searchuser.LifetimeIncorrect = Common_Variables.LifetimeIncorrect;
            }
        }
        serialise.Serialize(fileStream, userlist); // Serialise data using a list of user objects
        userlist.Clear();
    }
}
else
{
    MessageBox.Show("UserData file has been deleted during gameplay, to prevent further data loss, the game will now exit.", "Critical Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    Environment.Exit(Environment.ExitCode);
}
```

This bug was fixed by copying the code that is used to save the account statistics when the EndScreen is skipped in HomeScreen into EndScreen. This means that if the users log out their account or closes the program after completing the quiz, the updated account statistics will be saved.

I also added an error message that will show up if there is any error with the UserData file that explains the problem to the user and why the game will now exit.

Test 11

This test failed because if any of the end message files were missing it would throw a System.IO.FileNotFoundException exception, instead of showing the user an error message that explains the problem and returning them to the lobby.

Code Before:

```

if (Common_Variables.score <= 3)
{
    LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\VeryBadEnd.txt");
    int rand = random.Next(0, LabelArray.Count());
    ChosenLabel = LabelArray[rand].Split('/');
    EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
    EndMessage.Text = ChosenLabel[1];
    EndScoreTxt.ForeColor = Color.Red;
    if (Common_Variables.score == 0)
    {
        EndScoreTxt.ForeColor = Color.DarkRed;
    }
    EndScoreTxt.Text = Common_Variables.score.ToString();
}
else if (Common_Variables.score > 3 && Common_Variables.score <= 8)
{
    LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\BadEnd.txt");
    int rand = random.Next(0, LabelArray.Count());
    ChosenLabel = LabelArray[rand].Split('/');
    EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
    EndMessage.Text = ChosenLabel[1];
    EndScoreTxt.ForeColor = Color.OrangeRed;
    EndScoreTxt.Text = Common_Variables.score.ToString();
}
else if (Common_Variables.score > 8 && Common_Variables.score <= 12)
{
    LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\NormalEnd.txt");
    int rand = random.Next(0, LabelArray.Count());
    ChosenLabel = LabelArray[rand].Split('/');
    EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
    EndMessage.Text = ChosenLabel[1];
    Bronze.Image = Resources.CopperTrophy;
    EndScoreTxt.ForeColor = Color.Orange;
    EndScoreTxt.Text = Common_Variables.score.ToString();
}
else if (Common_Variables.score > 12 && Common_Variables.score <= 16)
{
    LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\GoodEnd.txt");
    int rand = random.Next(0, LabelArray.Count());
    ChosenLabel = LabelArray[rand].Split('/');
    EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
    EndMessage.Text = ChosenLabel[1];
    Bronze.Image = Resources.CopperTrophy;
    Silver.Image = Resources.SilverTrophy;
    EndScoreTxt.ForeColor = Color.Lime;
    EndScoreTxt.Text = Common_Variables.score.ToString();
}
else if (Common_Variables.score > 16)
{
    LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\VeryGoodEnd.txt");
    int rand = random.Next(0, LabelArray.Count());
    ChosenLabel = LabelArray[rand].Split('/');
    EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
    EndMessage.Text = ChosenLabel[1];
    Bronze.Image = Resources.CopperTrophy;
    Silver.Image = Resources.SilverTrophy;
    Gold.Image = Resources.GoldTrophy;
    EndScoreTxt.ForeColor = Color.LimeGreen;
    EndScoreTxt.Text = Common_Variables.score.ToString();
}
}

```

Code After:

```

try
{
    if (Common_Variables.score <= 3)
    {
        LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\VeryBadEnd.txt");
        int rand = random.Next(0, LabelArray.Count());
        ChosenLabel = LabelArray[rand].Split('/');
        EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
        EndMessage.Text = ChosenLabel[1];
        EndScoreTxt.ForeColor = Color.Red;
        if (Common_Variables.score == 0)
        {
            EndScoreTxt.ForeColor = Color.DarkRed;
        }
        EndScoreTxt.Text = Common_Variables.score.ToString();
    }
    else if (Common_Variables.score > 3 && Common_Variables.score <= 8)
    {
        LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\BadEnd.txt");
        int rand = random.Next(0, LabelArray.Count());
        ChosenLabel = LabelArray[rand].Split('/');
        EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
        EndMessage.Text = ChosenLabel[1];
        EndScoreTxt.ForeColor = Color.OrangeRed;
        EndScoreTxt.Text = Common_Variables.score.ToString();
    }
    else if (Common_Variables.score > 8 && Common_Variables.score <= 12)
    {
        LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\NormalEnd.txt");
        int rand = random.Next(0, LabelArray.Count());
        ChosenLabel = LabelArray[rand].Split('/');
        EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
        EndMessage.Text = ChosenLabel[1];
        Bronze.Image = Resources.CopperTrophy;
        EndScoreTxt.ForeColor = Color.Orange;
        EndScoreTxt.Text = Common_Variables.score.ToString();
    }
    else if (Common_Variables.score > 12 && Common_Variables.score <= 16)
    {
        LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\GoodEnd.txt");
        int rand = random.Next(0, LabelArray.Count());
        ChosenLabel = LabelArray[rand].Split('/');
        EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
        EndMessage.Text = ChosenLabel[1];
        Bronze.Image = Resources.CopperTrophy;
        Silver.Image = Resources.SilverTrophy;
        EndScoreTxt.ForeColor = Color.Lime;
        EndScoreTxt.Text = Common_Variables.score.ToString();
    }
    else if (Common_Variables.score > 16)
    {
        LabelArray = File.ReadAllLines(Common_Variables.path + @"\assets\txt\EndScreen\VeryGoodEnd.txt");
        int rand = random.Next(0, LabelArray.Count());
        ChosenLabel = LabelArray[rand].Split('/');
        EndMessage.Font = new Font("Lucida Sans", (float)Convert.ToDecimal(ChosenLabel[0]), FontStyle.Bold);
        EndMessage.Text = ChosenLabel[1];
        Bronze.Image = Resources.CopperTrophy;
        Silver.Image = Resources.SilverTrophy;
        Gold.Image = Resources.GoldTrophy;
        EndScoreTxt.ForeColor = Color.LimeGreen;
        EndScoreTxt.Text = Common_Variables.score.ToString();
    }
}
catch
{
    MessageBox.Show("Message text file is corrupted or does not exist, returning to lobby...", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    Common_Variables.score = 0;
    Common_Variables.q_number = 1;

    if (ShowLobby != null)
    {
        ShowLobby.Invoke(this, EventArgs.Empty);
    }
    Common_Variables.ExitEnd = true;
}
end.Show();
if (Common_Variables.ExitEnd)
{
    end.Hide();
}
}

```

Candidate Number: 2217

Centre Number: 71955

This fixed the bug as if the end message files are corrupted it will stop the program from crashing and take the user back to the Lobby, resetting the quiz variables so that the user can play the quiz again, since this is not a game breaking error. It will also show an error message to the user to explain why they are being taken back to the lobby instead of the EndScreen.

Test 12 & 13

This test failed as clicking both ReturnLobby or GoToLeaderboard would take you to either the Lobby or Leaderboard respectively, but not reset the quiz variables, so the quiz would instantly end, and the user would get the same score they did the previous playthrough of the quiz which would lead to duplicate leaderboard entries.

Code Before:

```
private void GoToLeaderboard_Click(object sender, EventArgs e)
{
    if (Common_Variables.muted == false)
    {
        player.Stream = Properties.Resources.click;
        player.Play();
    }

    if (ShowLeaderboard != null)
    {
        ShowLeaderboard.Invoke(this, EventArgs.Empty);
    }
    this.Close();
}

private void ReturnLobby_Click(object sender, EventArgs e)
{
    if (Common_Variables.muted == false)
    {
        player.Stream = Properties.Resources.click;
        player.Play();
    }

    if (ShowLobby != null)
    {
        ShowLobby.Invoke(this, EventArgs.Empty);
    }
    this.Close();
}
```

Code After:

```
private void GoToLeaderboard_Click(object sender, EventArgs e)
{
    if (Common_Variables.muted == false)
    {
        player.Stream = Properties.Resources.click;
        player.Play();
    }

    Common_Variables.score = 0;
    Common_Variables.q_number = 1;

    if (ShowLeaderboard != null)
    {
        ShowLeaderboard.Invoke(this, EventArgs.Empty);
    }
    this.Close();
}

private void ReturnLobby_Click(object sender, EventArgs e)
{
    if (Common_Variables.muted == false)
    {
        player.Stream = Properties.Resources.click;
        player.Play();
    }

    Common_Variables.score = 0;
    Common_Variables.q_number = 1;

    if (ShowLobby != null)
    {
        ShowLobby.Invoke(this, EventArgs.Empty);
    }
    this.Close();
}
```

This bug was fixed by setting the 2 quiz variables (Common_Variables.q_number & Common_Variables.score) to their default values so that if the user tries to play the quiz immediately after finishing the last play through they will actually be able to play the quiz.

TrueOrFalse

Test 26

This test failed because the random number generated in event_selected can be larger than the list of possible events for this question section, which will cause an index out of range exception, requiring a restart of the program to recover from.

Code Before:

```
private void SpecialEvent()
{
    int event_probability = random.Next(1, 21);
    int event_selected = random.Next(0, 6);
    switch (Common_Variables.difficulty)
    {
        case "n":
            if (event_probability == 1 || event_probability == 2)
            {
                switch (SpecialEventList[event_selected])
                {
                    case "SmallText":
                        if (Common_Variables.dyslexia)
                        {
                            QuestionHeader.Font = new Font(
                                AnswerInput.Font = new Font("C
                                break;
                        }
                    }
                }
            }
        }
    }
```

Exception User-Unhandled
System.IndexOutOfRangeException: 'Index was outside the bounds of the array.'
View Details | Copy Details | Start Live Share session...
▸ Exception Settings

Code After:

```
int event_probability = random.Next(1, 21);
int event_selected = random.Next(0, 4);
```

To fix this bug I lowered the highest number that could be generated from 5 to 3, as there are only 4 possible events that can be triggered in this section of the quiz, which eliminates the possibility of the quiz crashing due to event_probability falling out of range of the index of SpecialEventList

FillInTheBlank

Test 24

This test failed because the random number generated in event_selected can be larger than the list of possible events for this question section, which will cause an index out of range exception, requiring a restart of the program to recover from.

Code Before:

```
private void SpecialEvent()
{
    int event_probability = random.Next(1, 21);
    int event_selected = random.Next(0, 6);
    switch (Common_Variables.difficulty)
    {
        case "n":
            if (event_probability == 1 || event_probability == 2)
            {
                switch (SpecialEventList[event_selected])
                {
                    case "SmallText":
                        if (Common_Variables.dyslexia)
                        {
                            QuestionHeader.Font = new Font(
                                AnswerInput.Font = new Font("C
                                break;
                        }
                    }
                }
            }
        }
    }
```

Exception User-Unhandled
System.IndexOutOfRangeException: 'Index was outside the bounds of the array.'
View Details | Copy Details | Start Live Share session...
▸ Exception Settings

Code After:

```
int event_probability = random.Next(1, 21);
int event_selected = random.Next(0, 4);
```

To fix this bug I lowered the highest number that could be generated from 5 to 3, as there are only 4 possible events that can be triggered in this section of the quiz, which eliminates the possibility of the quiz crashing due to event_probability falling out of range of the index of SpecialEventList.

Testing Conclusion

Overall, I feel my testing was very rigorous in testing various different scenarios which could occur in a normal playthrough of my quiz, and some scenarios which are unlikely to happen but would have proved detrimental to my program if they had. There will be areas that I have missed during this round of testing, but I feel like the tests I have done are more than enough to cover the bugs a user would encounter in a typical playthrough of my quiz, which ensures that my program is robust and functioning as intended.

There were areas of my quiz which failed the tests, particularly in my EndScreen, but I have fixed these bugs as shown in the section above, from this I can conclude that my program.

Evaluation

Peer Review Questions

1. How well was this requirement met?

NFR1: The quiz must be fun and engaging to play.

2. How well was this requirement met?

NFR2: The quiz's UI must be easy to understand and simple to use.

3. How well was this requirement met?

NFR3: The rules of the quiz must be simple and not overly complex.

4. How well was this requirement met?

NFR4: The forms for the game must have a common look and feel.

5. How well was this requirement met?

NFR5: The Quiz must be robust.

6. How well was this requirement met?

NFR6: The main section of the game must have accessibility features to allow for more people to play the quiz.

7. If you could add or change something about this quiz, what would you do?

Evaluation of Functional General Requirements

FGR1: Use forms that are loaded onto a panel instead of pop-up windows to make navigation throughout the program smooth, with the option to go back and forth between panels.

I have fully implemented this requirement, as no forms are shown on pop-up windows in my program. All forms are shown on the HomePanel, and the user is able to move back to forms that they have previously been on clicking the BackBtn. There are buttons on some most forms that take the users to different forms on the same panel, such as moving from QuestionDesigner to TextViewer. The transition between forms is smooth and transparent to the user. Therefore, my implementation of forms and the movement between them fully meets the requirement set out in FGR1.

FGR2: The program will open with a welcome screen, that allows you to either go to the sign in screen or the create account screen.

I have fully implemented this requirement as, when the program is launched it will open with the WelcomeScreen form which has 2 buttons on it, CreateAccountBtn and LoginBtn, which will take the users to the CreateAccount form or Login form, depending on which button is clicked. Therefore, my implementation of forms and the movement between them fully meets the requirement set out in FGR2.

FGR3a: The accounts must have usernames no longer than 12 characters and no shorter than 4 characters.

I have met this requirement as the textbox used to input your desired account name is set to not accept any more characters after the 12th character, and account creation will not proceed if the username is shorter than 4 characters, showing an error message to the user telling them that their username is too short, fulfilling the requirement set out in FGR3a.

FGR3b: The accounts must be stored in a secure way.

I have met this requirement as all account information is not stored as plain text in a text file, rather it is serialised into a dat file which will not return readable data if opened in a text editor. It is also impossible to read a user's password unless you are debugging the program in Visual Studio, which the end user will not be able to do. This means that my user accounts are stored in a secure way, fulfilling the requirement set out in FGR3b.

FGR3c: The accounts must not contain duplicate usernames.

I have met this requirement as before any account is created the UserData dat file is deserialised and scanned to see if any accounts already exist with that username and the username will be rejected if any accounts already exist with that username with an error label appearing to tell the user that the username is already taken, fulfilling this requirement.

FGR3d: The accounts must have strong passwords that are not easy to guess (mixed case, has at least one symbol & number and longer than 3 characters).

I have met this requirement as any password that does not meet these requirements will be rejected and not allow account creation to proceed. The user will be told why their password was rejected with an error label. This ensures that only secure passwords that are difficult to guess or brute force can be used for accounts, fulfilling this requirement.

FGR3e: The accounts must have usernames that are made up of only alphanumeric characters.

I have met this requirement as the username given by the user during account creation is checked to see if it contains only alphanumeric character, after any whitespace is removed from the username. If it does not account creation does not proceed and the user is told that their username cannot contain symbols by an error label, fulfilling this requirement.

FGR3f: The accounts must store statistics about the user's previous games, such as Lifetime correct & Lifetime incorrect answers.

I have met this requirement as each user's accounts has variables that hold the user's lifetime correct answers, lifetime incorrect answers and lifetime correct / incorrect answers for each of the 5 sections of the quiz. These variables are updated with the newest information when the user reaches the end of the quiz, and the user can see these statistics on the Lobby, fulfilling this requirement.

FGR3g: The accounts must store the settings that the user has enabled.

I have met this requirement as each user's account has 4 variables that store the state of the 4 settings variables when the user last applied settings on the Settings page. These variables are loaded in when the user signs into their account, so the user does not need to reapply their settings each time they log into their account, fulfilling this requirement.

FGR4a: The account creation screen must provide meaningful feedback to users as to why their username/password did not pass validation.

I have met this requirement, as each reason for a user's desired username or password rejection has its own error label to tell the user why their username / password is unsuitable, such as the text in ConfirmPassword & PasswordInput not matching showing a label that says "Passwords do not match." or the username containing non alphanumeric characters showing a label that says "Username cannot contain a symbol.". Each of the error labels will also turn its associated header red when it is shown to make it clear to the user which textbox caused the error, fulfilling this requirement.

FGR4b: The account creation screen must allow users to unhide their password to double check their passwords spelling.

I have met this requirement because any text typed into either PasswordInput or ConfirmPassword are obfuscated by default because both of these textboxes have the PasswordChar property enabled and set to an Asterix (*). If the user wants to unhide the text on these textboxes they can click the PassShow button, which will set the PasswordChar property for both of the textboxes to \0 which removes the effect of the PasswordChar property. The image on PassShow button changes to reflect that the password is unhidden and clicking the button again will rehide the passwords but setting the PasswordChar properties for both of the buttons back to the Asterix and change the image on PassShow to reflect this, fulfilling the requirement.

FGR4c: The account creation screen must make users type in their password again to confirm their password.

I have met this requirement for account creation to take place, the user must type their desired password into the ConfirmPassword textbox. If this password does not match the password typed into PasswordInput exactly, account creation will not proceed and the user will be told by an error label that the password typed into PasswordInput and the one typed into ConfirmPassword do not match, fulfilling the requirement.

FGR5: The user must be able to sign into their account, if they have one registered.

I believe that I have fully met this requirement due to the Login form, as the user is able to input their accounts username and password into textboxes that are labelled so the user knows where to input their credentials. If these credentials are correct for one of the stored accounts, the settings and account statistics stored in that account are loaded into the programs Common_Variables, which means that the account is signed into. Users cannot sign into accounts that they do not know the credentials for, preventing them from signing into accounts they do not own. This means that the user can only sign into accounts that they own, fulfilling the FGR5 requirement.

FGR6: The login screen must not specify which inputted credential was wrong, only that one or both were wrong, to mitigate brute force account cracking attempts.

I have met this requirement as there is only one error label on the Login form that triggers if either the username is not found on any of the accounts, or if the password is wrong for the account. It will turn both of the header red so that the user cannot tell if the username or password was incorrect, or if both of them were incorrect, fulfilling the requirement.

FGR7: The Login screen must allow users to unhide their password.

I have met this requirement because any text typed into PasswordInput is obfuscated by default because this textbox has the PasswordChar property enabled and set to an Asterix (*). If the user wants to unhide the text on this textbox they can click the PassShow button, which will set the PasswordChar property for both of the textboxes to \0 which removes the effect of the PasswordChar property. The image on PassShow button changes to reflect that the password is unhidden and clicking the button again will rehide the passwords but setting the PasswordChar properties for both of the buttons back to the Asterix and change the image on PassShow to reflect this, fulfilling the requirement.

FGR8: Have a lobby where the user can choose to go to any of the main sections and which also displays statistics about their account.

I have met this requirement as when the user signs into their account or makes a new account, they are taken to the Lobby form, which will show the users account statistics and has 4 buttons to take the user to the other features of the program (Difficulty selection, Leaderboard, Settings form & the Question Designer) as well as a button to start the quiz. These buttons and the 2 labels that show the users account statistics fulfil this requirement.

FGR9: The user must be able to add their own questions to the quiz to add replay value to the quiz.

I have met this requirement due to the QuestionDesigner form. This form allows the user to add questions to 4 of the question types in the quiz, Multiple Choice, Checkbox, True or False and Fill in the Blank. This form allows the user to preview said questions so that they can set the font size for the QuestionHeader correctly and see how the question will look like in-game. This form also prevents users from adding duplicate questions in the same question type, ensuring that each question in a question type is unique to prevent questions from becoming stale. This form also allows the user to mark their question as either an easy question or hard question, for quiz difficulty purposes.

FGR10: The user must be able to change the difficulty of the quiz.

I have met this requirement with the DifficultySelect form. This form allows the user to change the state of the difficulty variable. This variable affects how many points the user gets for a correct answer, the possibility of a special event appearing and what questions are given to the user to answer. The current difficulty is shown by giving the button that represents the current difficulty a border, with said border being either green, orange, or red, depending on the difficulty the button represents. The user is able to change the difficulty of the quiz by clicking one of these buttons, fulfilling this requirement.

FGR11: The user must be able to access the Leaderboard from the lobby and at the end of the game.

This requirement is met by both the Lobby and EndScreen forms, both of these forms have buttons that, when clicked, will take the user to the Leaderboard form.

FGR12: The leaderboard must be sortable by each category stored in it, either by ascending order or descending order.

This requirement is met as the Leaderboard form has 4 buttons that sort the leaderboard by the 4 values that are stored in LeaderboardData, Username, Score, LifetimeCorrect & LifetimeIncorrect, when clicked. There is a 5th button that allows the user to decide if the leaderboard is ordered in ascending order or descending order, the user changes what order the leaderboard is in by clicking the button, its text changing to tell the user the current order of the leaderboard.

FGR13: The user must be able to change the volume of the quiz or mute it.

I have met this requirement as the user is able to change the volume of the quiz in the Settings form by using the trackbar and applying the settings, which will change the user's system volume to the value set on the trackbar using the user32.dll system file. The user is also able to mute the quiz at any time using the MuteToggle button. Clicking this button while the game is unmuted will set the muted variable to true, and every sound effect in the game first checks the current state of this variable and will not play if it is set to true, fulfilling this requirement.

FGR14: The quiz will end with an optional end screen, which will have an option to take them to the Leaderboard or return to the lobby.

I have met this requirement as the user can choose to skip the EndScreen by enabling the Skip End Screen option in Settings. If that option is enabled the quiz will take the user back to the Lobby when the quiz ends, instead of taking them to EndScreen. If the user has not enabled this option, the user will be taken to the EndScreen which has 2 buttons on it, one that will take the user to the Leaderboard when clicked and another that takes the user to Lobby when clicked, fulfilling this requirement.

FGR15: The user must be able to sign out of their account and log in with a different account without closing the program.

I have met this requirement as the user can press the BackBtn on Lobby to go back to the WelcomeScreen, which will allow the user to either log in to another account or to make a new account.

FGR16: There must be an admin mode which allows the user to delete the file that contains the accounts and the file that contains the leaderboard for maintenance reasons, it will also show the user the answer to the current question if they are playing the quiz and allow the user to make custom end messages.

I have met this requirement as the user is able to enable admin mode in the Settings form, which will unhide the admin settings. These admin settings allow the user to delete the UserData or LeaderboardData files for maintenance purposes. The user will also be able to add custom end messages for each of the 5 end message categories. The user will not be able to save a duplicate of a message that already exists in that category, but they will be able to preview what their message will look like so that they can set the correct font size for their message. While admin mode is enabled, the user will also be told the answer for the question they are answering, fulfilling the requirement.

FGR17: There must be a help centre to explain the features of the quiz.

I have met this requirement with my Tutorial form. This form can be accessed by clicking the TutorialBtn and it will open the Tutorial form on the page of the tutorial for the section of the program that the user was just on, for example clicking the TutorialBtn while making a Multiple Choice question will take the user to the section of the tutorial that explains how to make a Multiple Choice question. The user can navigate through the different pages of the tutorial using the GoForward and GoBack buttons. Each page has an image of the screen being explained and an explanation on how to use that section of the program, fulfilling this requirement.

FGR18: The quiz must call the user by their name instead of using generic terms.

I have met this requirement as the welcome text in Lobby uses the current user's username instead of a generic term like "player" or "user".

FGR19: The quiz must give users meaningful error messages if an unrecoverable error occurs or in an error needs user intervention.

I have met this requirement as the sections of my quiz where the user is likely to encounter some errors, such as creating or logging into an account or the Question Designer, all have error labels that tell the user what the error was, for example an error label appearing if the users password does not have a symbol in CreateAccount or if the question that the user is trying to add has already been used in QuestionDesigner.

For more serious warnings, like when a user tries to sign into an account when a UserData file does not exist or if the maximum number of IDs has been used in a question file, a message box appears that explains the problem to the user, and in the case of the Login error, explains what the user needs to do to fix the problem.

For catastrophic errors, such as a critical file (UserData or a question file) being deleted or corrupted during gameplay the quiz will show a message box to the user explaining that a critical file is corrupted or missing and that the quiz will exit to avoid further data loss.

These message boxes all have icons that show their severity. Errors that the user can fix, like trying to sign into an account when there is no UserData file, use the blue i symbol to indicate to the user that the message box is giving them information on how to fix the error. More serious errors that will not crash the game, such as the end message text files being missing or corrupted, will have a yellow warning symbol to indicate to the user that the error is severe, but not game breaking. Errors that prevent the game from continuing will use the red error symbol to indicate to the user that a catastrophic error has taken place and the game cannot continue.

FGR20: There should be a settings screen where the user can change settings, like whether or not to skip the end screen.

I have met this requirement with my Settings form. The user can change 4 settings in that form, such as whether or not admin mode is enabled, if the EndScreen is skipped, if questions will be read out and if dyslexia mode will be enabled. These settings all change the state of variables that the rest of the program use to determine if the settings are enabled or not, fulfilling this requirement.

FGR21: The user should be able to view editable text files in the program and delete lines from them.

I have met this requirement with the TextViewer form. The user accesses this form from clicking the button on QuestionDesigner or Settings to view existing questions / messages. It will load the contents of these text files onto a dataGridView so the user can see the questions/messages that already exist in that category. There is a button that also allows the user to delete the selected line from the text file, in case the user made a mistake in a question / message or if they just want to delete the question / message. This system fulfils this requirement.

FGR22: The users name, score, lifetimecorrect answers and lifetime incorrect answers will be saved to the leaderboard.

I have met this requirement as when the user finishes the quiz, their current score along with their current lifetime correct and lifetime incorrect answers will be saved to the leaderboard with their current username and can be viewed on the leaderboard.

Evaluation of Functional In-Game Requirements

FIR1: The quiz will play a sound to indicate as to whether or not the user answered the question correctly.

I have met this requirement, as if the user gets a question correct, the game will play click.wav to tell the user that they got the question right and it would play wrong.wav if they got the question wrong. These sound effects will not play if the game is muted.

FIR2: There must be special events that can happen during the quiz that will make the question harder to answer.

This requirement has been met as every time a new question is pulled from the sections question text file it will generate a random number between 1 & 20 and if the quiz is in normal or hard mode this number will determine if a special event is triggered (special events do not appear on easy mode). If the quiz is in normal mode, an event will be triggered if the number is either 1 or 2, which means that an event is triggered roughly 10% of the time, if the quiz is in hard mode an event will be triggered if the number is either 1, 2 or 3, which is roughly 15% of the time.

If an event is triggered it will then pick a number between 0 and the total number of events for that section, as not all sections have all events as it would make questions either too difficult or impossible to answer, such as FillInTheBlank leaving out the Wingdings event. The full list of possible events is LargeText, SmallText, TimedQuestion, ObstructedAnswer and Wingdings. These events make the quiz more challenging as the questions are harder to answer, fulfilling this requirement.

FIR3: The quiz will contain 20 questions.

I have met this requirement as when the quiz moves to a new section it checks to see if the current question number is over 20, and if it is it will end the quiz and send the user to either the EndScreen or Lobby, depending on if they have disabled the EndScreen. It is impossible for this to trigger before 20 questions have been answered, so the quiz will always be 20 questions, fulfilling this requirement.

FIR4: The sections of the quiz must be randomised, and no sections should show up twice in one playthrough, to make the quiz more interesting and add replay value.

This requirement has been met as whenever the quiz is picking the next question section to show, it will generate a random number between 0 and the current length of the list that contains the unused sections of the quiz. It will then load the section of the quiz that is stored in the selected position of the list and remove that value from the list so that the section cannot show up again in that playthrough. Since the quiz is 20 questions

long and a section contains 5 questions, it is not guaranteed that every section of the quiz will show up in a playthrough, which adds to the randomness of the quiz.

FIR5: The questions for the quiz must be randomised and never show up twice in one playthrough to make the quiz more interesting and replayable.

This requirement is met as when a question form pulls a new question it will read a random line from its question text file and then check its unique identifier against the list of used IDs, and if the pulled questions ID is on the list of used IDs, a new question will be grabbed from the text file and the ID will be checked again.

If the IDs not on the list the questions data (Header text & font size, question options etc) will be loaded in and the questions ID will be added to the list of used IDs, so that the question will not be generated again for the remainder of that section of the quiz. This ensures that the questions for the entire quiz will be randomised and will be unable to show up twice in one playthrough, fulfilling this requirement.

FIR6: The user must be able to see their current score, question number and section of the quiz.

This requirement is met by the QuestionUI label on HomeScreen. It displays the users current score, the current question number and the current question section. This label is updated with the newest information every time a question is answered, so it is always up to date.

FIR7: The user should hit a button to check their answer, instead of it being checked as soon as the click on an option.

This requirement is met with the AnsCheck button. The answer checking code for any of the questions forms can only be triggered by clicking the AnsCheck button, ensuring that the user can click/type and answer or combination of answers without having to worry about the answer being checked until they are sure of their answer.

FIR8: Points gained should be proportionate to the user's difficulty.

This requirement has been met as the answer checking code for all of the question forms have 3 possible point rewards for a correct answer. If the quiz is in easy mode any correct answer will award 0.5 points, in normal mode it will award 0.75 points for a correct answer and hard mode will award 1 point for a correct answer.

This means that on easy mode the maximum score that can be achieved is 10/20, on normal mode it is 15/20 and only on hard mode can a user achieve a full 20/20 score, fulfilling this requirement.

FIR9: The user must supply an answer to the question.

This requirement is met by the NoAns labels in each question form. If the user does not input an answer for the question, or tries to answer with the default textbox text for FillInTheBlank, the answer checking code will catch that no answer has been provided and show the NoAns label before exiting the answer checking code early, so that the user can provide an answer to the question.

Evaluation of Non-Functional Requirements

NFR1: The quiz must be fun and engaging to play.

I believe that I have met this requirement as the quiz has multiple different sections that do not last for more than 5 questions, helping to prevent a question type from becoming stale. The user is able to change the difficulty of the quiz so that it still poses a challenge to them, without being so difficult that it is unenjoyable. The special events also help to make the quiz more fun and engaging as it changes how the question is answered, as a user might have to try to guess what the rest of the option says if it is obfuscated or try to guess what the question is if the QuestionHeader font is set to wingdings.

Peer Review 1: “The quiz has five different styles of questions and the ability to change the difficulty, this allows for a variety of questions as well as being able to change how hard the questions are to suit your level of knowledge on computing. This allows the quiz to engage the player while not being overly difficult or too easy. In my opinion this requirement is well met to a high standard.”

Peer Review 2: “The game was engaging although it could use a larger variety of question types.”

NFR2: The quiz’s UI must be easy to understand and simple to use.

I believe that I have met this requirement as each button in the program has text on it that tells the user what the button does, or a symbol that tells the user what the button does, such as the MuteToggle or BackBtn. There is a header on the top of every form that tells the user what form they are on, and the window text is also changed to tell the user the form they are currently on, minimising chances of the user getting confused. However, the dataGridViews need to be highlighted when they are first loaded to see their contents due to a bug in WinForms, this can be confusing for the user and might make them think there is a problem with the program, which hampers my fulfilment of this requirement somewhat.

Peer Review 1: “The quiz UI is simple to understand. All the buttons are either labelled with their function or have an icon which infers the button's function using standard and recognisable symbols. It is difficult to accidentally do something damaging as the buttons to delete or add data ask for confirmation beforehand. In my opinion this requirement is well met, however it would've been better if the dataGridView tables showing the existing questions and end screen messages didn't need to be selected before being able to see its contents.”

Peer Review 2: “The UI was mostly easy to navigate but checking your answer is confusing as a new user.”

NFR3: The rules of the quiz must be simple and not overly complex.

I believe that I have met this requirement, as there are no overly complex rules to my quiz. The different types of questions are very standard for quiz games and are self-explanatory. If the user is confused by the rules, there is an explanation on how to answer each question type in the Tutorial form, ensuring that everyone can understand the rules for the game.

Peer Review 1: “The five question styles are simple to understand, they are conventional question types such as fill in the blank, multiple choice and drag and drop. I think this requirement is met to a high standard.”

Peer Review 2: “The rules for the quiz were easy to understand and are readily available through a button at the bottom of the screen.”

NFR4: The forms for the game must have a common look and feel.

I believe that I have met this requirement as many elements of the UI are shared between all forms. They all have a background colour of #522081 and they all use #360f5a as secondary background colours for elements like buttons. All titles for the forms are located at 20, 30 and all buttons look the same, unless necessary. All of my forms use the Lucida Sans font in bold, unless the dyslexia mode is enabled, in which case the question forms will all be in Comic Sans MS instead. The visual elements of the forms are all aligned to either the left or right, with no elements in the dead centre of the screen.

Peer Review 1: “Each form has the same look and feel, with a common purple background and white text. The fonts and colours used are uniform across all the forms. I think this requirement is met to a high standard.”

Peer Review 2: “The form designs were consistent and aesthetically pleasing throughout the quiz.”

NFR5: The Quiz must be robust.

I believe that I have met this requirement, as the parts of my code that would be most likely to cause an unexplained error due to their reliance of external files are contained within try catch statements that will prevent the game from crashing, for example, the game will not crash if the end message text files are corrupted and there is an attempt to load them in. If the problem is unrecoverable, like the UserData file being deleted during gameplay, it will not prevent the crash, but it will make the program fail gracefully, explaining the problem to the user and closing the problem. The testing part of my program also helped to remove bugs in my code, however I do believe that there may be some areas in the application where errors or mistakes may occur due to these areas not being robust enough.

Peer Review 1: “The quiz has no obvious bugs. If an error occurs such as all the questions being removed the game gracefully exits itself, preventing a crash. I think this is met to a high standard however it could've been improved by restoring the missing data upon an error involving missing data instead of requiring a reinstall.”

Peer Review 2: “While playing the quiz I experienced no major bugs/errors.”

NFR6: The main section of the game must have accessibility features to allow for more people to play the quiz.

I believe that I have met this requirement decently as my quiz has 3 features that help make the quiz more accessible. There is the option to have the program readout the stock questions when they appear, which will help people who have vision problems play my quiz, as they can listen to the questions instead of having to read them. If a user has dyslexia, they can enable the dyslexia mode which will change the font for the quiz change to *Comic Sans MS* which is easier to read for people with dyslexia, compared to other fonts. If a user has sensitive hearing or does not like the sound effects in the game for what ever reason, they can mute the game at any point which will stop all sound effects from playing, making the quiz more accessible to them. However, there were many steps I could have taken to improve the accessibility of my quiz, such as having the options be read out aswell as the question and having the *Comic Sans MS* font be applied globally but I could not do this due to time constraints, which is a significant caveat to this requirement.

Peer Review 1: “The game includes two accessibility features, a screen reader to help with users who have difficulty reading the text on screen and a font change to help with dyslexia. These features provide increased accessibility to allow more people to play the game. This requirement was met to a high standard, allowing a broader audience to access the game.”

Peer Review 2: “The “dyslexia” mode made it much easier to read the question text during the quiz.”

Conclusion

Evaluation of my final product

Strengths

I believe that I have made a good program due to the fact that I have met or partially met all of the user requirements that I set out initially in order to make as good a game as possible.

The user feedback form told me that another one of the strengths of my program is that none of the play testers encountered any bugs throughout their playthrough. Users also commented that my UI was easy to understand as buttons were clearly labelled or used symbols to show their functionality. User 1 commented that forcing the user to click come buttons twice, such as DeleteUsrDat helped to ensure that the user did not accidentally delete something they did not mean to.

User 1 also commented that there was a variety in the question types and in the questions, they said that this allowed them to engage with the quiz more as they could change the difficulty of the quiz to meet their level of knowledge.

Users also commented that the rules of my final quiz were simple to understand, and they also mentioned that the Tutorial was a good source of information if there was any confusion over the rules.

Users also said that the accessibility features in my quiz allowed more people to play the quiz, with User 2 specifically saying that the dyslexia mode helped them to play the quiz.

One of the strengths of my program is that I followed a consistent naming scheme for the UI assets and functions of my quiz and used sensible variable names. This, alongside commenting my code, will help with future maintenance as the code for the quiz is very readable and explained.

Another strength of my quiz is that I organised all of the assets my quiz uses into a separate assets folder, which has the assets organised by file type, with relative file path used to make sure the program can access the assets folder. This will also help with future maintenance as users can easily find where every asset is stored and can even add their own, in the case of the wav files used to read out the questions.

Another strength of my program is that the font sizes for the QuestionHeaders and EndMessage are set dynamically. Each question and end message has an attached font size, this means that no question will run off of the label or look too big/too small (when the LargeText or SmallText events aren't triggered).

Another strength of my program is the QuestionDesigner, the QuestionDesigner allows the user to add custom questions to 4 of the quiz's 5 sections. The QuestionDesigner was carefully designed to prevent the user from accidentally breaking the quiz, as such any / are removed from any of the inputs at that character is used to split the line of text into its different values. The user also can't save a question if it does not have an answer, doesn't have a full list of options or doesn't have a difficulty. This allows users to make

the quiz's questions more up to date in the future or to add more difficult / easy questions, if the user thinks the quiz's difficulty isn't right.

Weaknesses & Improvements

User 1 pointed out that you must highlight the entire table in either TextView or Leaderboard to actually see the values on the table. This is unintuitive and could easily confuse the user. If I were to do this project again I would look into different ways of displaying the data that do not use dataGridViews, as from what I can tell this issue seems to be a bug with dataGridViews in WinForms.

User 1 also pointed out that I could have implemented a system where a backup of the stock question files is made for use when there is a problem with the main question files, instead of making the user remake the files or reinstall the program when a question file is corrupted or deleted. This is a good idea that I would have implemented if I had thought of it during development. I will implement a similar system in future products that rely on external text files, to make these projects more robust.

When asked what they would like to add or change about my quiz, User 1 said "If I were to add something to the quiz I'd like to add an option to restore defaults on customisable features in order to avoid needing to reinstall the game when a file is missing.", and User 2 said "more customization e.g. being able to change the background colour to something other than purple", these are both fair suggestions that I would have implemented into this project if I had thought of these ideas during development and I will be sure to implement these features in future projects where applicable.

One of the weaknesses of my program that wasn't mentioned in the peer review is that my program could have been better optimised. When switching between forms quickly there will be times when it takes a second for the buttons either move or change image, for example moving from a form back to the lobby may sometimes cause it to show BackBtn64 for a second instead of Logout64.png. These visual oddities do not affect the functionality of the program and do not last more than a second, but if they were eliminated it would help my program look more professional and less sloppy.

Self-Evaluation

General Opinion on Performance

When I started my project I had no experience with C# or with designing and making a GUI for a program, so it was daunting to be tasked with making a fully-fledged quiz game in C# with a graphical user interface. Despite encountering bugs with WinForms which caused UI elements to not show up where I had placed them on the design file, I feel like I have learned a lot about how to make an intuitive user interface using WinForms and I have also learned a lot about C# and object orientated programming in general. I have also gained useful experience in the use of Photoshop to make custom assets for my program and how to organise a file system in a way that makes it easy to find files that you need with my assets folder.

I am happy with what I have done in relation to the development of this program, and I am happy with its outcome.

Coding

This was the area that I was most confident in going into this project. I initially struggled with C#'s syntax, as it is very different from Python which is what I was used to, and classes. However within a couple of weeks I had gotten used to C#'s syntax and I had implemented classes into my program, namely the UserData class to store user account information and the Common_Variables global class, which stores variables that are commonly used in multiple forms (such as Common_Variables.path, which is used to get a relative file path to the assets folder), or variables that affect multiple forms such as Common_Variables.muted.

During development of my program, I used a number of object orientated techniques, such as the classes mentioned before and inheritance, which I used for my custom exception handling in my CreateAccount form. I initially struggled to understand how the event logic worked, but once it did I used events extensively throughout my program, to move between forms and to update the QuestionUI when a new question was generated.

The QuestionDesigner was the hardest form to code, as there are a lot of aspects of the form that I had to consider, I had to make sure the UI was set up correctly for the question type ensure that all / were removed from the textboxes so that the question could not crash the game, ensure that the preview of the question looked like the question form for that section and ensure that all files saved to the correct places and that all requirements for the question were met.

My lack of knowledge in C# earlier on in the program led to parts of my code being unoptimised, initially none of my forms closed when the user moved off of them, they were hidden instead. This led to my program being unoptimised and using needlessly high amounts of RAM, I later solved this problem by using this.Close(); instead of this.Hide(); which closes the form, saving on system resources.

Overall, I was able to make my program to the best of my ability, despite some initial setbacks due to never using C# or another object orientated language before.

Organisation & Time Management

This was the part of the project that I struggled the most with. My project suffered greatly from scope-creep, which led to me wasting time developing features that would never end up making it into the game due to time constraints, for example, the quiz was originally going to have an achievement system and a special question type in DragAndDrop where the user would assemble a computer by placing components on a motherboard.

Both of these features had to be cut due to time constraints, and the time wasted on developing these features that would never make it into the end product meant that there was less time for me to do the documentation for my project, which led to me falling behind in my write-up. This made me skip certain early sections of my documentation so that I could finish the larger sections of the write-up, namely the Final Designs and Testing, this meant that said smaller sections were neglected until the end, which particularly affected my User Requirements, as they were made after my designs were finished which caused them to become disjointed and more focused towards justifying features that I had already made, instead of outlining features that I would make. These time pressures also cause undue amounts of stress, which may have affected the quality of my documentation. In future projects, I will manage my time better and cut features that won't be finished in time earlier on, instead of wasting more time developing them like I did in this project.

Final Thoughts

In conclusion, I have built a quiz program based on computers that meets or partially meets all of the user requirements set out in Page 6. The program is robust, follows a design language throughout and was well received by my peers, with them saying that the quiz was fun and engaging. When I started this project I did not know how to use C# or object orientated programming in general. I also did not know much about how to design a good GUI but during the making of this project I learned a lot about C# and object orientated techniques, such as classes, objects, inheritance and event driven design. I also learned a lot about how to design an intuitive GUI that is aesthetically pleasing. Although there are some problems with my final program, like how it could be better optimised and how there is no back up system for the question files, I am very happy with how my program turned out.